LAPORAN PROPOSAL
PRAKTIKUM IOT - PRAKTIKUM PERJAR - TEKNOLOGI SERVER

SWIFT "Smart Water Integrated Flow and Turbidity Technology" Sistem Manajemen Air Otomatis Berbasis IoT: Monitoring, Analisis, Quality Control, dan Deteksi Anomali Debit Air di Lingkungan DTEDI SV UGM



Disusun oleh:

Azura Rizky Annisa	(24/535715/SV/24290)
Farhan Nur	(24/543276/SV/25180)
Noni Cindy Klaudia M	(24/545671/SV/25729)
Dafa Novran Hasjrullah	(24/541835/SV/24961)

Dosen Pengampu:

Achmad Solaeman, S.Tr.T., M.Eng.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA INTERNET

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

DAFTAR ISI

DAFTAR ISI 2
DAFTAR GAMBAR 6
DAFTAR TABEL 10
DAFTAR LAMPIRAN 12
BAB I PENDAHULUAN 13
1.1 Pemilihan Judul Proyek 13
1.2 Latar Belakang 13
1.3 Rumusan Masalah 15
1.4 Tujuan 15
1.5 Manfaat 16
a. Manfaat bagi Pengelola Fasilitas DTEDI SV UGM 16
b. Manfaat bagi Efisiensi Penggunaan Air 16
c. Manfaat bagi Keamanan dan Kualitas Air 16
d. Manfaat bagi Pengembangan Teknologi IoT 16
1.6 Batasan Masalah 17
BAB II TINJAUAN PUSTAKA 19
2.1 Landasan Teori Terkait 19
2.1.1 Internet of Things (IoT) 19
2.1.2 ESP32 19
2.1.3 Sensor Water Flow 19
2.1.4 Sensor Turbidity 19
2.1.5 Sensor Ultrasonik HC-SR04 19
2.1.6 Protokol MQTT (*Message Queuing Telemetry Transport*) 20
2.1.7 Komunikasi Wi-Fi IEEE 802.11 b/g/n 20
2.1.8 Relay Module 20
2.1.9 Buzzer 20
2.1.10 LCD 16x2 dengan Modul I2C 20
2.1.11 Pengukuran Debit Air 21
2.1.12 Pengukuran Tingkat Kekeruhan Air (NTU) 21
2.2 Metodologi Internet of Things (IoT) 22

2.3 Sensor dan Akuatur yang Digunakan 23
2.4 Platform IoT yang Digunakan 25
2.5 Studi Literatur Proyek Serupa 26
BAB III PERANCANGAN SISTEM 28
3.1 Desain Sistem IoT 28
3.1.1 Diagram Blok 28
3.1.2 Arsitektur Sistem 29
3.2 Alat dan Bahan 30
3.3 Diagram Alur Kerja (Flowchart) 32
3.4 Desain Perangkat Keras (Hardware) 34
3.5 Desain Perangkat Lunak 36
3.5.1 Algoritma Utama 36
3.5.2 Bahasa Pemrograman 37
1. Bahasa Pemrograman C/C++ pada ESP32 37
2. Bahasa Pemrograman HTML, CSS, dan JavaScript 38
3.6 Desain Dashboard 38
3.6.1 Tampilan Login Dashboard SWIFT 38
3.6.2 Halaman Dashboard saat uji coba alat IoT-nya secara realtime 39
BAB IV IMPLEMENTASI DAN PENGUJIAN 43
4.1 Implementasi Rangkaian dan Program 43
4.2 Konfigurasi Platform IoT 47
4.3 Pengujian Sistem 48
4.3.1 Pengujian Sensor 48
4.3.2 Pengujian Jaringan 54
4.3.3 Pengujian Kinerja Aplikasi 56
4.4 Hasil Tabel Data dari pengujian 58
4.5 Analisis Hasil pengujian 66
BAB V PERANCANGAN JARINGAN SWIFT 67
5.1 Analisis Bisnis 67
5.2 Analisis Kendala 68
5.3 Topologi dan Skalabilitas 69
5.3.1 Topologi Jaringan SWIFT 69

5.3.2 Inventaris Perangkat 70
5.3.3 Koneksi Perangkat IoT 72
5.3.4 Pembagian VLAN 73
5.3.5 Skalabilitas 75
5.3.6 Pengalamatan IP 76
5.3.7 Perutean 77
5.3.8 Konfigurasi VLAN pada Switch (SW-SWIFT) 78
5.3.9 Konfigurasi Router-on-a-Stick pada R-SWIFT 80
5.3.10 Konfigurasi PC-ADMIN PC 82
5.3.11 Konfigurasi Server-SRV SWIFT 83
5.3.12 Konfigurasi Access Point-AP SWIFT 84
5.3.13 Konfigurasi Smartphone 85
5.3.14 Konfigurasi ESP32 87
5.4 Desain Keamaan 90
5.4.1 Konfigurasi Access Control List (ACL) 90
5.4.2 Konfigurasi Port Security 92
5.5 Simulasi dan Dokumentasi 93
5.5.1 Test VLAN 93
5.5.2 Test ROUTING 93
5.5.3 Test Ping PC ADMIN ke Router 94
5.5.4 Test Ping PC ADMIN ke Server 94
5.5.5 Test Ping User 01 ke Server (Jika ACL aktif, seharusnya Request Timed Out) 95
5.5.6 Test Ping User 01 dan 02 ke Access Point 95
5.5.7 Test Ping PC Admin ke User 01 96
5.5.8 Test Ping PC Admin ke User 02 96
5.5.9 Test Ping PC Admin ke ESP32 97
5.5.10 Test Ping User 01 ke ESP32 97
5.5.11 Test Ping User 02 ke ESP32 98
5.5.12 Test Ping User 02 ke ESP32 98
BAB VI PENUTUP 100
6.1 Kesimpulan 100
6.2 Saran 100

LAMPIRAN 104
A. Listing Kode Program 104
B. Gambar pendukung 110
C. Hasil pengukuran 114
D. Dokumentasi foto alat atau tampilan antarmuka 116

DAFTAR GAMBAR

Gambar 3.1.1. Diagram Blok SWIFT 28

Gambar 3.1.2. Arsitektur SWIFT di DTEDI SV UGM 29

Gambar 3.3.1. Flowchart Sistem IoT SWIFT 32

Gambar 3.4.1. Rancangan Hardware IoT 34

Gambar 3.6.1. Tampilan Login Dashboard SWIFT 38

Gambar 3.6.2. Halaman Dashboard saat uji coba alat IoT-nya secara realtime 39

Gambar 4.1.1. Ilustrasi proyek SWIFT 43

Gambar 4.1.2. Hasil implementasi lcd, sensor ultrasonic, water flow, and turbidity 43

Gambar 4.3.1.1. Pengujian Sensor Ultrasonik 48

Gambar 4.3.1.2. Pengujian Sensor Turbidity saat kondisi aman (air jernih) 49

Gambar 4.3.1.3. Pengujian Sensor Turbidity saat kondisi bahaya (air kotor) 50

Gambar 4.3.1.4. Pengujian Sensor Water Flow saat dialiri air 51

Gambar 4.3.1.5. Pemantauan ketika dua logika water pump on atau off 52

Gambar 4.3.1.6. Pemantauan ketika water pump satu on atau dua off 53

Gambar 4.3.2.1. LCD melakukan konek ke wifi kampus 54

Gambar 4.3.2.2. LCD telah berhasil login ke wifi kampus 55

Gambar 4.3.2.3. Berhasil login ke web ip 10.33.102.177 56

Gambar 4.3.3.1. Login ke web SWIFT 10.33.102.177 56

Gambar 4.3.3.2. Halaman web SWIFT yang menampilkan beberapa data sensor dan akutor 57

Gambar 4.3.3.3. Halaman web SWIFT yang menampilkan data log secara realtime 58

Gambar 5.3.1.1. Topologi Jaringan SWIFT 69

Gambar 5.3.8.1. Konfigurasi VLAN pada Switch 78

Gambar 5.3.8.2. Konfigurasi Port pada Router 78

Gambar 5.3.8.3. Mengaktifkan Interface vlan 20 pada Router 81

Gambar 5.3.8.4. Konfigurasi Port pada SERVER 79

Gambar 5.3.8.5. Konfigurasi Port pada Access Point 79

Gambar 5.3.8.6. Konfigurasi Port pada ESP32 79

Gambar 5.3.8.7. Hasil Konfigurasi VLAN 79

Gambar 5.3.9.1. Mengaktifkan Interface utama pada Router 80

Gambar 5.3.9.2. Mengaktifkan Interface vlan 10 pada Router 80

Gambar 5.3.9.3. Mengaktifkan Interface vlan 20 pada Router 81

Gambar 5.3.9.4. Mengaktifkan Interface vlan 30 pada Router 81

Gambar 5.3.9.5. Mengaktifkan Interface vlan 40 pada Router 81

Gambar 5.3.9.6. Hasil Konfigurasi Router-on-a-Stick pada R-SWIFT 81

Gambar 5.3.10.1. Konfigurasi PC ADMIN 82

Gambar 5.3.11.1. 1Konfigurasi Server SWIFT 83

Gambar 5.3.12.1. Konfigurasi AP SWIF 84

Gambar 5.3.13.1. Konfigurasi User 01 pada Smartphone 85

Gambar 5.3.13.2. Konfigurasi User 02 pada Smartphone 85

Gambar 5.3.14.1. Konfigurasi Network ESP32-SWIF 87

Gambar 5.3.14.2. Program ESP32-SWIFT 89

Gambar 5.4.1.1. Konfigurasi pembuatan ACL pada keamanan jaringan SWIFT 90

Gambar 5.4.1.2. Konfigurasi pemasangan ACL pada keamanan jaringan SWIFT 90

Gambar 5.4.1.3. Konfigurasi ACL pada keamanan jaringan SWIFT 90

Gambar 5.4.2.1. Konfigurasi Port Security pada PC ADMIN 92

Gambar 5.4.2.2. Hasil Konfigurasi Port Security pada PC ADMIN 92

Gambar 5.5.1.1. Hasil Konfigurasi VLAN..... 93

Gambar 5.5.2.1. Hasil Konfigurasi Routing 93

Gambar 5.5.3.1. Hasil Ping PC ADMIN ke Router 94

Gambar 5.5.4.1. Hasil Ping PC ADMIN ke Server..... 94

Gambar 5.5.5.1. Hasil Ping User 01 ke Server..... 95

Gambar 5.5.6.1. Hasil User 01 dan User 02 berhasil terhubung ke AP 95

Gambar 5.5.7.1. Hasil Ping PC Admin ke User 01 96

Gambar 5.5.8.1. Hasil Ping PC Admin ke User 02 96

Gambar 5.5.9.1. Hasil Ping PC Admin ke ESP32 97

Gambar 5.5.10.1. Hasil Ping User 01 ke ESP32 97

Gambar 5.5.11.1. Hasil Ping User 02 ke ESP32 98

Gambar 5.5.12.1. Hasil Serial output ESP32..... 98

DAFTAR TABEL

Tabel 2.3.1. Daftar Komponen Perangkat Keras Sistem SWIFT 23

Tabel 2.4.1. Pembagian Peran Tiap Mata Kuliah dalam Sistem SWIFT 25

Tabel 3.5.1.1. Logika Kontrol dan Respons Sistem Berdasarkan Kondisi Sensor 36

Tabel 3.5.1.2. Tabel Kondisi Logika SWIFT 37

Tabel 4.1.1. Konfigurasi Pin ESP32 dan Komponen 44

Tabel 4.4.1. Data Sampel Pemantauan MQTT (25 Record dari 590 Total Record) 60

Tabel 4.4.2. Ringkasan Statistik Turbidity per Kelompok 50 Record 62

Tabel 4.4.3. Distribusi Frekuensi Nilai Turbidity Berdasarkan Kategori Sistem 63

Tabel 4.4.4 Sampel data dari pengujian tanggal 13 Juni 2026 64

Tabel 5.3.2.1. Perangkat Jaringan SWIFT 70

Tabel 5.3.2.2. Perangkat IoT SWIFT 71

Tabel 5.3.3.1. Koneksi Perangkat Jaringan SWIFT 72

Tabel 5.3.3.2. Koneksi Perangkat IoT SWIFT 73

Tabel 5.3.4.1. VLAN pada Perancangan Jaringan SWIFT 73

Tabel 5.3.5.1. Skalabilitas pada Perancangan Jaringan SWIFT 75

Tabel 5.3.6.1. Pengalamatan IP pada Perancangan Jaringan SWIFT 76

Tabel 5.3.6.2. Pengalamatan IP pada IoT SWIFT 76

DAFTAR LAMPIRAN

Gambar Kode Final SWIFT 109

Gambar Pendukung Demo IoT 13 Juni 2026..... 113

Gambar Hasil Pengukuran pada Demo Iot 29 Mei 2026..... 115

Gambar Hasil Data Log pada Dashboard SWIFT 117

Gambar Dokumentasi Alat Iot Final 116

BAB I
PENDAHULUAN

1.1 Pemilihan Judul Proyek

SWIFT “Smart Water Integrated Flow and Turbidity Technology” Sistem Manajemen Air Otomatis Berbasis IoT: Monitoring, Analisis, Kontrol Kualitas, dan Deteksi Anomali Debit Air di Lingkungan DTEDI, Sekolah Vokasi, Universitas Gadjah Mada.

1.2 Latar Belakang

Seiring berkembangnya teknologi di era Industri 4.0, penerapan sistem berbasis Internet of Things (IoT) mulai banyak digunakan untuk membantu pengelolaan berbagai sumber daya, termasuk air. Dalam konteks lingkungan kampus terpadu, penggunaan air merupakan salah satu aspek penting yang mendukung berbagai aktivitas, seperti fasilitas ibadah, sanitasi, laboratorium, dan kebutuhan operasional lainny. Idealnya, pengelolaan air tidak hanya berfokus pada ketersediaan, tetapi juga mencakup pemantauan penggunaan dan kualitas air secara real-time agar dapat meningkatkan efisiensi serta menjamin kelayakan penggunaannya.

Pengelolaan air bersih di lingkungan kampus merupakan kebutuhan operasional yang kritis namun sering kali terabaikan dari sisi manajemen data. Di Gedung Herman Yohanes, DTEDI, Sekolah Vokasi UGM, terdapat sejumlah titik penggunaan air, seperti toilet, wastafel, dan area selasar yang beroperasi setiap hari dengan volume konsumsi yang cukup signifikan. Ironisnya, hingga saat ini tidak terdapat sistem yang mampu memantau, menganalisis, dan mengendalikan penggunaan air tersebut secara digital dan real-time.

Berbagai penelitian membuktikan efektivitas sistem manajemen air berbasis IoT. Vinchi et al. (2025) mengimplementasikan pemantauan debit air menggunakan sensor water flow terintegrasi ESP32, dengan deviasi rata-rata pembacaan hanya 0,033–0,1 liter terhadap pengukuran manual [7]. Wahid dan Noman (2020) menunjukkan bahwa integrasi monitoring kualitas dan kuantitas dalam satu sistem IoT mendukung pengambilan keputusan yang lebih cepat dan berbasis data [6]. Yasin et al. (2021) menyimpulkan bahwa

sistem IoT berbasis server mandiri mampu menyediakan monitoring dan kontrol sumber daya air dari jarak jauh dengan biaya terjangkau [3].

Sistem SWIFT hadir untuk menjawab kesenjangan ini dengan membangun bukan sekadar alat monitoring, melainkan sebuah Sistem Manajemen Air Otomatis yang mampu: (1) memonitor debit dan kualitas air secara real-time, (2) menganalisis pola penggunaan untuk mendeteksi anomali dan kebocoran, (3) mengendalikan aliran secara otomatis berdasarkan kondisi sensor, dan (4) menjalankan quality control aktif termasuk flushing otomatis saat air keruh terdeteksi.

Berangkat dari permasalahan tersebut, proyek ini mengembangkan sistem SWIFT (Smart Water Integrated Flow and Turbidity Technology), yaitu sebuah sistem IoT terintegrasi yang menggabungkan sensor water flow untuk mengukur debit aliran air dan sensor turbidity untuk mengukur tingkat kejernihan air. Sistem ini tidak hanya berfungsi sebagai alat manajemen air, tetapi juga dilengkapi dengan mekanisme klasifikasi kondisi air secara otomatis dan sistem notifikasi berbasis threshold, sehingga pengelola fasilitas dapat segera mengambil tindakan yang tepat berdasarkan informasi yang akurat dan real-time.

Proyek SWIFT merupakan hasil kolaborasi tiga mata kuliah, Praktikum IoT, Praktikum Perancangan Jaringan, dan Teknologi Server yang bersama-sama membangun sistem lengkap mulai dari lapisan perangkat keras (device layer), lapisan komunikasi jaringan (network layer), hingga lapisan pengolahan data dan visualisasi berbasis server mandiri (data center layer). Khusus untuk lapisan server, implementasi menggunakan Virtual Machine (VM) berbasis VirtualBox, sistem operasi Ubuntu Server 22.04 LTS, dan platform orkestrasi container K8s (Kubernetes ringan) sebagai infrastruktur server yang mandiri, terdokumentasi, dan dapat direproduksi.

1.3 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, proyek ini merumuskan permasalahan sebagai berikut:

a. Bagaimana kondisi aktual pengelolaan air di DTEDI dan apa dampak nyata dari tidak adanya sistem monitoring terintegrasi?
b. Bagaimana merancang sistem IoT yang mampu memantau debit (L/min) dan kualitas air (NTU) secara real-time, serta mengklasifikasikannya ke dalam kategori Aman, Waspada, dan Bahaya menggunakan ESP32?
c. Bagaimana membangun mekanisme quality control aktif, bukan sekadar menurunkan debit, tetapi juga mencakup flushing otomatis, penutupan aliran, dan notifikasi pengelola ketika kualitas air melampaui ambang batas bahaya?
d. Bagaimana merancang arsitektur jaringan end-to-end yang andal dari node IoT (ESP32) menuju server VM melalui protokol MQTT di jaringan Wi-Fi kampus?
e. Bagaimana mengimplementasikan server mandiri berbasis VM (VirtualBox + Ubuntu Server 22.04 LTS + K8s) dengan layanan terkontainerisasi Mosquitto, dan SWIFT.html?
f. Bagaimana kinerja keseluruhan sistem SWIFT mencakup akurasi sensor, stabilitas cluster K8s, latensi MQTT, dan responsivitas dashboard dalam kondisi pengujian nyata?

1.4 Tujuan

Proyek SWIFT bertujuan untuk:

a. Mengidentifikasi dan mendokumentasikan kondisi aktual pengelolaan air di DTEDI sebagai justifikasi kebutuhan sistem manajemen air berbasis IoT.
b. Merancang dan mengimplementasikan prototipe sistem IoT terintegrasi yang mampu memanajemen debit dan kualitas air secara real-time menggunakan sensor YF-S201 dan turbidity berbasis ESP32.
c. Membangun mekanisme quality control aktif, yaitu klasifikasi otomatis (Aman/Waspada/Bahaya), flushing otomatis saat kondisi Bahaya, penutupan aliran via water valve, dan notifikasi real-time kepada pengelola.

d. Mendesain arsitektur jaringan end-to-end yang menghubungkan ESP32 dengan server VM melalui protokol MQTT di jaringan Wi-Fi kampus secara andal dan aman.
e. Mengimplementasikan server mandiri berbasis VM dengan empat layanan terkontainerisasi di Kubernetes dan mendokumentasikan seluruh langkah instalasi per-step sebagai referensi teknis yang dapat dipertanggungjawabkan.
f. Mengevaluasi kinerja sistem secara komprehensif dan memvalidasi manfaat nyata sistem terhadap efisiensi pengelolaan air di lingkungan kampus.

1.5 Manfaat

Manfaat yang diharapkan dari pengembangan sistem *SWIFT (Smart Water Integrated Flow and Turbidity Technology)* dapat ditinjau dari beberapa aspek, yaitu:

a. Manfaat bagi Pengelola Fasilitas DTEDI SV UGM

Sistem SWIFT membantu pengelola fasilitas dalam melakukan pemantauan debit dan kualitas air secara *real-time*. Informasi kondisi air dapat dipantau langsung melalui dashboard sehingga proses identifikasi kebocoran, penggunaan air berlebih, maupun penurunan kualitas air dapat dilakukan lebih cepat dan akurat.

b. Manfaat bagi Efisiensi Penggunaan Air

Dengan adanya monitoring berbasis Internet of Things (IoT), penggunaan air dapat dianalisis berdasarkan data historis yang tersimpan pada sistem. Hal ini memungkinkan pengelola melakukan evaluasi pola konsumsi air sehingga penggunaan menjadi lebih efisien dan potensi pemborosan dapat dikurangi.

c. Manfaat bagi Keamanan dan Kualitas Air

Sistem mampu melakukan klasifikasi kondisi air berdasarkan parameter debit dan tingkat kekeruhan (NTU). Ketika terdeteksi kondisi tidak normal, sistem dapat memberikan notifikasi peringatan serta melakukan pengendalian otomatis terhadap aktuator seperti pompa dan *solenoid valve* untuk menjaga kualitas distribusi air.

d. Manfaat bagi Pengembangan Teknologi IoT

Proyek SWIFT dapat menjadi implementasi nyata penerapan teknologi Internet of Things pada lingkungan kampus, khususnya dalam pengelolaan sumber daya air.

Sistem ini juga dapat dijadikan dasar pengembangan penelitian lanjutan terkait *smart monitoring system* dan *smart campus* berbasis data.

1.6 Batasan Masalah

a. Industri yang di pilih-Smart Campus & Manajemen Air

Lingkungan yang dipilih adalah kampus terpadu DTEDI Sekolah Vokasi UGM, khususnya Gedung Herman Yohanes. Lingkungan ini dipilih karena memiliki konsumsi air harian yang cukup tinggi karena terdapat perkebunan mini dan banyaknya tanaman bunga di selasar sana, namun belum memiliki sistem monitoring yang memadai. Implementasi mencakup area toilet, wastafel, dan area selasar yang beroperasi setiap hari. Pada prototipe, sistem dipasang pada satu titik aliran utama di lingkungan laboratorium untuk pengujian, dengan desain yang memungkinkan scale-up ke multi-titik tanpa mengubah arsitektur inti.

b. System IoT Sensor dan Aktuator

Ruang lingkup sistem IoT mencakup penggunaan dua sensor utama yang diintegrasikan pada satu mikrokontroler ESP32:

- Sensor Water Flow (YF-S201): dipasang pada jalur pipa untuk mendeteksi debit aliran air, menghasilkan pulsa digital yang dihitung ESP32 untuk menentukan laju aliran dalam satuan liter per menit (L/min).
- Sensor Turbidity mengukur tingkat kejernihan air dengan memancarkan cahaya ke dalam air dan mengukur intensitas hamburan baliknya; nilai yang dihasilkan dikonversi oleh ESP32 menjadi satuan NTU (Nephelometric Turbidity Unit) sebagai indikator kualitas air.
- Selain dua sensor utama, sistem juga mengintegrasikan aktuator berupa solenoid valve 12V yang dikontrol via relay untuk membuka/menutup aliran air secara otomatis, DC water pump 5V untuk sirkulasi air pada prototipe, buzzer aktif sebagai notifikasi lokal, dan LCD 16x2 dengan modul I2C untuk display status secara langsung tanpa koneksi internet. Seluruh komponen dirangkai pada breadboard menggunakan kabel jumper.

Dalam implementasi prototipe, sistem juga memanfaatkan DC water pump sebagai sumber aliran air buatan yang mensimulasikan kondisi penggunaan air pada kran, sehingga pengujian dapat dilakukan dalam skala laboratorium. Seluruh komponen dirangkai pada breadboard dan papan uji menggunakan kabel jumper.

c. Implementasi Rancangan Jaringan

Ruang lingkup jaringan dibatasi pada komunikasi data antara ESP32 dan server VM melalui jaringan Wi-Fi yang tersedia di lingkungan kampus atau hotspot pribadi. ESP32 terhubung ke access point Wi-Fi dan mengirimkan data sensor menggunakan protokol MQTT ke Mosquitto broker yang berjalan di dalam Kubernetes cluster di VM. Rancangan jaringan berfokus pada konektivitas dasar yang stabil dan efisien untuk mendukung pengiriman data sensor secara periodik setiap 5–10 detik. Desain topologi jaringan mencakup analisis kebutuhan bisnis, kebutuhan teknis, constraint analysis, desain VLAN, IP addressing, routing, dan security design, disimulasikan menggunakan Cisco Packet Tracer.

d. Deployment Dashboard

Ruang lingkup server mencakup implementasi infrastruktur server mandiri berbasis Virtual Machine dengan spesifikasi: VirtualBox sebagai hypervisor, Ubuntu Server 22.04 LTS sebagai sistem operasi, dan Kubernetes sebagai platform orkestrasi container. Empat layanan utama di-deploy sebagai Pod dalam Kubernetes: Mosquitto (MQTT Broker) pada port 1883, dan swift.html (Dashboard) pada ip 10.33.102.177 untuk dapat diakses via browser.

- Mosquitto (MQTT Broker)→ menerima data sensor dari ESP32 pada port 31883.
- SWIFT.html (Dashboard)→ menampilkan monitoring real-time dan sistem alert yang dapat diakses via browser.

BAB II
TINJAUAN PUSTAKA

2.1 Landasan Teori Terkait

2.1.1 Internet of Things (IoT)

Internet of Things (IoT) merupakan konsep komunikasi antarperangkat fisik yang saling terhubung melalui jaringan internet untuk melakukan pertukaran data secara otomatis tanpa interaksi manusia secara langsung. Pada sistem SWIFT, konsep IoT diterapkan untuk menghubungkan sensor, mikrokontroler ESP32, jaringan Wi-Fi, MQTT Broker, dan dashboard monitoring sehingga kondisi air dapat dipantau secara *real-time*.

2.1.2 ESP32

ESP32 merupakan mikrokontroler yang memiliki kemampuan pemrosesan data sekaligus konektivitas Wi-Fi terintegrasi. Pada sistem SWIFT, ESP32 berfungsi sebagai pusat kendali (*main controller*) yang membaca data sensor, melakukan klasifikasi kondisi air, mengendalikan aktuator, serta mengirimkan data ke server menggunakan protokol MQTT.

2.1.3 Sensor Water Flow

Sensor *water flow* digunakan untuk mengukur debit aliran air berdasarkan jumlah pulsa yang dihasilkan rotor internal saat dialiri air. Semakin besar aliran air, semakin tinggi frekuensi pulsa yang dihasilkan. Pada sistem SWIFT, nilai pulsa tersebut diolah oleh ESP32 untuk menghasilkan informasi debit air dalam satuan liter per menit (*L/min*).

2.1.4 Sensor Turbidity

Sensor *turbidity* berfungsi untuk mengukur tingkat kekeruhan air berdasarkan intensitas cahaya yang diterima sensor. Semakin tinggi jumlah partikel tersuspensi di dalam air, maka nilai kekeruhan akan semakin besar. Hasil pembacaan sensor dikonversi menjadi satuan *Nephelometric Turbidity Unit* (NTU) untuk menentukan klasifikasi kondisi air.

2.1.5 Sensor Ultrasonik HC-SR04

Sensor ultrasonik HC-SR04 digunakan untuk mengukur jarak atau tinggi permukaan air dengan memanfaatkan pantulan gelombang ultrasonik. Sensor bekerja dengan

mengirimkan sinyal suara frekuensi tinggi yang kemudian dipantulkan kembali oleh permukaan objek. Pada sistem SWIFT, sensor ini digunakan untuk memantau tinggi air pada wadah penyimpanan atau prototipe pengujian.

2.1.6 Protokol MQTT (*Message Queuing Telemetry Transport*)

MQTT merupakan protokol komunikasi ringan berbasis mekanisme *publish-subscribe* yang umum digunakan pada sistem IoT. Pada sistem SWIFT, ESP32 berperan sebagai *publisher* yang mengirimkan data sensor ke MQTT Broker, kemudian dashboard SWIFT menerima data tersebut untuk divisualisasikan secara *real-time*. MQTT dipilih karena ringan, efisien, dan mampu bekerja baik pada jaringan dengan bandwidth terbatas.

2.1.7 Komunikasi Wi-Fi IEEE 802.11 b/g/n

Wi-Fi merupakan teknologi komunikasi nirkabel yang memungkinkan perangkat terhubung ke jaringan internet. Pada sistem SWIFT, modul Wi-Fi bawaan ESP32 digunakan untuk menghubungkan perangkat ke jaringan kampus sehingga data sensor dapat dikirim menuju MQTT Broker dan dashboard monitoring tanpa memerlukan media kabel.

2.1.8 Relay Module

Relay module merupakan komponen elektronik yang berfungsi sebagai saklar otomatis untuk mengendalikan perangkat dengan kebutuhan daya lebih besar. Pada sistem SWIFT, relay digunakan untuk mengontrol pompa air dan *solenoid valve* berdasarkan hasil klasifikasi kondisi air yang diproses oleh ESP32.

2.1.9 Buzzer

Buzzer merupakan perangkat output suara yang digunakan sebagai notifikasi lokal ketika sistem mendeteksi kondisi tertentu. Pada sistem SWIFT, buzzer diaktifkan ketika kondisi air memasuki status waspada atau bahaya sehingga pengguna dapat segera mengetahui adanya gangguan.

2.1.10 LCD 16x2 dengan Modul I2C

LCD 16x2 digunakan sebagai media tampilan lokal untuk menampilkan informasi penting seperti debit air, tingkat kekeruhan, dan status sistem. Penggunaan modul I2C

memungkinkan komunikasi data dilakukan hanya menggunakan dua jalur utama, yaitu SDA dan SCL, sehingga penggunaan pin ESP32 menjadi lebih efisien.

2.1.11 Pengukuran Debit Air

Debit air merupakan jumlah volume air yang mengalir dalam satuan waktu tertentu dan umumnya dinyatakan dalam liter per menit (*L/min*). Pengukuran debit pada sistem SWIFT dilakukan menggunakan sensor *water flow* untuk mengetahui pola penggunaan air serta mendeteksi potensi kebocoran.

2.1.12 Pengukuran Tingkat Kekeruhan Air (NTU)

Tingkat kekeruhan air menunjukkan jumlah partikel tersuspensi yang terdapat di dalam air. Semakin tinggi nilai NTU, maka kualitas kejernihan air semakin menurun. Pada sistem SWIFT, parameter NTU digunakan sebagai dasar klasifikasi kondisi air menjadi aman, waspada, atau bahaya.

2.2 Metodologi Internet of Things (IoT)

Sistem SWIFT dirancang dengan pendekatan problem-first, dimulai dari observasi kondisi lapangan DTEDI, kemudian dirumuskan masalah spesifik, baru kemudian dipilih teknologi IoT yang paling tepat. Metodologi pelaksanaan proyek SWIFT terdiri dari enam tahap utama yang berjalan secara terstruktur:

1. Observasi dan analisis lapangan kondisi pengelolaan air di DTEDI Gedung Herman Yohanes.
2. Perumusan kebutuhan sistem mencakup kebutuhan IoT, kebutuhan jaringan, dan kebutuhan server secara terintegrasi.
3. Perancangan arsitektur sistem berlapis tiga, yaitu Device Layer, Network Layer, dan Data Center Layer.
4. Implementasi per layer, yaitu perakitan hardware IoT, konfigurasi jaringan dan simulasi Cisco Packet Tracer, serta instalasi server VM dan deployment Kubernetes.
5. Integrasi end-to-end, yaitu pengujian konektivitas ESP32 → MQTT → SWIFT.html.
6. Evaluasi dan pengukuran kinerja sistem berdasarkan indikator keberhasilan kuantitatif.

2.3 Sensor dan Akuatur yang Digunakan

Tabel 2.3.1 berikut merangkum seluruh komponen perangkat keras yang digunakan dalam sistem SWIFT, beserta fungsi dan jumlahnya:

Tabel 2.3.1. Daftar Komponen Perangkat Keras Sistem SWIFT

Komponen	Jenis	Fungsi	Jumlah
Water Flow Sensor (YF-S401)	Sensor	Mengukur debit dan volume aliran air secara real-time melalui pulsa digital 1. Working range: 0.3-6L/min-2. Water pressure: ≤0.8 MPa	1 unit
Turbidity Sensor	Sensor	Mengukur tingkat kekeruhan air berdasarkan hamburan Cahaya, outputnya NTU	1 unit
Sensor Ultrasonik HC-SR04	Sensor	Mengukur ketinggian air dengan memancarkan gelombang ultrasonic kea rah permukaan air dan menangkap pantulannya untuk menentukan jarak antara sensor dan air, data jarak yang didapatkan dioleh oleh mikrokontroler esp32 untuk mendapatkan ketinggiann air yang akurat secara real-time	1 unit
LGZD Sensor v1.1	Driver/Amplifier	Mengubah sinyal optik menjadi sinyal elektrik yang	1 unit

		dapat dibaca oleh mikrokontroler.	
Mikrokontroler ESP32(ISM 2.4G 802.11/b/g/n-FCC ID:2BB77-ESP32-X)	Mikrokontroler	Unit pemroses utama: membaca sensor, mengolah data, mengirim via Wi-Fi MQTT	1 unit
DC Water Pump Mini 5V	Aktuator	Mensimulasikan aliran air dalam prototipe laboratorium	2 unit
Water Valve DC 12V (Model YFPDJ-23, VOLTAGE DC 12V, J-20250829)	Aktuator	Mengatur buka/tutup aliran air secara elektrik berdasarkan perintah sistem	1 unit
Relay Module 5V(10A 250VAC-10A 125VAC, 10A 30VDC-10A 28VDC, SRD-05VDC-SL-C)	Aktuator	Saklar elektronik untuk mengontrol pompa dan solenoid valve dari ESP32	1 unit
LCD 16x2 + I2C Module	Display	Menampilkan data debit dan status air secara lokal tanpa perlu smartphone	1 unit
Buzzer Aktif 5V	Aktuator/Output Suara	Memberikan notifikasi suara untuk kondisi air Aman/Waspada/ Bahaya atau sistem bermasalah	1 unit

Kabel Jumper	Pendukung	Menghubungkan antar komponen pada breadboard/papan uji	Secukupnya
Breadboard / Papan Uji-MB102-830	Pendukung	Kerangka perakitan komponen sementara tanpa solder untuk protopipe	1 buah
Adaptor 12V DC & Adaptor 5 V DC	Catu Daya	Sumber daya pompa dan Water Valve	1 unit
Resistor 1kΩ & 2kΩ	Pendukung	Voltage divider untuk melindungi pin GPIO ESP32 dari sinyal 5V sensor	21kΩ (2) & 2kΩ(2)

2.4 Platform IoT yang Digunakan

Tabel 2.4.1 berikut menjelaskan platform IoT yang digunakan dan pembagian peran antar tiga mata kuliah yang berkolaborasi dalam membangun sistem SWIFT:

Tabel 2.4.1. Pembagian Peran Tiap Mata Kuliah dalam Sistem SWIFT

Mata Kuliah	Layer	Komponen	Tugas Utama
Praktikum IoT	Device Layer	ESP32, Water Flow Sensor (YF-S201), Turbidity Sensor, DC Water Pump, Water Valve, Relay Module, Buzzer, LCD 16x2	Wiring perangkat keras, pemrograman firmware Arduino IDE, logika kontrol otomatis (threshold NTU + debit), pengiriman data MQTT ke broker lokal (VM)
Perancangan Jaringan	Network Layer	Wi-Fi Router / Hotspot, akses jaringan lokal kampus (DTEDI), VM	Desain topologi jaringan, Analisis kebutuhan teknis & bisnis, VLAN, IP,

		dengan mode Bridged Adapter	Addressing, Routing, konfigurasi konektivitas ESP32 ke VM, simulasi Cisco Packet Tracer
Teknologi Server	Data Center Layer	VM (VirtualBox) + Ubuntu Server 22.04 + K8s Cluster: Mosquitto (MQTT Broker), SWIFT.html (Dashboard)	Instalasi & konfigurasi VM + Ubuntu, deployment cluster K8S, setup MQTT broker (Mosquitto), visualisasi dashboard SWIFT.html, dokumentasi per-step implementasi server

2.5 Studi Literatur Proyek Serupa

Beberapa penelitian terdahulu telah membahas penerapan Internet of Things (IoT) pada sistem monitoring air. Penelitian oleh Vinchi et al. (2025) mengembangkan sistem pemantauan debit air berbasis IoT menggunakan sensor *water flow* untuk memonitor penggunaan air secara *real-time*. Hasil penelitian menunjukkan bahwa sistem mampu membantu identifikasi pola penggunaan air dan mendukung pengelolaan distribusi air yang lebih efisien.

Penelitian lain oleh Wahid dan Noman (2020) mengembangkan sistem monitoring kualitas dan kuantitas air berbasis IoT dengan memanfaatkan sensor kualitas air dan komunikasi jaringan untuk pemantauan jarak jauh. Sistem tersebut menunjukkan bahwa integrasi sensor dan jaringan internet dapat meningkatkan efektivitas pengawasan kondisi air secara berkelanjutan.

Selain itu, penelitian Nugroho et al. (2025) memanfaatkan sensor ultrasonik dan ESP32 untuk melakukan monitoring parameter air berbasis web. Hasil penelitian menunjukkan bahwa pengiriman data secara *real-time* mempermudah proses analisis kondisi air dan pengambilan keputusan.

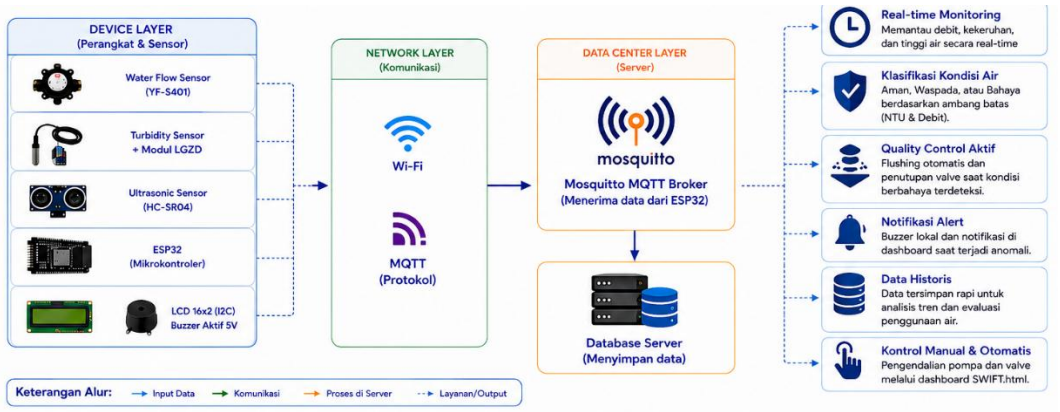
Berdasarkan beberapa penelitian tersebut, proyek SWIFT dikembangkan sebagai sistem manajemen air terintegrasi yang tidak hanya melakukan monitoring debit dan kualitas air, tetapi juga menyediakan mekanisme klasifikasi kondisi air, notifikasi peringatan, serta pengendalian aktuator secara otomatis melalui dashboard monitoring berbasis web.

BAB III
PERANCANGAN SISTEM

3.1 Desain Sistem IoT

3.1.1 Diagram Blok

Arsitektur sistem SWIFT dirancang berbasis loT dengan pendekatan 3-layer architecture, yang terdiri dari Device Layer, Network Layer, dan Data Center Layer. Sistem ini memungkinkan monitoring, analisis, dan kontrol kualitas air secara real-time serta deteksi anomali debit air di lingkungan DTEDI SV UGM.



Gambar 3.1.1. Diagram Blok SWIFT

1. Device Layer (Perangkat & Sensor)

Berperan sebagai pengumpul data dari lingkungan air melalui sensor-sensor yang terhubung ke ESP32. Data ditampilkan secara lokal pada LCD dan buzzer sebagai peringatan awal.

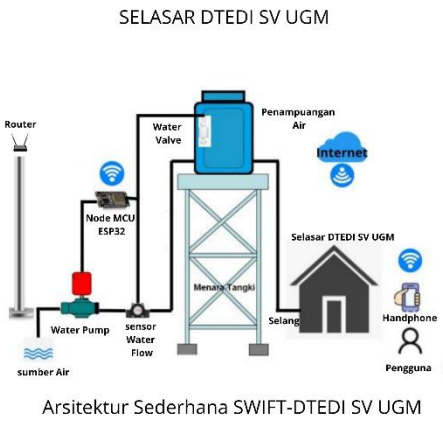
2. Network Layer (Komunikasi)

Data dari ESP32 dikirim melalui Wi-Fi menggunakan protokol MQTT untuk memastikan pengiriman data yang ringan, efisien, dan andal.

3. Data Center Layer (Server)

MosquittoMQTT Broker menerima dan mengelola data sebelum disimpan ke database. Sistem kemudian mengolah data untuk monitoring, kontrol otomatis, notifikasi, dan penyimpanan historis.

3.1.2 Arsitektur Sistem



Gambar 3.1.2. Arsitektur SWIFT di DTEDI SV UGM

Gambar 3.1.2 menunjukkan arsitektur terintegrasi sistem SWIFT (*Smart Water Integrated Flow and Turbidity Technology*) yang diterapkan di lingkungan DTEDI Sekolah Vokasi UGM. Arsitektur ini menggunakan pendekatan IoT end-to-end architecture, yaitu integrasi menyeluruh mulai dari lapisan perangkat fisik (*device layer*), jaringan komunikasi (*network layer*), hingga pusat pengolahan data (*data center layer*).

Pada device layer, sistem terdiri atas sensor water flow, sensor turbidity, sensor ultrasonik, relay module, pompa air, water valve, LCD, buzzer, dan mikrokontroler ESP32. Sensor berfungsi melakukan monitoring kondisi air secara real-time, sedangkan aktuator digunakan untuk melakukan tindakan otomatis terhadap aliran air.

Pada network layer, ESP32 terhubung ke jaringan Wi-Fi kampus atau hotspot dan mengirimkan data menggunakan protokol MQTT (Message Queuing Telemetry Transport). Protokol ini dipilih karena ringan, efisien, serta sesuai untuk komunikasi perangkat IoT dengan keterbatasan bandwidth dan daya komputasi.

Selanjutnya pada data center layer, data diterima oleh Mosquitto MQTT Broker yang berjalan pada server Virtual Machine berbasis Ubuntu Server dan Kubernetes. Data kemudian diproses dan divisualisasikan pada dashboard monitoring SWIFT sehingga pengelola fasilitas dapat memantau debit, kualitas air, status valve, notifikasi anomali, dan histori penggunaan air secara real-time melalui browser.

Arsitektur ini memungkinkan sistem SWIFT tidak hanya berfungsi sebagai alat monitoring, tetapi juga sebagai sistem manajemen air otomatis yang mampu melakukan analisis, quality control, dan pengambilan tindakan secara mandiri berdasarkan kondisi aktual di lapangan.

3.2 Alat dan Bahan

Berdasarkan hasil observasi awal di lingkungan toilet dan area selasar DTEDI, ditemukan bahwa penggunaan air berlangsung setiap hari dalam volume cukup besar, namun pemantauan terhadap debit maupun kualitas kejernihan air belum dilakukan secara digital. Pengelola fasilitas belum memiliki data historis yang akurat mengenai pola konsumsi air, sehingga deteksi kebocoran atau penurunan kualitas air baru diketahui setelah ada laporan manual dari pengguna. Kondisi ini sesuai dengan yang diidentifikasi oleh Vinchi et al. (2025), yang menyatakan bahwa pencatatan pemakaian air secara manual sangat rentan terhadap kesalahan data dan menyulitkan evaluasi berkelanjutan [7].

Berdasarkan analisis tersebut, komponen IoT yang dibutuhkan sistem SWIFT mencakup sensor water flow untuk mengukur debit aliran air, sensor turbidity untuk mengukur tingkat kejernihan, mikrokontroler ESP32 sebagai otak sistem yang mengolah data dari kedua sensor sekaligus mengelola konektivitas Wi-Fi, DC water pump sebagai sumber aliran dalam prototipe, serta solenoid valve sebagai aktuator yang mengatur aliran air berdasarkan kondisi yang terdeteksi.

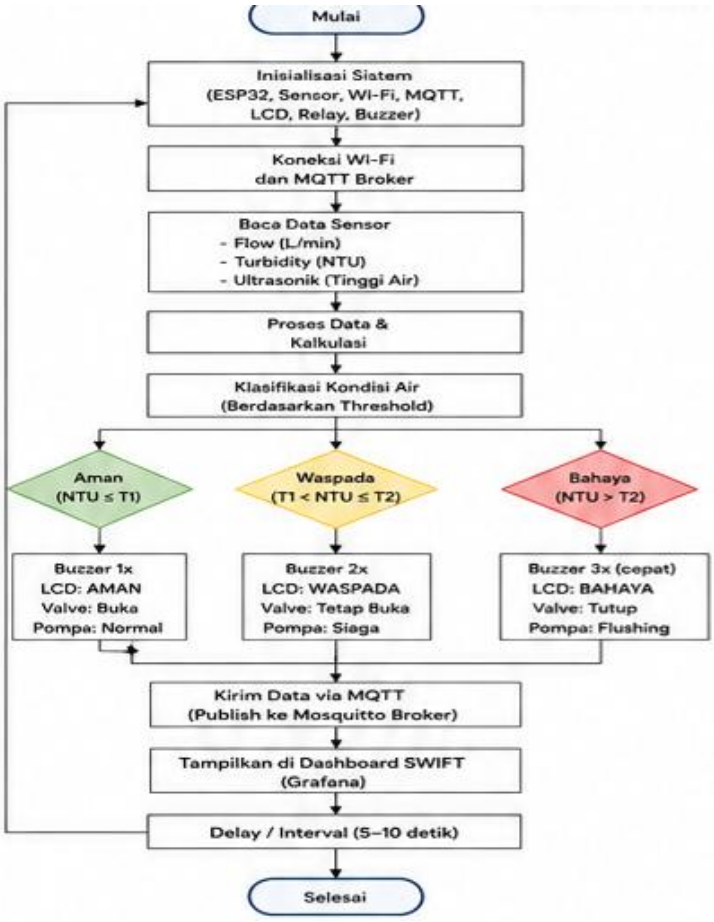
Berdasarkan analisis tersebut, kebutuhan komponen IoT pada sistem SWIFT dirumuskan sebagai berikut:

- Mikrokontroler ESP32: Bertindak sebagai unit pemroses data utama (brain) yang mengolah sinyal dari sensor water flow dan turbidity, menyediakan konektivitas Wi-Fi untuk transmisi data MQTT, serta memiliki kompatibilitas tinggi dengan berbagai platform cloud maupun server lokal.
- Sensor Water Flow (YF-S201): Berfungsi untuk mengukur debit aliran air secara real-time pada setiap titik pipa utama toilet kampus guna memantau volume penggunaan. Sensor menghasilkan pulsa digital yang dihitung ESP32 untuk menentukan laju aliran dalam satuan L/min.

- Sensor Turbidity: Berfungsi untuk mengukur tingkat kejernihan atau kekotoran air berdasarkan jumlah partikel tersuspensi (seperti lumpur, pasir, atau mikroorganisme). Nilai ADC dari sensor dikonversi ke satuan NTU untuk memastikan kualitas air tetap terjaga.
- DC Water Pump: Digunakan dalam lingkungan prototipe/simulasi untuk mensirkulasikan air agar sensor dapat membaca aliran dan tingkat kekeruhan secara konsisten saat pengujian laboratorium.
- Sumber Daya (Adaptor 12V DC): Menggunakan adaptor eksternal yang terpasang secara permanen pada titik instalasi untuk menjamin ketersediaan daya bagi pompa dan solenoid valve. ESP32 tidak dapat menyuplai arus untuk komponen berdaya tinggi secara langsung.
- Kabel Jumper (Set): Kabel pendek berujung pin untuk menghubungkan antar komponen elektronik di atas breadboard. Tersedia dalam tipe male-to-male, male-to-female, dan female-to-female. Memudahkan modifikasi dan debugging rangkaian selama pengembangan prototipe.
- LCD 16x2 + Modul I2C: Layar kristal cair yang menampilkan 16 karakter pada 2 baris. Modul I2C mengkonversi antarmuka menjadi komunikasi 2 kabel (SDA/SCL) sehingga hanya membutuhkan 2 pin GPIO. Menampilkan nilai debit (L/min) dan status kondisi air (AMAN/WASPADA/BAHAYA) secara lokal tanpa memerlukan koneksi internet.
- Buzzer Aktif 5V: Komponen penghasil bunyi dengan osilator internal yang aktif cukup dengan tegangan DC 5V. Diprogram dengan pola bunyi berbeda: 1x = sistem aktif normal, 2x = kondisi waspada, 3x cepat = kondisi bahaya. Dikendalikan via GPIO 25 sebagai sinyal digital ON/OFF.
- Relay Module 5V: Saklar elektromagnetik yang memungkinkan ESP32 (logika 3,3V/5V) mengontrol perangkat berdaya tinggi seperti pompa 12V atau solenoid valve 12V. Tanpa relay, ESP32 tidak dapat langsung mengendalikan komponen bertegangan tinggi karena perbedaan level tegangan yang dapat merusak mikrokontroler.

- Breadboard / Papan Uji: Papan prototipe berlubang dengan strip konduktor internal yang memungkinkan perakitan rangkaian tanpa penyolderan. Digunakan sebagai platform perakitan sementara selama fase pengembangan dan pengujian untuk memudahkan modifikasi dan penggantian komponen.
- Water Valve 12V: Katup elektrik dengan kumparan elektromagnetik yang membuka atau menutup aliran air berdasarkan sinyal listrik. Ketika arus mengalir melalui kumparan, plunger bergerak membuka lubang katup, mengizinkan air mengalir. Dikontrol via relay untuk membuka (AMAN) atau menutup (BAHAYA/keruh) aliran secara otomatis maupun manual melalui dashboard.

3.3 Diagram Alur Kerja (Flowchart)



Gambar 3.3.1. Flowchart Sistem IoT SWIFT

Penjelasan Flowchart Sistem IoT SWIFT

1. Mulai

Sistem diaktifkan dan seluruh perangkat dipersiapkan.

2. Inisialisasi Sistem

ESP32 menginisialisasi sensor, modul Wi-Fi, koneksi MQTT, LCD, relay, dan buzzer.

3. Koneksi Wi-Fi dan MQTT Broker

ESP32 mencoba terhubung ke jaringan Wi-Fi dan broker MQTT. Jika gagal, sistem akan mengulang koneksi sampai berhasil.

4. Baca Data Sensor

ESP32 membaca nilai dari sensor water flow, turbidity, dan ultrasonic secara periodik.

5. Proses Data & Kalkulasi

Data mentah diolah dan dikonversi sesuai satuan, misalnya debit (L/min), NTU, dan tinggi air (cm).

6. Klasifikasi Kondisi Air

Nilai NTU dibandingkan dengan ambang batas:

- Aman (NTU ≤ T1)
- Waspada (T1 < NTU ≤ T2)
- Bahaya (NTU > T2)

7. Aksi Sistem Berdasarkan Kondisi

Pada setiap kondisi, sistem memberikan aksi berbeda:

- Aman: valve terbuka, pompa normal, buxzer bunyi 1x.
- Waspada: valve tetap terbuka, pompa dalarn mode siaga, buzzer bunyi 2x.
- Bahaya: valve tertutup, pompa flushing aktif, buzzer bunyi 3x cepat.

8. Kirim Data via MQTT

Data hasil pembacaan dan status dikirim ke broker Mosquitto menggunakan protokol MQTT.

9. Tampilkan di Dashboard SWIFT

SWIFT.html menampilkan data secara real-time dalam bentuk grafik dan indikator.

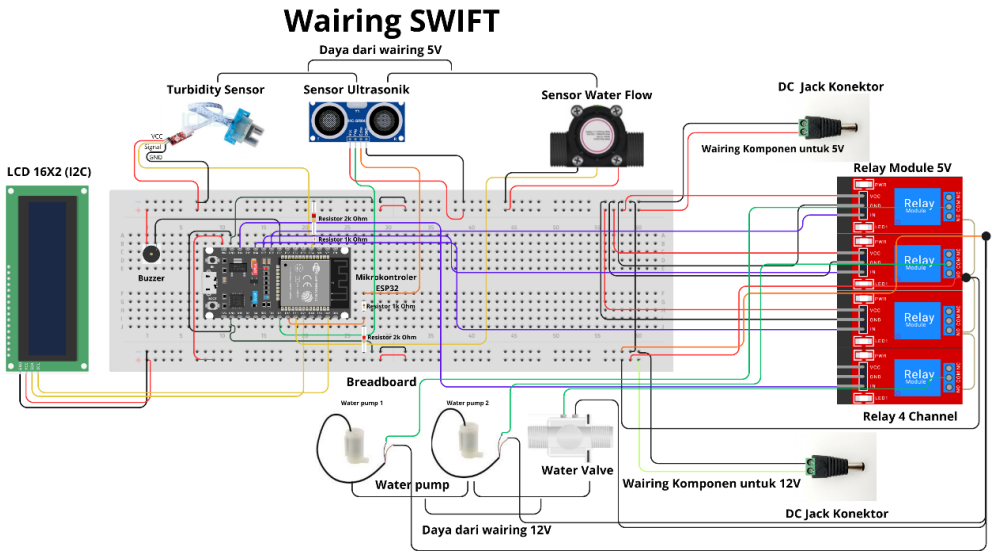
10. Delay / Interval

Sistem melakukan jeda 5-10 detik sebelum membaca data sensor kembali.

11. Selesai / Loop

Proses kembali ke langkah 4 secara berulang untuk monitoring kontinu.

3.4 Desain Perangkat Keras (Hardware)



Gambar 3.4.1. Rancangan Hardware IoT

Desain sistem IoT SWIFT mengacu pada diagram blok yang menggambarkan hubungan terintegrasi antar komponen. Sensor water flow (YF-S201) dipasang langsung pada jalur pipa atau selang aliran air. Sensor bekerja dengan cara memutar rotor internal akibat tekanan aliran air; sensor hall effect yang tertanam di dalamnya

menghasilkan pulsa digital dengan frekuensi sebanding dengan laju aliran. ESP32 menghitung jumlah pulsa per satuan waktu untuk menentukan debit air dalam liter per menit (Vinchi et al., 2025) [7].

Sementara itu, sensor turbidity dipasang berdampingan pada titik yang sama. Sensor ini bekerja dengan memancarkan cahaya inframerah ke dalam air dan mengukur intensitas hamburan baliknya. Semakin banyak partikel tersuspensi dalam air (lumpur, pasir, mikroorganisme), semakin besar hamburan yang terdeteksi, dan semakin tinggi nilai NTU yang dihasilkan. Nugroho et al. (2025) dalam penelitiannya tentang sistem pemantauan debit air berbasis ultrasonik dan IoT menegaskan bahwa pembacaan sensor yang real-time dan terlogging ke cloud memungkinkan analisis pola penggunaan air yang lebih mendalam [11].

Data dari kedua sensor kemudian diproses oleh ESP32 untuk menentukan klasifikasi kondisi air. Sistem menggunakan nilai ambang batas (threshold) sebagai berikut:

- Kondisi Aman: turbidity 0-89 NTU dan debit dalam rentang normal (0,5–5 L/min)
- Kondisi Waspada: turbidity 90–95 NTU atau debit di luar rentang normal
- Kondisi Bahaya: turbidity > 100 NTU atau debit sangat tinggi (indikasi kebocoran)

3.5 Desain Perangkat Lunak

3.5.1 Algoritma Utama

Tabel 3.5.1.1 berikut menjelaskan Algoritma logika kontrol sistem SWIFT berdasarkan kondisi yang terdeteksi sensor:

Tabel 3.5.1.1. Logika Kontrol dan Respons Sistem Berdasarkan Kondisi Sensor

Kondisi Terdeteksi	Sensor Aktif	Respons Sistem	Status
Air jernih, debit normal (0,5–5 L/min)	Turbidity + Flow	Sistem monitoring aktif, data tersimpan periodik yang bisa dipantau melalui dashboard SWIFT, buzzer 1x	AMAN
Air jernih, volume kecil <= 3 cm (air pada wadah penuh)	Turbidity + Flow	Pump ON, notifikasi 'Air Mengalir Normal' di dashboard, buzzer 1x	AMAN
Terjadi lonjakan Debit air tiba-tiba	Water Flow Sensor	Alert 'Lonjakan Debit' ke dashboard, buzzer 2x, logging	⚠ Waspada
Turbidity 90–95 NTU	Turbidity Sensor	Alert 'Air Mulai Keruh', buzzer 2x, notifikasi pengelola	⚠ Waspada
Tidak ada aliran dalam waktu lama	Water Flow Sensor	Notifikasi 'Tidak Ada Aliran'--kemungkinan kebocoran/penyumbatan	⚠ Waspada
Turbidity > 95 NTU (air keruh)	Turbidity Sensor	Solenoid valve TUTUP, buzzer 3x, flushing otomatis 30 detik, re-check	▨ Bahaya

Air sangat keruh & debit tidak normal	Turbidity + Flow	Sistem hentikan aliran, notifikasi darurat pengelola, logging lengkap	Bahaya

Tabel 3.5.1.2. Tabel Kondisi Logika SWIFT

Prioritas	Kondisi	Pompa1	Pompa2	Valve
0	Sensor error (jarak ≥ 98cm)	OFF	OFF	TUTUP
1	NTU > 95 → flushing	OFF	ON	TUTUP
2	NTU 90–95 + flow > 5	OFF	ON	TUTUP
3	NTU 0–89, flow normal	OFF	OFF	BUKA
4	Flow > 5, NTU bersih	OFF	OFF	BUKA
5	Tidak ada aliran 15 detik	OFF	OFF	BUKA
6	Aman + wadah kosong	ON	OFF	BUKA
7	Aman + wadah penuh	OFF	OFF	BUKA
8	Semua normal	OFF	OFF	BUKA

3.5.2 Bahasa Pemrograman

Pada pengembangan sistem SWIFT, bahasa pemrograman digunakan untuk mendukung proses pembacaan sensor, komunikasi data, pengendalian aktuator, serta visualisasi data pada dashboard monitoring. Bahasa pemrograman yang digunakan meliputi bahasa pemrograman pada mikrokontroler dan bahasa pemrograman berbasis web.

1. Bahasa Pemrograman C/C++ pada ESP32

Bahasa pemrograman C/C++ digunakan pada mikrokontroler ESP32 melalui platform Arduino IDE. Bahasa ini digunakan untuk mengendalikan proses pembacaan sensor *water flow*, sensor turbidity, dan sensor ultrasonik, serta melakukan pengolahan data

dan klasifikasi kondisi air. Selain itu, program juga bertugas mengontrol aktuator seperti relay, pompa air, *solenoid valve*, buzzer, dan LCD 16x2.

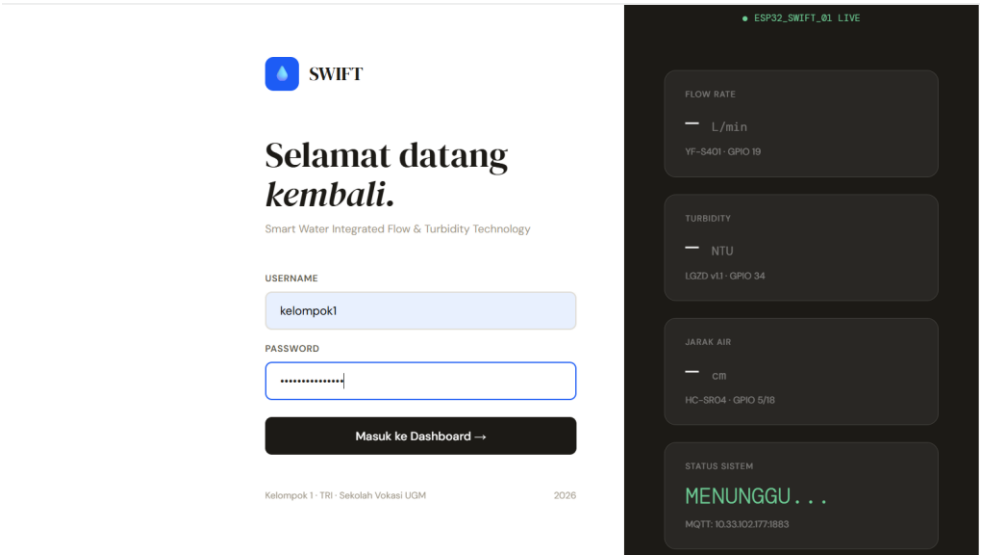
ESP32 juga diprogram untuk mengirimkan data hasil pembacaan sensor ke MQTT Broker menggunakan protokol MQTT melalui jaringan Wi-Fi sehingga data dapat ditampilkan secara *real-time* pada dashboard SWIFT.

2. Bahasa Pemrograman HTML, CSS, dan JavaScript

Dashboard SWIFT dikembangkan menggunakan kombinasi HTML, CSS, dan JavaScript. HTML digunakan untuk menyusun struktur tampilan dashboard, CSS digunakan untuk mengatur tata letak dan desain antarmuka, sedangkan JavaScript digunakan untuk memproses data yang diterima dari MQTT Broker dan menampilkan informasi monitoring secara dinamis.

3.6 Desain Dashboard

3.6.1 Tampilan Login Dashboard SWIFT



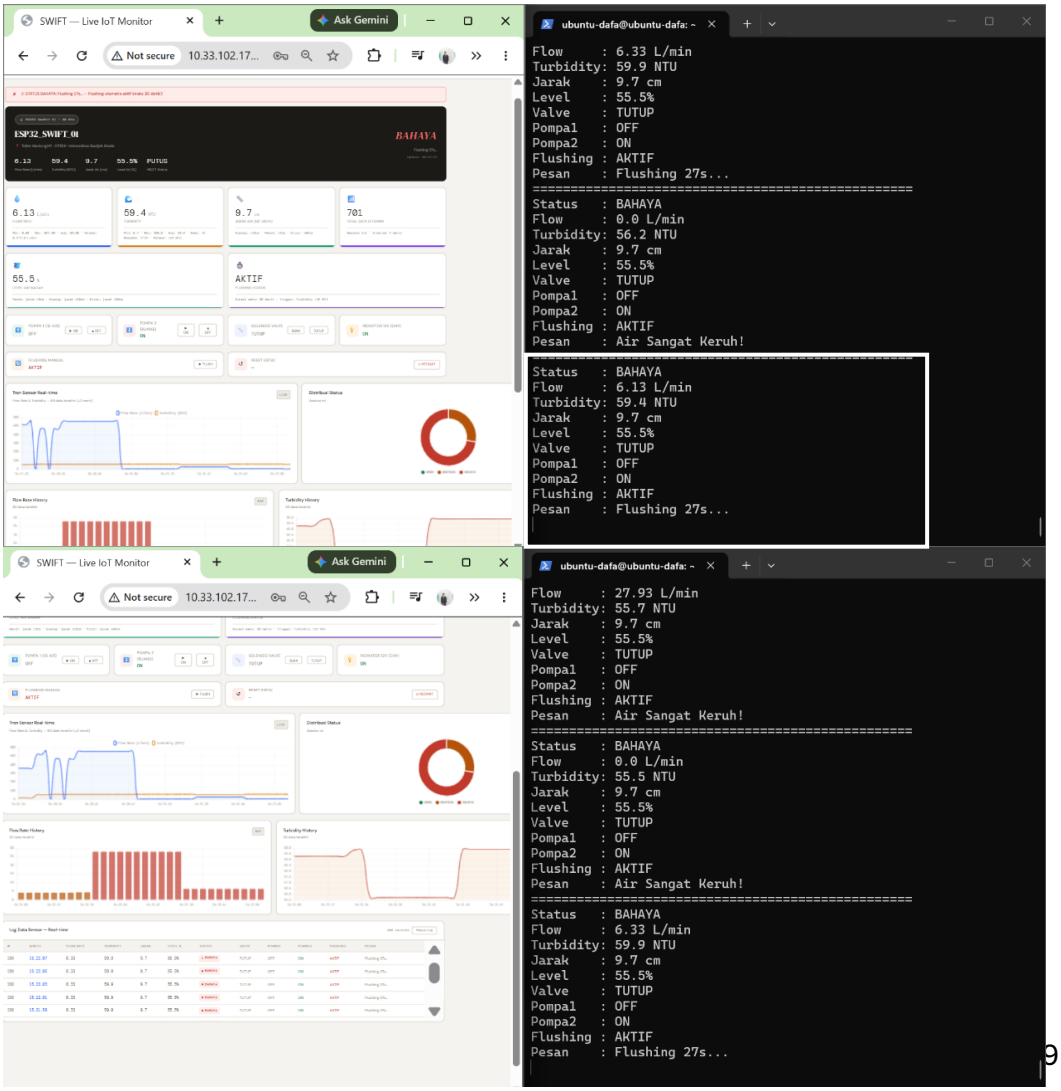
Gambar 3.6.1. Tampilan Login Dashboard SWIFT

Halaman login merupakan gerbang pertama sebelum pengguna dapat mengakses seluruh fitur monitoring pada dashboard SWIFT. Tampilan ini dirancang sederhana namun

fungsional, dengan tujuan menjaga keamanan akses agar hanya pihak yang berwenang yang dapat memantau data sensor secara langsung.

Halaman login dapat diakses melalui browser dengan mengetikkan alamat IP VPS pada kolom URL, yaitu http://10.33.102.177/swift.html. Setelah halaman dimuat, pengguna diminta memasukkan kredensial berupa username dan password. Kredensial yang digunakan pada pengujian ini adalah username “kelompok1” dan password “kelompok112345”. Setelah tombol Masuk ke Dashboard ditekan, sistem memverifikasi kredensial tersebut dan apabila valid, pengguna akan diarahkan langsung ke halaman utama dashboard monitoring real-time. Mekanisme autentikasi ini memastikan bahwa data sensor tidak dapat diakses oleh pihak yang tidak memiliki hak akses, sehingga integritas dan kerahasiaan data sistem SWIFT tetap terjaga.

3.6.2 Halaman Dashboard saat uji coba alat IoT-nya secara realtime



Gambar 3.6.2. Halaman Dashboard saat uji coba alat IoT-nya secara realtime

Setelah berhasil masuk, pengguna akan disambut oleh halaman utama dashboard yang menampilkan data sensor secara langsung dan terus diperbarui setiap beberapa detik. Dashboard ini menjadi pusat kendali visual dari seluruh sistem SWIFT, memungkinkan pengelola fasilitas memantau kondisi air tanpa harus berada di dekat perangkat fisik. Berikut adalah penjelasan dari masing-masing field yang ditampilkan pada dashboard:

1. L/MIN - Flow Rate (Laju Aliran Air)

Field ini menampilkan laju aliran air yang melewati sensor water flow YF-S401 dalam satuan liter per menit (L/min). Nilai ini dihitung berdasarkan frekuensi pulsa yang dihasilkan oleh rotor di dalam sensor, di mana setiap rotasi menghasilkan sejumlah pulsa yang kemudian dikonversi oleh ESP32 menjadi nilai debit. Pada kondisi normal, nilai flow rate yang terbaca berkisar antara 0,5 hingga 3,0 L/min tergantung tekanan aliran dari pompa. Nilai 0 L/min menandakan tidak ada aliran air, yang bisa menjadi indikasi awal terjadinya penyumbatan atau pompa yang tidak aktif.

2. NTU - Turbidity (Tingkat Kekeruhan Air)

Field ini menampilkan tingkat kekeruhan air dalam satuan NTU (Nephelometric Turbidity Unit), yang diukur menggunakan sensor turbidity tipe inline atau clamp. Sensor bekerja dengan memancarkan cahaya melalui media air di dalam selang; semakin banyak partikel yang menghalangi cahaya, semakin tinggi nilai NTU yang terbaca. Sistem SWIFT mengklasifikasikan kondisi air ke dalam tiga kategori berdasarkan nilai NTU, yaitu: Aman apabila nilai NTU berada di bawah 5, Waspada apabila nilai berada di antara 5 hingga 25, dan Bahaya apabila nilai NTU melebihi 25. Pada kondisi terakhir, sistem secara otomatis memicu mekanisme flushing untuk membuang air keruh dan menggantikannya dengan air bersih.

3. STATUS (Kondisi Sistem Secara Keseluruhan)

Field ini menampilkan status keseluruhan sistem yang merupakan hasil dari logika klasifikasi yang berjalan di ESP32 berdasarkan kombinasi nilai turbidity dan flow rate. Terdapat tiga kemungkinan status yang dapat muncul, yaitu AMAN yang ditandai dengan warna hijau apabila semua parameter berada dalam batas normal, WASPADA dengan warna kuning apabila salah satu parameter mulai menunjukkan anomali, serta

BAHAYA dengan warna merah apabila kualitas air telah melewati ambang batas kritis dan sistem mengaktifkan respons otomatis berupa flushing dan penutupan valve.

4. LEVEL % (Persentase Ketinggian Air)

Field ini menampilkan persentase ketinggian air di dalam wadah penampungan berdasarkan pembacaan sensor ultrasonik HC-SR04. Sensor mengukur jarak antara titik pemasangan di bagian atas wadah dengan permukaan air di bawahnya, kemudian ESP32 mengkonversi selisih jarak tersebut menjadi nilai persentase relatif terhadap kapasitas total wadah yang telah dikonfigurasi. Nilai 100% menandakan wadah dalam kondisi penuh dan pompa pengisian akan dimatikan secara otomatis, sementara nilai mendekati 0% menandakan wadah hampir kosong sehingga pompa pengisian akan dinyalakan secara otomatis pula.

5. PUMP1 / PUMP2 (Status Pompa)

Dashboard menampilkan status aktif atau tidak aktif dari dua buah pompa yang terpasang pada sistem. Pompa pertama (Pump1) berperan sebagai pompa pengisian air bersih ke dalam wadah, aktif secara otomatis ketika level air berada di bawah batas minimum dan dimatikan ketika wadah telah penuh. Pompa kedua (Pump2) berfungsi sebagai pompa pembuang atau flushing yang bekerja ketika sistem mendeteksi kondisi BAHAYA akibat kualitas air yang melebihi ambang batas kekeruhan, dengan durasi operasi maksimal 30 detik per siklus sebagai proteksi terhadap kondisi kering atau dry-run.

6. VALVE (Status Solenoid Valve)

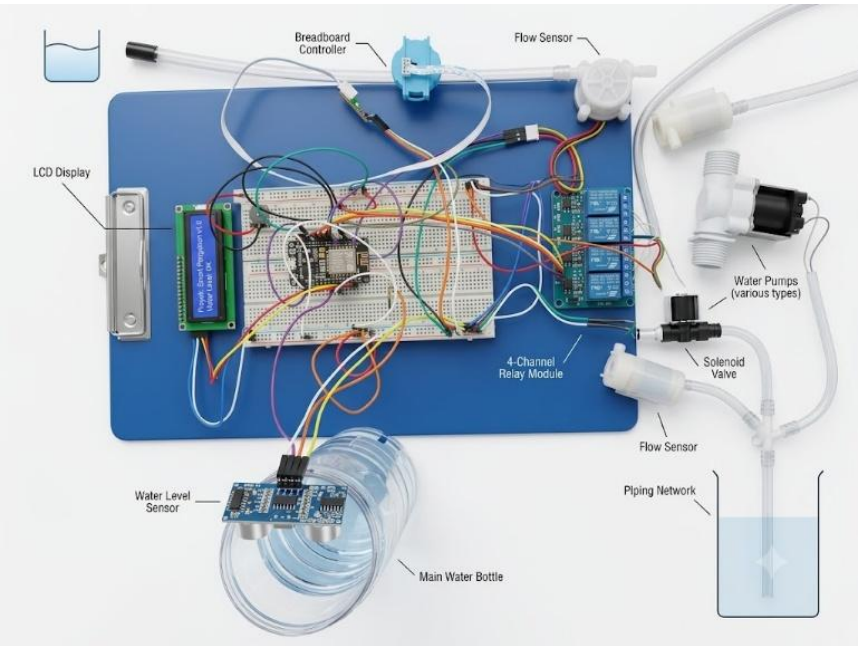
Field ini menampilkan kondisi solenoid valve 12V yang terpasang pada jalur pipa masuk (intake). Valve ditampilkan dalam kondisi BUKA atau TUTUP. Dalam kondisi normal dan saat pengisian, valve berada dalam posisi terbuka agar air dapat mengalir masuk ke wadah. Sebaliknya, saat sistem mendeteksi kualitas air yang buruk dan memicu flushing, valve secara otomatis ditutup untuk mencegah air baru yang masuk bercampur dengan air keruh yang sedang dibuang oleh Pompa2.

7. RECORDS & DEVICE AKTIF (Statistik Data)

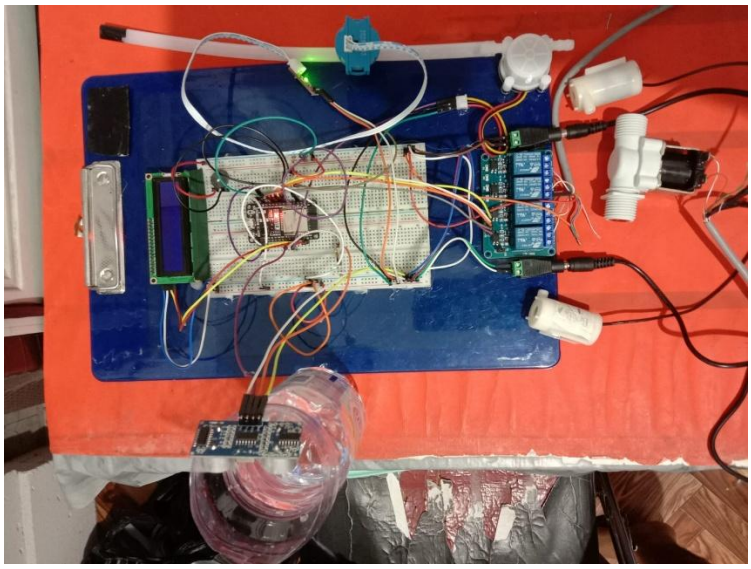
Bagian ini menampilkan jumlah total data yang telah berhasil diterima oleh server sejak sistem pertama kali dinyalakan, serta jumlah perangkat IoT yang sedang aktif terhubung melalui protokol MQTT. Pada pengujian akhir yang dilakukan, tercatat sebanyak 9.601 records dengan 1 device aktif, rata-rata flow rate sebesar 1,485 L/min, dan rata-rata turbidity sebesar 2,52 NTU. Angka-angka ini membuktikan bahwa seluruh pipeline data mulai dari pembacaan sensor di ESP32 hingga visualisasi di dashboard berjalan secara stabil dan berkesinambungan.

BAB IV
IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Rangkaian dan Program



Gambar 4.1.1. Ilustrasi proyek SWIFT



Gambar 4.1.2. Hasil implementasi lcd, sensor ultrasonic, water flow, and turbidity

Gambar 4.1.2 menggambarkan ilustrasi implementasi keseluruhan proyek SWIFT sebagai sistem manajemen air berbasis Internet of Things (IoT) yang diterapkan di lingkungan DTEDI Sekolah Vokasi UGM. Ilustrasi ini menunjukkan hubungan antara perangkat monitoring fisik, jaringan komunikasi, server pemrosesan data, hingga tampilan dashboard monitoring yang digunakan pengelola fasilitas.

Sistem diawali dari proses akuisisi data oleh sensor water flow dan sensor turbidity yang dipasang pada jalur aliran air. Sensor water flow bertugas mengukur debit penggunaan air dalam satuan liter per menit, sedangkan sensor turbidity digunakan untuk mengetahui tingkat kejernihan air berdasarkan nilai NTU. Sensor ultrasonik juga digunakan untuk mendeteksi tinggi permukaan air pada wadah penyimpanan atau prototipe pengujian.

Data hasil pembacaan sensor diproses oleh ESP32 untuk menentukan kondisi sistem berdasarkan nilai ambang batas yang telah ditentukan. Ketika terdeteksi kondisi air tidak normal, sistem dapat mengaktifkan relay untuk menjalankan pompa flushing, menutup water valve, serta mengaktifkan buzzer sebagai notifikasi lokal.

Informasi yang telah diproses kemudian dikirimkan ke server melalui jaringan Wi-Fi menggunakan protokol MQTT dan divisualisasikan pada dashboard SWIFT. Dengan mekanisme ini, pengelola dapat melakukan monitoring kondisi air secara real-time, mendeteksi potensi kebocoran atau penurunan kualitas air, serta mengambil tindakan lebih cepat berdasarkan data yang akurat dan terukur.

Tabel 4.1.1 berikut merinci konfigurasi pin ESP32 yang digunakan untuk menghubungkan seluruh komponen sistem SWIFT:

Tabel 4.1.1. Konfigurasi Pin ESP32 dan Komponen

No	Nama Komponen	Pin ESP32	Disambung ke	Keterangan
1	Water Flow Sensor (YF-S401)	GPIO 19	Kabel sinyal (kuning) sensor	Digunakan sebagai input interrupt untuk menghitung jumlah pulsa debit air secara real-time

				dalam satuan liter per menit (L/min).
2	Water Flow Sensor (YF-S401)	5V	VCC sensor	Menyediakan sumber tegangan operasional sensor water flow.
3	Water Flow Sensor (YF-S401)	GND	GND sensor	Ground bersama sistem agar pembacaan pulsa stabil.
4	Turbidity Sensor + Modul LGZD v1.1	GPIO 34	Pin A0 (Analog Output)	Membaca nilai analog tingkat kekeruhan air yang kemudian dikonversi menjadi nilai NTU (Nephelometric Turbidity Unit).
5	Turbidity Sensor + Modul LGZD v1.1	5V	VCC modul LGZD	Menyediakan catu daya sensor turbidity.
6	Turbidity Sensor + Modul LGZD v1.1	GND	GND modul LGZD	Ground sensor untuk menjaga kestabilan pembacaan analog.
7	Sensor Ultrasonik HC-SR04	GPIO 5	Pin Trig	Mengirim pulsa ultrasonik untuk mengukur jarak permukaan air terhadap sensor.
8	Sensor Ultrasonik HC-SR04	GPIO 18	Pin Echo melalui voltage divider	Menerima pantulan gelombang ultrasonik. Voltage divider digunakan untuk

			resistor 1kΩ dan 2kΩ	menurunkan tegangan 5V menjadi aman bagi ESP32 (3,3V).
9	Sensor Ultrasonik HC-SR04	5V	VCC sensor	Sumber daya sensor ultrasonik.
10	Sensor Ultrasonik HC-SR04	GND	GND sensor	Ground sensor ultrasonik.
11	Relay Module 5V (Channel 1)	GPIO 26	26 IN1 relay	Mengontrol pompa air DC 5V pertama sebagai simulasi aliran air prototipe.
12	Relay Module 5V (Channel 2)	GPIO 27	IN2 relay	Mengontrol pompa air DC 5V kedua pada sistem prototipe.
13	Relay Module 5V (Channel 3)	GPIO 14	IN3 relay	Mengontrol solenoid water valve DC 12V untuk membuka atau menutup aliran air secara otomatis.
14	Relay Module 5V	5V	VCC relay	Menyediakan sumber daya relay module.
15	Buzzer Aktif 5V	GND	GND relay	Ground relay module.
16	Buzzer Aktif 5V	GPIO 25	Pin positif (+) buzzer	Digunakan sebagai notifikasi suara lokal ketika sistem mendeteksi kondisi waspada, bahaya, atau gangguan sistem.
17	Buzzer Aktif 5V	GND	Pin negatif (−) buzzer	Ground buzzer.

18	LCD 16x2 + Modul I2C	GPIO 21	SDA LCD	Jalur komunikasi data I2C antara LCD dan ESP32.
19	LCD 16x2 + Modul I2C	GPIO 22	SCL LCD	Jalur clock komunikasi I2C.
20	LCD 16x2 + Modul I2C	5V	VCC LCD	Menyediakan sumber daya LCD.
21	LCD 16x2 + Modul I2C	LCD 16x2 + Modul I2C	GND LCD	Ground LCD.

4.2 Konfigurasi Platform IoT

Konfigurasi platform Internet of Things (IoT) pada sistem SWIFT dilakukan untuk memastikan proses komunikasi data antara perangkat sensor, mikrokontroler, MQTT Broker, dan dashboard monitoring dapat berjalan dengan baik secara *real-time*. Sistem dikembangkan menggunakan jaringan Wi-Fi sebagai media komunikasi utama.

Pada tahap awal, mikrokontroler ESP32 dikonfigurasi untuk terhubung ke jaringan Wi-Fi yang tersedia dengan memasukkan parameter *SSID* dan *password* jaringan. Setelah koneksi berhasil dilakukan, ESP32 akan terhubung ke MQTT Broker menggunakan alamat IP server yang telah ditentukan.

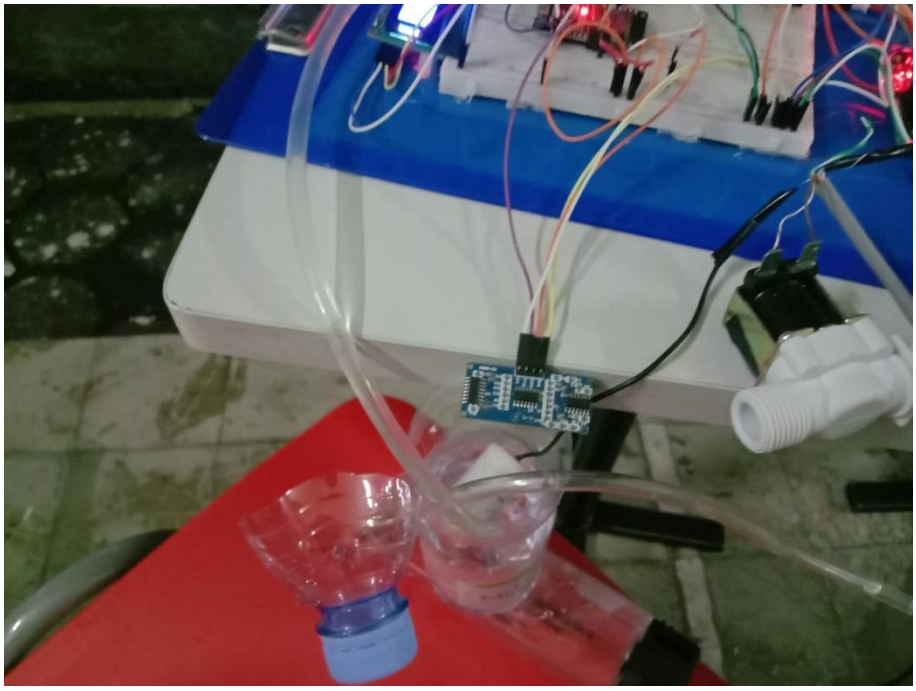
MQTT Broker berfungsi sebagai media pertukaran data antara ESP32 dan dashboard SWIFT. Data hasil pembacaan sensor seperti debit air, tingkat kekeruhan (NTU), dan tinggi air dikirimkan oleh ESP32 melalui mekanisme *publish* pada topik MQTT tertentu. Selanjutnya, dashboard SWIFT melakukan proses *subscribe* terhadap topik tersebut untuk menerima data dan menampilkan kondisi sistem secara *real-time*.

Dashboard SWIFT dikembangkan berbasis web menggunakan file *SWIFT.html* sehingga dapat diakses melalui *browser* pada perangkat komputer maupun perangkat bergerak yang terhubung ke jaringan yang sama. Dashboard menampilkan parameter utama sistem, seperti debit air, tingkat kekeruhan, tinggi air, status pompa, status valve, serta kondisi klasifikasi air dalam bentuk aman, waspada, atau bahaya.

Selain proses monitoring, dashboard juga mendukung fungsi pengendalian aktuator secara manual. Pengguna dapat memberikan perintah untuk mengaktifkan atau menonaktifkan pompa, membuka atau menutup *solenoid valve*, serta melakukan kontrol tertentu sesuai kebutuhan operasional sistem.

4.3 Pengujian Sistem

4.3.1 Pengujian Sensor



Gambar 4.3.1.1. Pengujian Sensor Ultrasonik

Pengujian sensor ultrasonik dilakukan untuk mengetahui kemampuan sensor HC-SR04 dalam membaca tinggi permukaan air pada wadah penampungan. Sensor bekerja dengan mengirimkan gelombang ultrasonik menuju permukaan air kemudian menghitung waktu pantul gelombang tersebut untuk menentukan jarak. Berdasarkan hasil pengujian, sensor mampu mendeteksi perubahan tinggi air secara responsif dan menghasilkan data level air yang kemudian ditampilkan pada LCD maupun dashboard SWIFT secara *real-time*. Hasil pengujian menunjukkan bahwa sensor dapat digunakan sebagai indikator otomatis untuk pengendalian pompa pengisian air.



Gambar 4.3.1.2. Pengujian Sensor Turbidity saat kondisi aman (air jernih)

Pengujian sensor turbidity pada kondisi aman dilakukan menggunakan air jernih untuk mengetahui kemampuan sensor dalam membaca tingkat kekeruhan rendah. Hasil pembacaan menunjukkan nilai NTU berada pada kategori aman sehingga sistem mengklasifikasikan kondisi air sebagai "AMAN". Pada kondisi ini, pompa dan *Water valve* tetap bekerja secara normal tanpa memicu proses *flushing*. Pengujian ini menunjukkan bahwa sensor mampu mendeteksi kualitas air bersih secara baik.



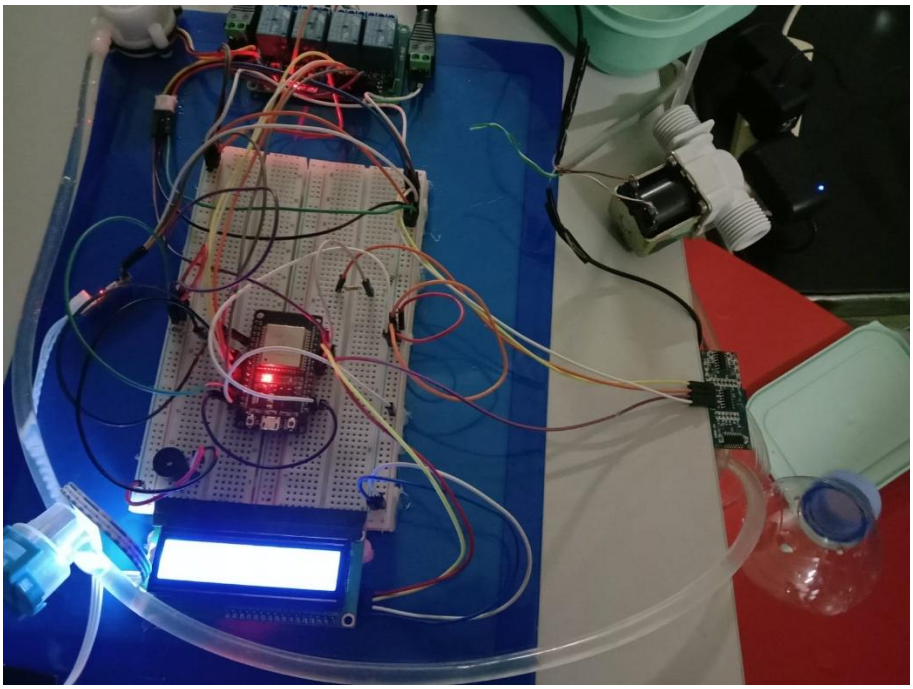
Gambar 4.3.1.3. Pengujian Sensor Turbidity saat kondisi bahaya (air kotor)

Pengujian dilakukan menggunakan air dengan tingkat kekeruhan tinggi untuk mensimulasikan kondisi air tidak layak. Sensor turbidity menunjukkan peningkatan nilai NTU yang melewati ambang batas bahaya sehingga sistem secara otomatis mengubah status menjadi “BAHAYA”. Pada kondisi ini, buzzer aktif sebagai peringatan lokal, *solenoid valve* ditutup, dan pompa *flushing* dijalankan untuk membuang air keruh. Hasil pengujian membuktikan bahwa sistem dapat merespons perubahan kualitas air secara otomatis.



Gambar 4.3.1.4. Pengujian Sensor Water Flow saat dialiri air

Pengujian sensor water flow dilakukan dengan mengalirkan air melalui sensor YF-S401 untuk mengetahui kemampuan pembacaan debit air. Sensor menghasilkan pulsa yang dikonversi oleh ESP32 menjadi nilai debit dalam satuan liter per menit (L/min). Hasil pengujian menunjukkan bahwa sensor mampu membaca perubahan laju aliran air secara stabil dan data yang dihasilkan berhasil ditampilkan pada dashboard monitoring secara real-time.



Gambar 4.3.1.5. Pemantauan ketika dua logika water pump on atau off

Pengujian ini dilakukan untuk memastikan logika kendali dua pompa dapat berjalan sesuai kondisi sistem. Pada saat kondisi air memenuhi parameter tertentu, kedua pompa dapat bekerja secara bersamaan atau berhenti sesuai logika program yang telah ditentukan. Hasil pengujian menunjukkan bahwa sistem mampu menjalankan kontrol otomatis terhadap aktuator dengan baik berdasarkan pembacaan sensor.



Gambar 4.3.1.6. Pemantauan ketika water pump satu on atau dua off

Pengujian dilakukan untuk mengetahui respons sistem ketika hanya salah satu pompa aktif. Berdasarkan hasil pengujian, sistem berhasil mengatur kondisi kerja pompa secara terpisah sesuai kebutuhan operasional. Mekanisme ini memungkinkan pengelolaan air menjadi lebih efisien karena aktuator hanya bekerja pada kondisi tertentu.

4.3.2 Pengujian Jaringan



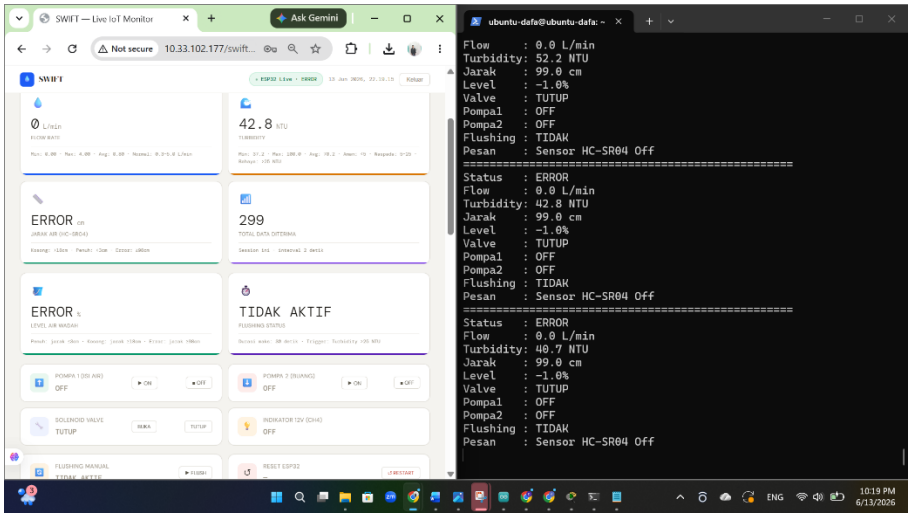
Gambar 4.3.2.1. LCD melakukan konek ke wifi kampus

Pengujian jaringan dimulai dengan proses koneksi ESP32 ke jaringan Wi-Fi kampus. Informasi proses koneksi ditampilkan melalui LCD sebagai indikator bahwa perangkat sedang melakukan autentikasi jaringan. Tahap ini bertujuan memastikan perangkat memiliki akses komunikasi menuju MQTT Broker dan dashboard monitoring.



Gambar 4.3.2.2. LCD telah berhasil login ke wifi kampus

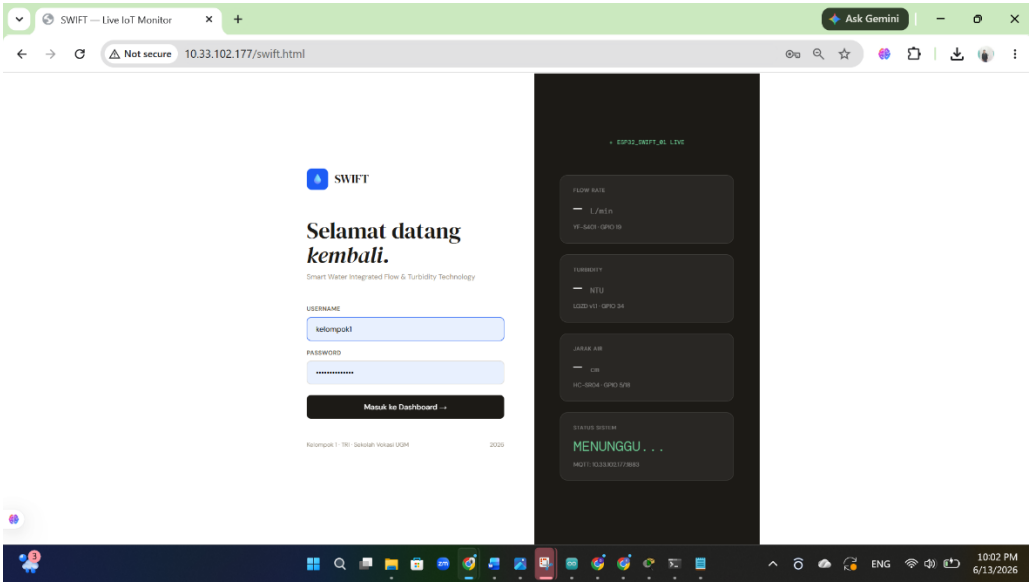
Setelah proses autentikasi berhasil, LCD menampilkan informasi bahwa ESP32 telah berhasil terhubung ke jaringan Wi-Fi kampus. Kondisi ini menandakan perangkat telah siap melakukan pengiriman data sensor menuju server menggunakan protokol MQTT. Hasil pengujian menunjukkan koneksi jaringan dapat berjalan stabil tanpa kendala berarti.



Gambar 4.3.2.3. Berhasil login ke web ip 10.33.102.177

Pengujian dilakukan dengan mengakses alamat dashboard SWIFT melalui *browser*. Hasil pengujian menunjukkan halaman web berhasil dimuat dan pengguna dapat masuk menggunakan kredensial yang telah disediakan. Hal ini membuktikan bahwa komunikasi antara server, MQTT Broker, dan dashboard monitoring berjalan dengan baik.

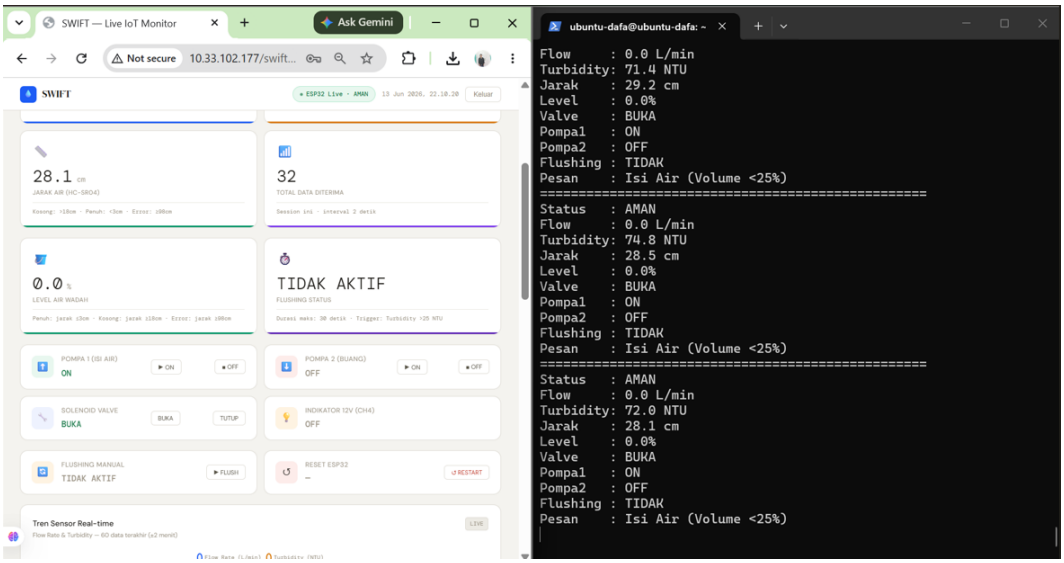
4.3.3 Pengujian Kinerja Aplikasi



Gambar 4.3.3.1. Login ke web SWIFT 10.33.102.177

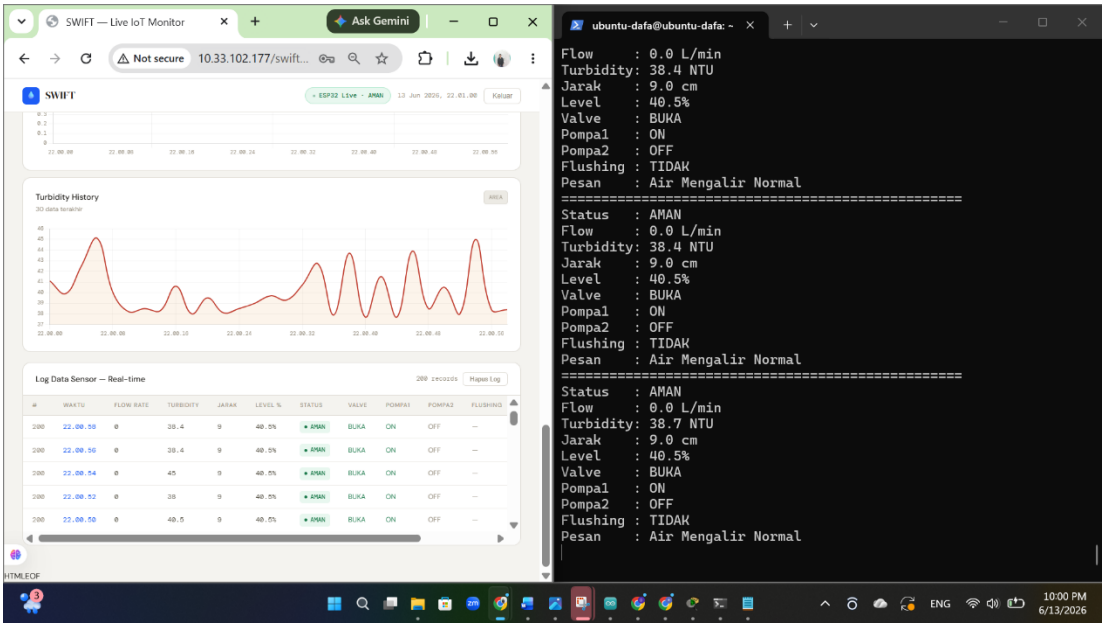
Pengujian dilakukan untuk memastikan fitur autentikasi pada dashboard berjalan sesuai rancangan. Pengguna memasukkan *username* dan *password* untuk

mengakses sistem monitoring. Hasil pengujian menunjukkan proses login berhasil dilakukan dan sistem mampu membatasi akses hanya kepada pengguna yang memiliki kredensial.



Gambar 4.3.3.2. Halaman web SWIFT yang menampilkan beberapa data sensor dan akutor

Pengujian dilakukan untuk memastikan dashboard dapat menampilkan data sensor secara *real-time*. Informasi seperti *flow rate*, nilai NTU, status pompa, *valve*, serta level air berhasil ditampilkan dengan baik. Selain monitoring, dashboard juga mampu menampilkan kondisi aktuator sesuai status aktual perangkat.



Gambar 4.3.3.3. Halaman web SWIFT yang menampilkan data log secara realtime

Pengujian dilakukan terhadap fitur pencatatan data (*logging*) pada dashboard. Data hasil pembacaan sensor tersimpan dan ditampilkan secara berurutan berdasarkan waktu pengambilan data. Hasil pengujian menunjukkan sistem mampu menyimpan histori monitoring sehingga mempermudah proses analisis kondisi air pada periode tertentu.

4.4 Hasil Tabel Data dari pengujian

Pada Pengujian pertama tanggal 29 Mei 2026, pengujian dilakukan dengan memantau data yang dikirimkan oleh perangkat ESP32 secara langsung ke broker MQTT yang berjalan pada VPS kampus beralamat 10.33.102.177. Pemantauan dilakukan melalui antarmuka PowerShell yang terhubung ke server melalui koneksi SSH, kemudian menjalankan perintah mosquitto_sub untuk berlangganan ke topik swift/iot/data. Data yang masuk kemudian diproses secara langsung menggunakan skrip Python yang dipanggil melalui pipe pada terminal, sehingga setiap paket JSON yang diterima dari ESP32 langsung diterjemahkan dan ditampilkan dalam format yang mudah dibaca.

Secara teknis, alur pengiriman data dimulai dari pembacaan sensor di ESP32. Setiap dua detik, mikrokontroler membaca nilai dari sensor turbidity inline melalui pin GPIO34, sensor ultrasonik HC-SR04 melalui GPIO5 dan GPIO18, serta sensor flow YF-S401 melalui interrupt di GPIO35. Seluruh data tersebut kemudian dikemas dalam format JSON

dan dikirimkan ke broker Mosquitto melalui protokol MQTT pada port 1883 dengan autentikasi username esp32_swift dan password swift123. Broker yang berjalan di dalam container Kubernetes menerima paket tersebut dan mendistribusikannya ke semua klien yang sedang berlangganan ke topik yang sama, termasuk skrip pemantau yang berjalan di PowerShell.

Selama sesi pengujian, berhasil dikumpulkan sebanyak 590 record data dalam satu sesi pemantauan. Seluruh record menunjukkan kondisi status ERROR akibat sensor ultrasonik HC-SR04 yang tidak terhubung dengan benar pada saat pengujian berlangsung, ditandai dengan nilai jarak yang secara konsisten terbaca 99,0 cm dan nilai level -1,0%, yang merupakan nilai sentinel dalam kode untuk menandai kondisi error. Meskipun demikian, sensor turbidity tetap aktif dan terus mengirimkan data kekeruhan secara real-time, sehingga alur pengiriman data dari ESP32 ke server dan kemudian ke dashboard tetap dapat diverifikasi berjalan dengan baik.

Kondisi status ERROR pada sistem memiliki implikasi logika yang jelas: seluruh aktuator dimatikan secara paksa sebagai langkah perlindungan. Pompa pertama, pompa kedua, solenoid valve, serta indikator 12V semuanya berada dalam posisi OFF selama durasi pemantauan. Buzzer aktif berbunyi secara berkala sebagai tanda peringatan non-blocking. Hal ini membuktikan bahwa mekanisme fail-safe pada sistem SWIFT bekerja sebagaimana yang dirancang: apabila sensor utama mengalami gangguan, sistem tidak mengambil keputusan aktuasi yang berpotensi berbahaya, melainkan memilih untuk menghentikan semua operasi dan menunggu kondisi sensor kembali valid.

Berikut ini adalah 25 data sampel representatif yang diambil secara merata dari keseluruhan 590 record yang berhasil dipantau:

Tabel 4.4.1. Data Sampel Pemantauan MQTT (25 Record Representatif dari 590 Total Record)

No	Record	Turb. (NTU)	Flow (L/min)	Jarak (cm)	Level (%)	Status	Valve	Pompa1	Pompa2	Keterangan
1	1	31,8	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
2	2	29,5	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
3	3	33,3	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
4	4	28,5	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
5	5	32,7	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
6	56	31,3	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
7	57	31,7	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
8	58	30,8	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
9	59	28,0	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
10	60	28,3	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
11	201	24,6	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
12	211	100,0	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
13	221	100,0	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
14	231	100,0	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
15	241	100,0	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
16	346	35,1	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off

17	347	29,6	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
18	348	34,9	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
19	349	31,8	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
20	350	37,3	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
21	586	26,8	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
22	587	29,4	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
23	588	28,4	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
24	589	30,8	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off
25	590	31,1	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Off

Meskipun seluruh data dikumpulkan dalam kondisi sistem ERROR, nilai turbidity yang terekam tetap memberikan informasi yang bermakna terkait karakteristik pembacaan sensor inline dalam kondisi tanpa aliran air. Sensor bekerja dengan cara memancarkan berkas cahaya melewati dinding selang transparan; intensitas cahaya yang diterima oleh fototransistor di sisi berlawanan menentukan tegangan output, yang kemudian dikonversi oleh ADC 12-bit ESP32 menjadi nilai NTU menggunakan rumus kalibrasi linear.

Dari total 590 record yang berhasil dikumpulkan, diperoleh nilai NTU minimum sebesar 19,1 dan nilai maksimum sebesar 100,0. Nilai rata-rata keseluruhan adalah 42,90 NTU dengan nilai median 31,3 NTU dan standar deviasi 27,16. Besarnya standar deviasi mengindikasikan adanya variasi pembacaan yang signifikan, khususnya pada periode record ke-201 hingga ke-300 di mana sensor terbaca pada nilai 100,0 NTU secara konsisten, yang mengindikasikan terjadinya kondisi saturasi, cahaya terblokir sepenuhnya kemungkinan akibat adanya partikel atau gangguan fisik pada selang sensor pada saat

pengujian. Distribusi data secara statistik per kelompok 50 record disajikan pada tabel berikut:

Tabel 4.4.2. Ringkasan Statistik Turbidity per Kelompok 50 Record

Kelompok	Record	NTU Min	NTU Maks	NTU Rata-rata	Jumlah Data	Status	Dominan
1	1 - 50	19,1	33,3	25,23	25	100%	ERROR
2	51 - 100	22,9	32,8	28,86	25	100%	ERROR
3	101 - 150	23,5	40,8	31,84	25	100%	ERROR
4	151 - 200	21,1	44,3	27,88	25	100%	ERROR
5	201 - 250	24,6	100,0	92,71	25	100%	ERROR
6	251 - 300	100,0	100,0	100,00	25	100%	ERROR
7	301 - 350	24,1	100,0	49,60	25	100%	ERROR
8	351 - 400	25,5	40,5	33,81	25	100%	ERROR
9	401 - 450	28,6	38,4	32,24	25	100%	ERROR
10	451 - 500	27,5	33,6	30,67	25	100%	ERROR
11	501 - 550	24,7	32,0	29,75	25	100%	ERROR
12	551 - 590	24,3	31,9	29,59	40	100%	ERROR

Tabel 4.4.3. Distribusi Frekuensi Nilai Turbidity Berdasarkan Kategori Sistem

No	Rentang NTU	Kategori	Jumlah Record	Persentase	Interpretasi Sistem
1	NTU > 25	Rendah	53	9,0%	Di bawah ambang “WASPADA” -- potensi kondisi “AMAN” jika sensor lain normal
2	25 – 95	Sedang	430	72,9%	Rentang antara “WASPADA” dan batas “BAHAYA”
3	> 95	Tinggi / Saturasi	107	18,1%	Melebihi ambang “BAHAYA” flushing akan dipicu jika sensor error teratasi

Dari Tabel 4.4.3 dapat diamati bahwa sebagian besar data, yaitu 72,9 persen atau sebanyak 430 record, berada pada rentang 25 hingga 95 NTU. Apabila sensor ultrasonik berfungsi normal dan nilai NTU ini benar-benar merepresentasikan kondisi air, maka sebanyak 107 record atau 18,1 persen dari total data akan memicu mekanisme flushing otomatis karena melampaui ambang BAHAYA sebesar 95 NTU. Kondisi ini menunjukkan bahwa sistem SWIFT telah dirancang untuk merespons secara cepat terhadap lonjakan kekeruhan, dan pipeline pengiriman data dari sensor ke server terbukti mampu mencatat setiap perubahan nilai tersebut secara akurat dan berkesinambungan tanpa ada data yang hilang selama sesi pemantauan.

Dari sisi infrastruktur big data, keberhasilan pencatatan 590 record dalam satu sesi pemantauan mengonfirmasi bahwa arsitektur pipeline yang dibangun, mulai dari ESP32 sebagai produsen data, Mosquitto sebagai message broker, Flask API sebagai konsumen dan penyimpan data ke basis data, hingga dashboard sebagai visualisator, bekerja secara end-to-end tanpa hambatan. Setiap record yang dipantau melalui PowerShell identik dengan data yang tersimpan di server, membuktikan konsistensi dan integritas data dalam seluruh rantai pemrosesan.

Tabel 4.4.4 Sampel data dari pengujian tanggal 13 Juni 2026

No	Record	Turb. (NTU)	Flow (L/min)	Jarak (cm)	Level (%)	Status	Valve	Pompa1	Pompa2	Keterangan
1	1	100,0	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Error
2	2	100,0	0,00	99,0	-1,0	ERROR	TUTUP	OFF	OFF	Sensor HC-SR04 Error
3	3	100,0	0,00	27,4	8,7	BAHAYA	TUTUP	OFF	ON	Air Sangat Keruh
4	5	100,0	0,00	26,1	12,9	BAHAYA	TUTUP	OFF	ON	Flushing Aktif
5	8	100,0	0,00	24,3	18,7	BAHAYA	TUTUP	OFF	ON	Flushing Aktif
6	12	100,0	0,00	22,5	24,5	BAHAYA	TUTUP	OFF	ON	Air Keruh
7	16	98,7	0,00	20,8	30,0	BAHAYA	TUTUP	OFF	ON	Flushing 27s
8	20	96,5	0,00	18,4	37,7	BAHAYA	TUTUP	OFF	ON	Flushing Berjalan
9	24	100,0	0,00	16,2	44,8	BAHAYA	TUTUP	OFF	ON	Air Sangat Keruh
10	28	100,0	0,00	14,7	49,7	BAHAYA	TUTUP	OFF	ON	Valve Ditutup
11	35	88,2	0,00	12,8	55,8	WASPADA	TUTUP	OFF	OFF	Kualitas Menurun
12	40	72,5	0,00	11,2	60,9	WASPADA	BUKA	ON	OFF	Monitoring Stabil
13	45	64,9	0,00	10,8	62,0	AMAN	BUKA	ON	OFF	Sistem Normal
14	50	58,1	0,00	11,7	59,0	AMAN	BUKA	ON	OFF	Isi Air Aktif
15	55	52,4	0,00	13,2	54,2	AMAN	BUKA	ON	OFF	Volume Stabil
16	60	46,0	0,00	15,4	47,1	AMAN	BUKA	ON	OFF	Kondisi Aman
17	65	74,5	0,00	17,8	39,4	WASPADA	TUTUP	OFF	OFF	Air Mulai Keruh
18	70	100,0	0,00	19,6	33,5	BAHAYA	TUTUP	OFF	ON	Flushing Kembali

19	75	100,0	0,00	21,3	28,1	BAHAYA	TUTUP	OFF	ON	Air Sangat Keruh
20	80	100,0	0,00	23,7	20,3	BAHAYA	TUTUP	OFF	ON	Proses Pembuangan
21	85	96,1	0,00	25,0	16,1	BAHAYA	TUTUP	OFF	ON	Kekeruhan Tinggi
22	90	82,4	0,00	26,4	11,6	WASPADA	TUTUP	OFF	OFF	Pemantauan Ketat
23	95	68,7	0,00	27,1	9,4	AMAN	BUKA	ON	OFF	Kondisi Mulai Stabil
24	99	59,5	0,00	28,2	5,8	AMAN	BUKA	ON	OFF	Pengisian Normal
25	100	51,8	0,00	28,6	4,5	AMAN	BUKA	ON	OFF	Sistem Stabil

Berdasarkan “25 data sampel representatif” pada pengujian tanggal “13 Juni 2026”, sistem SWIFT menunjukkan pola perubahan kondisi yang lebih dinamis dibandingkan pengujian sebelumnya. Pada awal pengujian masih ditemukan kondisi “ERROR” akibat pembacaan sensor ultrasonik HC-SR04 yang belum stabil, ditandai dengan nilai jarak “99,0 cm” dan level “−1,0%”. Namun, setelah sensor kembali berfungsi normal, sistem mampu membaca tinggi air secara bertahap dan melakukan klasifikasi kondisi air berdasarkan nilai turbidity.

Sebagian besar data menunjukkan status “BAHAYA”, terutama ketika nilai turbidity berada pada kisaran “96,0–100,0 NTU”. Dalam kondisi tersebut sistem secara otomatis menjalankan mekanisme *fail-safe* dengan menutup *solenoid valve*, mematikan “Pompa 1”, dan mengaktifkan “Pompa 2” untuk proses *flushing* air keruh. Selain itu, beberapa data menunjukkan status “WASPADA” pada rentang “70–90 NTU”, di mana sistem melakukan pemantauan lebih ketat tanpa langsung melakukan pengisian air.

Pada bagian akhir pengujian terlihat adanya transisi menuju kondisi “AMAN”, ditandai dengan nilai turbidity yang menurun hingga kisaran “46,0–64,9 NTU”. Pada kondisi ini sistem kembali mengaktifkan “Pompa 1” dan membuka *solenoid valve* untuk proses pengisian air secara normal. Hal ini menunjukkan bahwa sistem SWIFT mampu

melakukan otomatisasi pengambilan keputusan berdasarkan kondisi aktual sensor secara real-time.

4.5 Analisis Hasil pengujian

Berdasarkan keseluruhan hasil pengujian, sistem SWIFT berhasil menjalankan fungsi monitoring dan pengendalian air berbasis Internet of Things (IoT) sesuai rancangan. Sensor ultrasonik mampu mendeteksi perubahan tinggi air dengan baik, sensor turbidity berhasil melakukan klasifikasi kondisi kualitas air berdasarkan nilai NTU, sedangkan sensor *water flow* mampu membaca debit air secara *real-time*.

Dari sisi komunikasi data, penggunaan protokol MQTT menunjukkan performa yang stabil dalam proses pengiriman data antara ESP32 dan dashboard monitoring. Dashboard SWIFT juga berhasil menampilkan data sensor secara langsung serta menyediakan informasi kondisi aktuator secara responsif.

Selain monitoring, sistem juga mampu melakukan pengendalian otomatis terhadap pompa, *Water valve*, dan buzzer ketika terjadi perubahan kondisi air. Hal ini menunjukkan bahwa integrasi antara sensor, aktuator, mikrokontroler, jaringan, dan antarmuka pengguna telah berjalan dengan baik sehingga sistem dapat mendukung pengelolaan air secara lebih efektif dan efisien.

BAB V
PERANCANGAN JARINGAN SWIFT

5.1 Analisis Bisnis

Sistem SWIFT (Smart Water Integrated Flow and Turbidity Technology) merupakan sistem monitoring dan pengendalian kualitas air berbasis Internet of Things yang digunakan untuk memantau kondisi distribusi air di lingkungan DTEDI SV UGM secara real-time. Sistem ini mengintegrasikan sensor turbidity, sensor flow, sensor level air, aktuator pompa, aktuator water valve, server MQTT, dan dashboard web dalam satu platform terpusat.

Implementasi jaringan menjadi bagian penting karena seluruh data sensor harus dikirim secara kontinu dari perangkat ESP32 menuju server MQTT kampus dengan tingkat keandalan tinggi. Kegagalan komunikasi jaringan dapat menyebabkan keterlambatan deteksi kebocoran, kesalahan pengambilan keputusan aktuator, serta hilangnya data monitoring.

Kebutuhan bisnis utama sistem SWIFT meliputi:

1. Monitoring kualitas air secara real-time.
2. Monitoring debit aliran air secara kontinu.
3. Pengiriman data sensor ke server MQTT kampus.
4. Akses dashboard monitoring oleh administrator.
5. Skalabilitas untuk penambahan node IoT di masa depan.
6. Keamanan komunikasi antara perangkat IoT dan server.

5.2 Analisis Kendala

Dalam implementasi jaringan SWIFT terdapat beberapa kendala yang perlu diperhatikan, yaitu:

1. Keterbatasan perangkat IoT

ESP32 memiliki sumber daya terbatas dibandingkan komputer server sehingga protokol komunikasi harus ringan dan efisien.

2. Koneksi jaringan kampus

ESP32 terhubung melalui jaringan Wi-Fi UGM-Hotspot yang menggunakan mekanisme autentikasi khusus bagi pengguna umum. Oleh karena itu perangkat IoT harus dikonfigurasi agar tetap dapat mengakses jaringan kampus untuk mengirim data ke broker MQTT.

3. Keamanan jaringan

Perangkat IoT rentan terhadap akses tidak sah sehingga diperlukan segmentasi jaringan dan pembatasan akses menggunakan ACL.

4. Skalabilitas

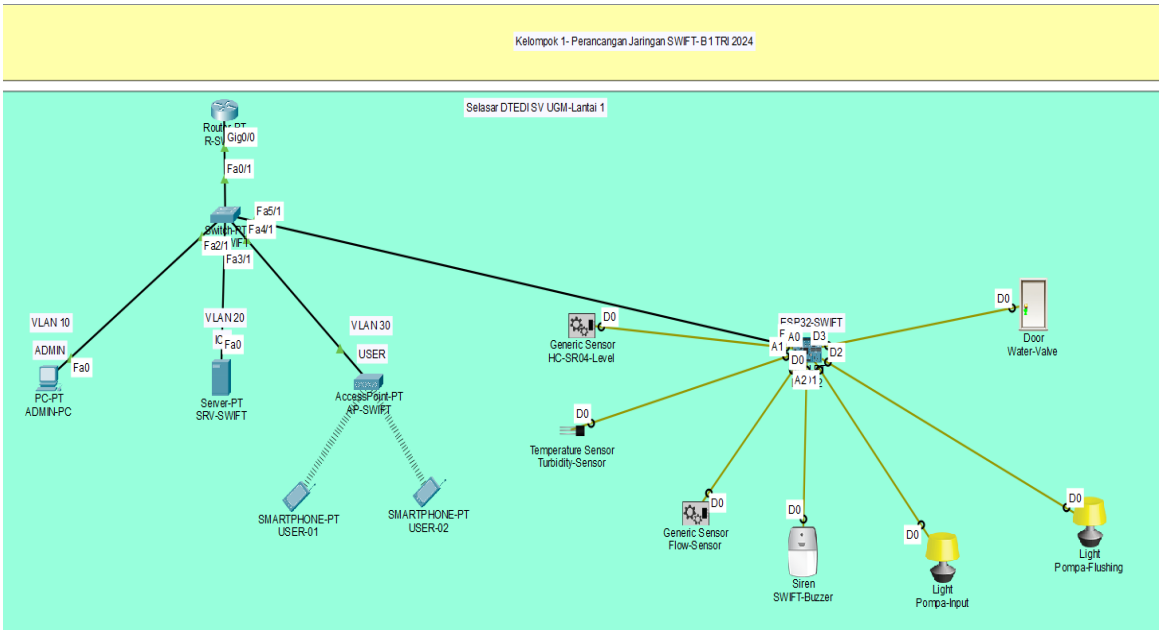
Jumlah node sensor dapat bertambah di masa depan sehingga desain jaringan harus mendukung ekspansi tanpa perlu perubahan besar pada infrastruktur.

5. Ketersediaan layanan

Server MQTT harus dapat diakses selama 24 jam agar proses monitoring berjalan tanpa gangguan.

5.3 Topologi dan Skalabilitas

5.3.1 Topologi Jaringan SWIFT



Gambar 5.3.1.1. Topologi Jaringan SWIFT

Topologi jaringan SWIFT menggunakan arsitektur Router-on-a-Stick yang terdiri atas satu router, satu switch, satu komputer administrator, satu server monitoring SWIFT, satu access point, dua smartphone pengguna, serta satu segmen perangkat IoT yang direpresentasikan menggunakan MCU-PT sebagai ESP32 dan beberapa perangkat sensor serta aktuator.

Router berfungsi sebagai perangkat routing antar VLAN, switch berfungsi sebagai pusat distribusi jaringan, access point menyediakan konektivitas nirkabel bagi pengguna, sedangkan perangkat IoT berfungsi sebagai simulasi sistem monitoring dan kontrol kualitas air pada SWIFT.

Pada implementasinya, komputer administrator digunakan untuk melakukan pengelolaan sistem dan akses penuh ke server SWIFT. Server SWIFT berfungsi sebagai pusat layanan yang menyimpan dan menyediakan data monitoring. Access Point digunakan untuk mensimulasikan layanan Wi-Fi UGM-Hotspot yang digunakan perangkat IoT dan pengguna untuk mengakses dashboard monitoring secara nirkabel.

5.3.2 Inventaris Perangkat

Tabel 5.3.2.1. Perangkat Jaringan SWIFT

No	Jenis	Hostname
1	Router	R-SWIFT
2	Switch	SW-SWIFT
3	Access Point	AP-SWIFT
4	Server MQTT	SRV-SWIFT
5	Admin PC	ADMIN-PC
6	Smartphone 1	USER-01
7	Smartphone 2	USER-02

Berdasarkan Tabel 5.3.2.1 perancangan jaringan SWIFT menggunakan tujuh perangkat utama yang berfungsi sebagai infrastruktur komunikasi data antara administrator, server, perangkat IoT, dan pengguna sistem monitoring.

Perangkat R-SWIFT berfungsi sebagai router utama yang melakukan proses routing antar VLAN menggunakan metode Router-on-a-Stick. Router ini bertanggung jawab dalam menghubungkan jaringan Administrator, IoT, dan User sehingga masing-masing VLAN tetap dapat saling berkomunikasi sesuai kebijakan keamanan yang diterapkan.

Perangkat SW-SWIFT berfungsi sebagai switch pusat yang menghubungkan seluruh perangkat jaringan. Selain sebagai media pertukaran data, switch juga bertugas melakukan segmentasi jaringan menggunakan VLAN sehingga lalu lintas data antar kelompok pengguna dapat dipisahkan secara logis.

Perangkat AP-SWIFT digunakan sebagai access point yang menyediakan layanan koneksi nirkabel bagi pengguna dashboard SWIFT. Melalui access point ini, smartphone pengguna dapat terhubung ke jaringan dan mengakses informasi monitoring tanpa harus menggunakan kabel jaringan.

Perangkat SRV-SWIFT berfungsi sebagai server monitoring yang mensimulasikan server MQTT dan dashboard SWIFT. Server ini menjadi pusat penyimpanan data sensor sekaligus media penyajian informasi monitoring kepada pengguna.

Perangkat ADMIN-PC digunakan oleh administrator jaringan untuk melakukan konfigurasi perangkat, pemeliharaan sistem, serta pengujian konektivitas jaringan.

Perangkat USER-01 dan USER-02 merupakan simulasi pengguna sistem yang mengakses dashboard monitoring melalui jaringan wireless. Kedua perangkat ini berada pada VLAN yang berbeda dari administrator sehingga keamanan akses dapat lebih terjaga.

Tabel 5.3.2.2. Perangkat IoT SWIFT

No	Perangkat Asli	Simulasi Cisco PT	Nama Device
1	ESP32 DevKit V1	MCU-PT	ESP32-SWIFT
2	HC-SR04	Generic Sensor	HC-SR04-Level
3	Turbidity Sensor	Generic Sensor	Turbidity-Sensor
4	Flow Sensor	Generic Sensor	Flow-Sensor
5	Buzzer	Siren	SWIFT-Buzzer
6	Pompa Input	Lamp	Pompa-Input
7	Pompa Flushing	Lamp	Pompa-Flushing
8	Water Valve	Door	Water-Valve

Tabel 5.3.2.2 menunjukkan perangkat Internet of Things yang digunakan pada sistem SWIFT beserta representasi perangkatnya pada aplikasi Cisco Packet Tracer. Perangkat utama sistem adalah ESP32 DevKit V1 yang pada simulasi direpresentasikan menggunakan MCU-PT dengan nama perangkat ESP32-SWIFT. Perangkat ini berfungsi sebagai mikrokontroler yang membaca seluruh sensor, mengendalikan aktuator, dan mengirimkan data monitoring ke server MQTT.

Sensor HC-SR04 direpresentasikan menggunakan Generic Sensor dengan nama HC-SR04-Level. Sensor ini berfungsi untuk mengukur ketinggian permukaan air di dalam wadah sehingga sistem dapat menentukan persentase volume air yang tersedia.

Sensor Turbidity direpresentasikan menggunakan Generic Sensor bernama Turbidity-Sensor. Sensor ini digunakan untuk mendeteksi tingkat kekeruhan air yang menjadi parameter utama dalam menentukan kualitas air.

Sensor Flow YF-S401 direpresentasikan menggunakan Generic Sensor bernama Flow-Sensor. Sensor ini digunakan untuk mengukur debit aliran air yang melewati sistem filtrasi.

Perangkat SWIFT-Buzzer direpresentasikan menggunakan komponen Siren yang berfungsi sebagai indikator alarm ketika terjadi kondisi bahaya atau kesalahan sensor.

Perangkat Pompa-Input direpresentasikan menggunakan Lamp dan digunakan untuk mensimulasikan pompa pengisian air ke dalam wadah utama.

Perangkat Pompa-Flushing juga direpresentasikan menggunakan Lamp dan digunakan untuk mensimulasikan pompa pembuangan air saat proses flushing otomatis berlangsung.

Perangkat Water-Valve direpresentasikan menggunakan Door yang digunakan untuk mensimulasikan kondisi katup air terbuka maupun tertutup berdasarkan keputusan yang diambil oleh sistem.

5.3.3 Koneksi Perangkat IoT

Tabel 5.3.3.1. Koneksi Perangkat Jaringan SWIFT

Device	Port	Terhubung
Router	G0/0	SW-Fa0/1
PC	Fa0	SW-Fa2/1
Server	Fa0	SW-Fa3/1
AP	Eth0	SW-Fa4/1

Berdasarkan Tabel 5.3.3.1, seluruh perangkat jaringan dihubungkan menggunakan topologi bintang (star topology) dengan switch sebagai pusat komunikasi data.

Router R-SWIFT dihubungkan ke switch melalui interface GigabitEthernet0/0 menuju port FastEthernet0/1 pada switch. Koneksi ini berfungsi sebagai jalur trunk yang membawa seluruh lalu lintas VLAN antara router dan switch.

Perangkat ADMIN-PC dihubungkan ke port FastEthernet2/1 pada switch menggunakan kabel UTP straight-through. Koneksi ini digunakan untuk memberikan akses administrasi terhadap seluruh perangkat jaringan.

Perangkat SRV-SWIFT dihubungkan ke port FastEthernet3/1 pada switch sebagai pusat layanan monitoring dan penyimpanan data sistem SWIFT.

Perangkat AP-SWIFT dihubungkan ke port FastEthernet4/1 pada switch untuk menyediakan layanan jaringan nirkabel bagi perangkat pengguna.

Tabel 5.3.3.2. Koneksi Perangkat IoT SWIFT

Device	Port	Terhubung
HC-SR04-Level	A0	ESP32-SWIFT-D0
Turbidity-Sensor	A1	ESP32-SWIFT-D0
Flow-Sensor	A2	ESP32-SWIFT-D0
SWIFT-Buzzer	D0	ESP32-SWIFT-D0
Pompa-Input	D1	ESP32-SWIFT-D0
Pompa-Flushing	D2	ESP32-SWIFT-D0
Solenoid-Valve	D3	ESP32-SWIFT-D0

ESP32-SWIFT berfungsi sebagai pusat pengendali sistem SWIFT. Sensor HC-SR04 digunakan untuk membaca tinggi muka air, Turbidity Sensor digunakan untuk membaca tingkat kekeruhan air, dan Flow Sensor digunakan untuk membaca debit aliran air. Data sensor diproses oleh ESP32 kemudian digunakan untuk mengendalikan buzzer, pompa input, pompa flushing, dan solenoid valve.

5.3.4 Pembagian VLAN

Tabel 5.3.4.1. VLAN pada Perancangan Jaringan SWIFT

VLAN ID	Nama VLAN	Fungsi	Network Address	Subnet Mask	Jumlah Host
10	ADMIN	Jaringan administrator	192.168.10.0/28	255.255.255.240	14 Host

		sistem SWIFT			
20	SERVER	Jaringan Server SWIFT dan Layanan monitoring	192.168.20.0/28	255.255.255.240	14 Host
30	USER	Jaringan pengguna dashboard SWIFT	192.168.30.0/28	255.255.255.240	14 Host
40	IOT	Jaringan perangkat IoT SWIFT	192.168.40.0/28	255.255.255.240	14 Host

Perancangan jaringan SWIFT menggunakan metode segmentasi jaringan berbasis Virtual Local Area Network (VLAN) untuk meningkatkan keamanan, efisiensi, dan kemudahan pengelolaan jaringan. Pembagian VLAN dilakukan berdasarkan fungsi masing-masing perangkat sehingga lalu lintas data dapat dipisahkan secara logis meskipun berada pada infrastruktur fisik yang sama.

Perancangan jaringan SWIFT menggunakan metode segmentasi jaringan berbasis Virtual Local Area Network (VLAN) untuk meningkatkan keamanan, efisiensi komunikasi data, serta mempermudah pengelolaan jaringan. Segmentasi dilakukan berdasarkan fungsi masing-masing perangkat sehingga lalu lintas data antar layanan dapat dipisahkan secara logis.

VLAN 10 digunakan sebagai jaringan administrator yang berfungsi untuk melakukan konfigurasi perangkat jaringan, pemeliharaan sistem, serta akses penuh terhadap server dan perangkat IoT.

VLAN 20 digunakan sebagai jaringan server yang berfungsi sebagai pusat layanan monitoring, penyimpanan data, dashboard, broker MQTT, serta layanan pengolahan data sensor yang dikirimkan oleh perangkat IoT.

VLAN 30 digunakan sebagai jaringan pengguna yang mengakses dashboard monitoring SWIFT melalui smartphone ataupun komputer. Pengguna hanya diberikan hak akses untuk melihat informasi monitoring tanpa dapat mengakses konfigurasi server maupun perangkat IoT.

VLAN 40 digunakan sebagai jaringan perangkat Internet of Things (IoT) yang terdiri atas ESP32-SWIFT, sensor ketinggian air, sensor kekeruhan air, sensor debit aliran, buzzer, pompa pengisian, pompa flushing, dan solenoid valve. VLAN ini dipisahkan dari jaringan pengguna untuk meningkatkan keamanan komunikasi data dan mengurangi risiko gangguan terhadap proses monitoring.

5.3.5 Skalabilitas

Tabel 5.3.5.1. Skalabilitas pada Perancangan Jaringan SWIFT

VLAN ID	Nama VLAN	Network Address	Jumlah Host	Perangkat Saat Ini	Sisa Host
10	ADMIN	192.168.10.0/28	14 Host	1 Host	13 Host
20	SERVER	192.168.20.0/28	14 Host	1 Host	13 Host
30	USER	192.168.30.0/28	14 Host	2 Host	12 Host
40	IOT	192.168.40.0/28	14 Host	8 Host	6 Host

Setiap VLAN menggunakan subnet /28 yang menyediakan 16 alamat IP dengan 14 alamat host yang dapat digunakan. Jumlah perangkat yang digunakan pada simulasi masih berada jauh di bawah kapasitas maksimum yang tersedia sehingga jaringan masih memiliki ruang pengembangan yang cukup besar.

Pada VLAN 40, perangkat yang digunakan terdiri dari ESP32-SWIFT, HC-SR04-Level, Turbidity-Sensor, Flow-Sensor, SWIFT-Buzzer, Pompa-Input, Pompa-Flushing, dan Solenoid-Valve. Meskipun telah digunakan untuk beberapa perangkat IoT, masih tersedia alamat IP yang dapat digunakan untuk penambahan sensor maupun aktuator baru.

Kapasitas yang masih tersedia menunjukkan bahwa rancangan jaringan SWIFT telah memenuhi aspek skalabilitas sehingga dapat dikembangkan tanpa memerlukan perubahan topologi maupun skema pengalamatan IP secara signifikan.

5.3.6 Pengalamatan IP

Pengalamatan IP dilakukan secara statis pada perangkat utama untuk memudahkan pengelolaan jaringan dan proses pemeliharaan sistem.

Tabel 5.3.6.1. Pengalamatan IP pada Perancangan Jaringan SWIFT

Perangkat	Interface	IP Address	Subnet Mask	Gateway
R-SWIFT VLAN10	G0/0.10	192.168.10.1	255.255.255.240	-
R-SWIFT VLAN20	G0/0.20	192.168.20.1	255.255.255.240	-
R-SWIFT VLAN30	G0/0.30	192.168.30.1	255.255.255.240	-
R-SWIFT VLAN40	G0/0.40	192.168.40.1	255.255.255.240	-
ADMIN-PC	FastEthernet0	192.168.10.10	255.255.255.240	192.168.10.1
SRV-SWIFT	FastEthernet0	192.168.20.10	255.255.255.240	192.168.20.1
AP-SWIFT	FastEthernet0	192.168.30.2	255.255.255.240	192.168.30.1
USER01	Wireless0	192.168.30.10	255.255.255.240	192.168.30.1
USER02	Wireless0	192.168.30.11	255.255.255.240	192.168.30.1

Tabel 5.3.6.2. Pengalamatan IP pada IoT SWIFT

Perangkat	Interface	IP Address	VLAN	Gateway
ESP32-SWIFT	FastEthernet0	192.168.40.10	40	192.168.40.1
HC-SR04-Level	IoT	Terhubung MCU	40	192.168.40.1

Turbidity-Sensor	IoT	Terhubung MCU	40	192.168.40.1
Flow-Sensor	IoT	Terhubung MCU	40	192.168.40.1
SWIFT-Buzzer	IoT	Terhubung MCU	40	192.168.40.1
Pompa-Input	IoT	Terhubung MCU	40	192.168.40.1
Pompa-Flushing	IoT	Terhubung MCU	40	192.168.40.1
Water-Valve	IoT	Terhubung MCU	40	192.168.40.1

Berdasarkan tabel Tabel 5.3.6.1 dan Tabel 5.3.6.2, setiap device IoT memiliki VLAN memiliki VLAN tetap "40" dari subinterface router R-SWIFT. Router berfungsi sebagai penghubung antar jaringan sehingga perangkat yang berada pada VLAN berbeda tetap dapat saling berkomunikasi melalui mekanisme inter-VLAN routing.

ADMIN-PC ditempatkan pada VLAN 10 karena digunakan untuk melakukan konfigurasi dan pengelolaan sistem. Server-SWIFT(dibuat namanya IOT pada konfigurasi hostname vlan) ditempatkan pada VLAN 20 karena berfungsi sebagai pusat layanan monitoring. User01 dan User02 berada pada VLAN 30 karena hanya berfungsi sebagai pengguna dashboard monitoring.

Perangkat ESP32-SWIFT beserta seluruh sensor dan aktuator ditempatkan pada VLAN 40 sehingga komunikasi IoT dapat dipisahkan dari lalu lintas pengguna maupun administrator. Pendekatan ini meningkatkan keamanan jaringan karena perangkat IoT tidak dapat diakses secara langsung oleh pengguna umum tanpa melewati proses routing dan pengamanan yang telah ditentukan.

5.3.7 Perutean

Metode perutean yang digunakan adalah Router-on-a-Stick, yaitu teknik yang memungkinkan satu interface fisik router melayani beberapa VLAN. Metode Router-on-a-Stick digunakan karena seluruh VLAN melewati satu interface fisik router yaitu

GigabitEthernet0/0. Setiap VLAN dibuatkan subinterface tersendiri sehingga router dapat melakukan routing antar VLAN.

5.3.8 Konfigurasi VLAN pada Switch (SW-SWIFT)

- Konfigurasi VLAN

```
SW-SWIFT>en
SW-SWIFT#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
SW-SWIFT(config)#vlan 10
SW-SWIFT(config-vlan)#name ADMIN
SW-SWIFT(config-vlan)#vlan 20
SW-SWIFT(config-vlan)#name IOT
SW-SWIFT(config-vlan)#vlan 30
SW-SWIFT(config-vlan)#name USER
SW-SWIFT(config-vlan)#vlan 40
SW-SWIFT(config-vlan)#name IOT_DEVICE
SW-SWIFT(config-vlan)#end
SW-SWIFT#
%SYS-5-CONFIG_I: Configured from console by console

SW-SWIFT#wr
Building configuration...
[OK]
SW-SWIFT#
```

Gambar 5.3.8.1. Konfigurasi VLAN pada Switch

- Konfigurasi Port Router

  Karena Router terhubung ke Fa0/1

```
SW-SWIFT(config)#interface fa0/1
SW-SWIFT(config-if)#switchport mode trunk
SW-SWIFT(config-if)#exit
```

Gambar 5.3.8.2. Konfigurasi Port pada Router

- Port ADMIN-PC

  Karena ADMIN-PC terhubung ke Fa2/1

```
SW-SWIFT(config)#interface fa2/1
SW-SWIFT(config-if)#switchport mode access
SW-SWIFT(config-if)#switchport access vlan 10
SW-SWIFT(config-if)#exit
```

Gambar 5.3.8.3. Konfigurasi Port pada PC ADMIN

- Port SERVER

  Karena Server terhubung ke Fa3/1

```
SW-SWIFT(config)#interface fa3/1
SW-SWIFT(config-if)#switchport mode access
SW-SWIFT(config-if)#switchport access vlan 20
SW-SWIFT(config-if)#exit
```

Gambar 5.3.8.4. Konfigurasi Port pada SERVER

- Port AP

  Karena AP terhubung ke Fa4/1

```
SW-SWIFT(config)#interface fa4/1
SW-SWIFT(config-if)#switchport mode access
SW-SWIFT(config-if)#switchport access vlan 30
SW-SWIFT(config-if)#exit
```

Gambar 5.3.8.5. Konfigurasi Port pada Access Point

- Port ESP32

  Karena ESP32 terhubung ke Fa5/1

```
SW-SWIFT(config)#interface fa5/1
SW-SWIFT(config-if)#switchport mode access
SW-SWIFT(config-if)#switchport access vlan 40
SW-SWIFT(config-if)#exit
```

Gambar 5.3.8.6. Konfigurasi Port pada ESP32

- Hasil konfigurasi VLAN

```
SW-SWIFT#show vlan brief

VLAN Name                                Status    Ports
---- ----------------------------------- --------- -------
1    default                             active
10   ADMIN                               active    Fa2/1
20   IOT                                 active    Fa3/1
30   USER                                active    Fa4/1
40   IOT_DEVICE                          active    Fa5/1
1002 fddi-default                        active
1003 token-ring-default                  active
1004 fddinet-default                     active
1005 trnet-default                       active
SW-SWIFT#
```

Gambar 5.3.8.7. Hasil Konfigurasi VLAN

Virtual Local Area Network (VLAN) digunakan untuk membagi jaringan fisik menjadi beberapa jaringan logis yang terpisah. Pada sistem SWIFT, VLAN diterapkan untuk meningkatkan keamanan, mempermudah manajemen jaringan,

serta mengurangi lalu lintas broadcast antar perangkat. Setiap perangkat ditempatkan pada VLAN yang sesuai dengan fungsinya sehingga komunikasi jaringan menjadi lebih terstruktur.

Konfigurasi VLAN dilakukan dengan membuat VLAN 10 (ADMIN), VLAN 20 (SERVER), VLAN 30 (USER), dan VLAN 40 (IoT). Setelah VLAN dibuat, masing-masing port switch dikonfigurasi sebagai access port dan ditempatkan pada VLAN yang sesuai. Port yang terhubung ke router dikonfigurasi sebagai trunk port agar dapat membawa lalu lintas dari seluruh VLAN menuju router untuk proses routing antar VLAN.

Hasil konfigurasi menunjukkan bahwa seluruh perangkat telah tergabung pada VLAN masing-masing dan siap melakukan komunikasi sesuai kebijakan jaringan yang diterapkan.

5.3.9 Konfigurasi Router-on-a-Stick pada R-SWIFT

- Aktifkan interface utama

```
R-SWIFT>en
R-SWIFT#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R-SWIFT(config)#interface g0/0
R-SWIFT(config-if)#no shutdown
R-SWIFT(config-if)#exit
R-SWIFT(config)#
```

Gambar 5.3.9.1. Mengaktifkan Interface utama pada Router

- VLAN 10

```
R-SWIFT(config)#interface g0/0.10
R-SWIFT(config-subif)#encapsulation dot1Q 10
R-SWIFT(config-subif)#ip address 192.168.10.1 255.255.255.240
R-SWIFT(config-subif)#exit
```

Gambar 5.3.9.2. Mengaktifkan Interface vlan 10 pada Router

- VLAN 20

```
R-SWIFT(config)#interface g0/0.20
R-SWIFT(config-subif)#encapsulation dot1Q 20
R-SWIFT(config-subif)#ip address 192.168.20.1 255.255.255.240
R-SWIFT(config-subif)#exit
```

Gambar 5.3.9.3. Mengaktifkan Interface vlan 20 pada Router

- VLAN 30

```
R-SWIFT(config)#interface g0/0.30
R-SWIFT(config-subif)#encapsulation dot1Q 30
R-SWIFT(config-subif)#ip address 192.168.30.1 255.255.255.240
R-SWIFT(config-subif)#exit
```

Gambar 5.3.9.4. Mengaktifkan Interface vlan 30 pada Router

- VLAN 40

```
R-SWIFT(config)#interface g0/0.40
R-SWIFT(config-subif)#encapsulation dot1Q 40
R-SWIFT(config-subif)#ip address 192.168.40.1 255.255.255.240
R-SWIFT(config-subif)#exit
```

Gambar 5.3.9.5. Mengaktifkan Interface vlan 40 pada Router

- Hasil Konfigurasi

```
R-SWIFT#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/0.30  192.168.30.1    YES manual up                    up
GigabitEthernet0/0.40  192.168.40.1    YES manual up                    up
GigabitEthernet1/0     unassigned      YES unset  administratively down down
Serial2/0              unassigned      YES unset  administratively down down
GigabitEthernet8/0     unassigned      YES unset  administratively down down
GigabitEthernet9/0     unassigned      YES unset  administratively down down
R-SWIFT#
```

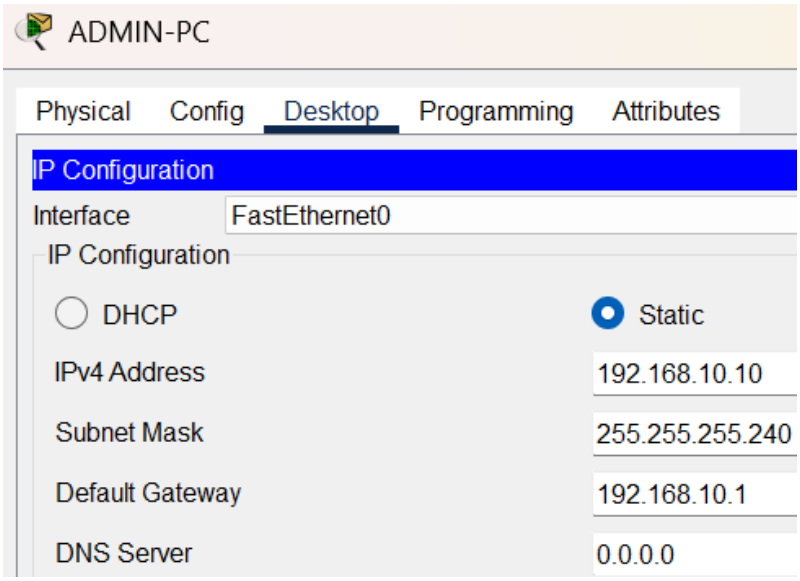
Gambar 5.3.9.6. Hasil Konfigurasi Router-on-a-Stick pada R-SWIFT

Metode Router-on-a-Stick digunakan untuk memungkinkan komunikasi antar VLAN menggunakan satu antarmuka fisik router. Pada konfigurasi ini dibuat beberapa subinterface yang masing-masing mewakili VLAN tertentu.

Langkah konfigurasi diawali dengan mengaktifkan interface utama yang terhubung ke switch. Selanjutnya dibuat subinterface untuk VLAN 10, VLAN 20, VLAN 30, dan VLAN 40 dengan menambahkan encapsulation dot1Q sesuai nomor VLAN serta memberikan alamat IP sebagai gateway bagi setiap VLAN.

Melalui konfigurasi ini, router dapat melakukan proses inter-VLAN routing sehingga perangkat yang berada pada VLAN berbeda dapat saling berkomunikasi sesuai aturan keamanan yang diterapkan. Hasil pengujian menunjukkan bahwa proses routing antar VLAN berjalan dengan baik dan gateway pada setiap VLAN dapat diakses.

5.3.10 Konfigurasi PC-ADMIN PC

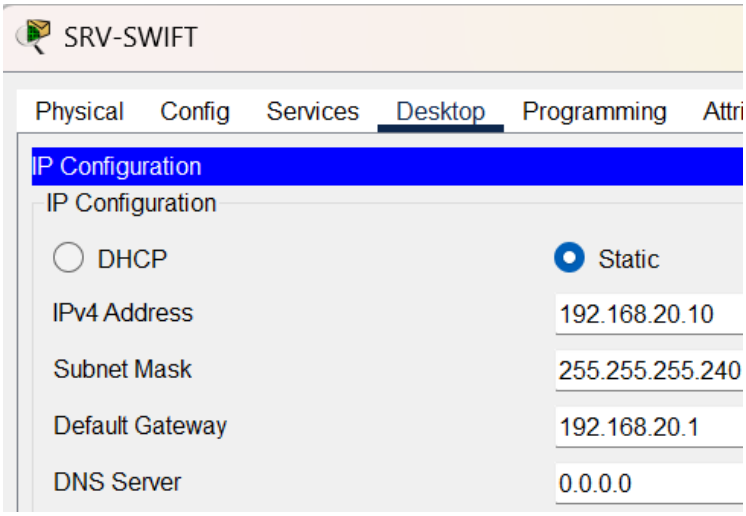


Gambar 5.3.10.1. Konfigurasi PC ADMIN

PC ADMIN berfungsi sebagai perangkat administrator yang digunakan untuk melakukan pengelolaan dan pemantauan jaringan SWIFT. Konfigurasi dilakukan dengan memberikan alamat IP statis sesuai subnet VLAN 10 serta mengatur default gateway menuju alamat gateway VLAN ADMIN pada router.

Dengan konfigurasi tersebut, PC ADMIN dapat mengakses seluruh layanan jaringan yang diperlukan, termasuk server SWIFT dan perangkat IoT. Selain itu, PC ADMIN juga memiliki hak akses yang lebih luas dibandingkan perangkat pengguna biasa karena digunakan untuk keperluan administrasi dan pemeliharaan sistem.

5.3.11 Konfigurasi Server-SRV SWIFT

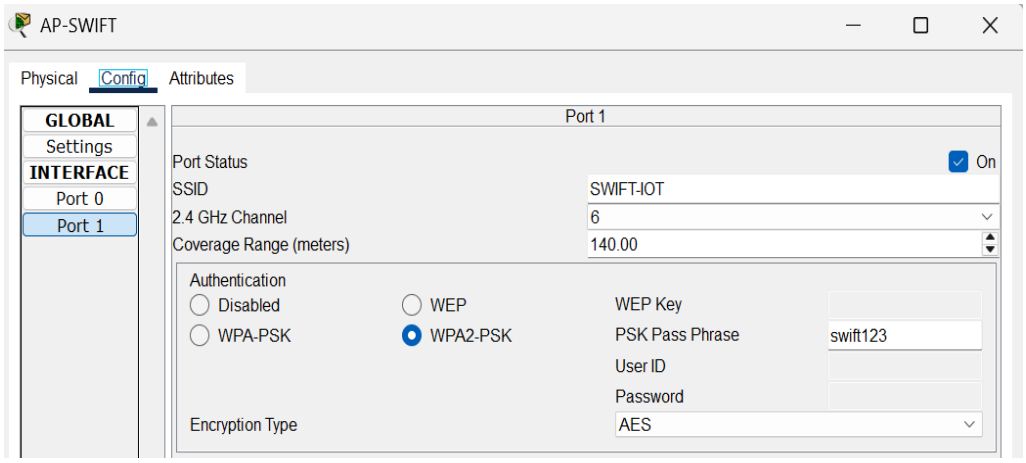


Gambar 5.3.11.1. 1Konfigurasi Server SWIFT

Server SWIFT berfungsi sebagai pusat layanan yang menyediakan penyimpanan data, monitoring, dan akses dashboard sistem. Konfigurasi server dilakukan dengan memberikan alamat IP statis pada VLAN SERVER agar alamat server tetap konsisten dan mudah diakses oleh administrator maupun aplikasi monitoring.

Selain pengaturan alamat IP, server juga dikonfigurasi untuk menyediakan layanan yang dibutuhkan oleh sistem SWIFT sehingga data yang dikirim dari perangkat IoT dapat diproses dan ditampilkan kepada pengguna. Dengan konfigurasi tersebut, server dapat beroperasi sebagai pusat komunikasi dan penyimpanan data pada jaringan SWIFT.

5.3.12 Konfigurasi Access Point-AP SWIFT



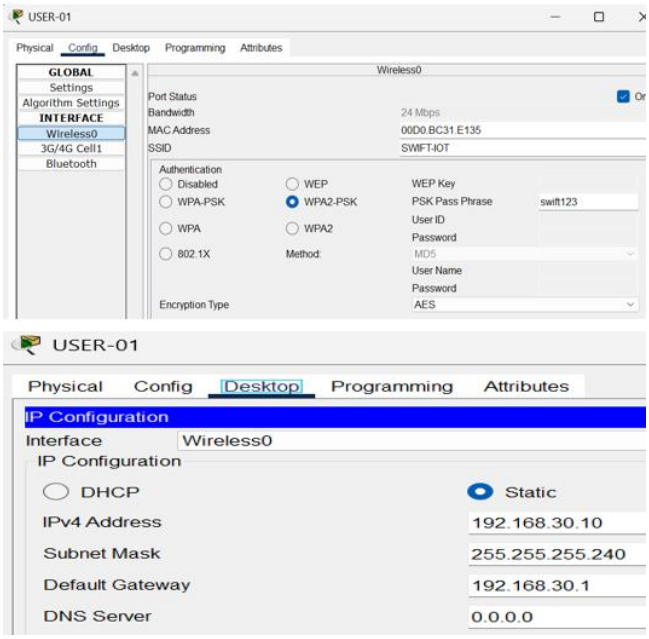
Gambar 5.3.12.1. Konfigurasi AP SWIFT

Access Point (AP) digunakan untuk menyediakan konektivitas nirkabel bagi perangkat pengguna yang akan mengakses sistem SWIFT. Konfigurasi dilakukan dengan menentukan nama jaringan (SSID), pengaturan keamanan Wi-Fi, serta menghubungkan access point ke VLAN USER melalui switch.

Melalui konfigurasi ini, smartphone pengguna dapat terhubung ke jaringan SWIFT secara nirkabel tanpa menggunakan koneksi kabel. Access point berfungsi sebagai media komunikasi antara perangkat pengguna dan infrastruktur jaringan yang telah dirancang.

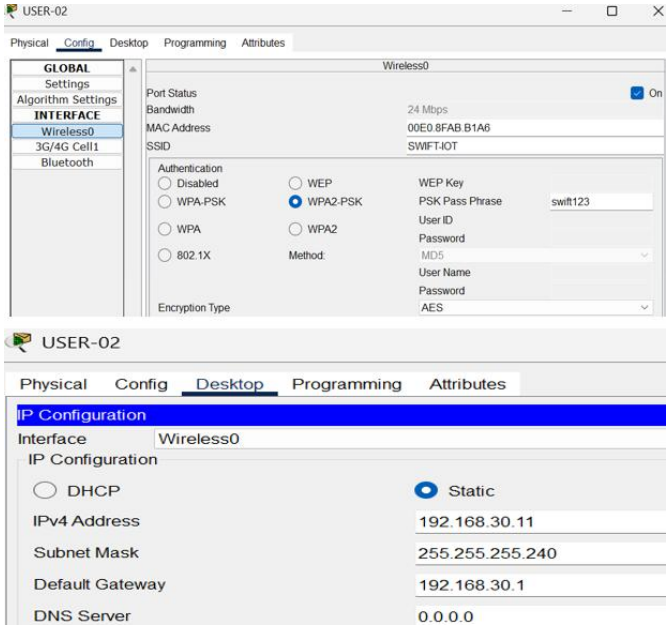
5.3.13 Konfigurasi Smartphone

- User 01



Gambar 5.3.13.1. Konfigurasi User 01 pada Smartphone

- User 02



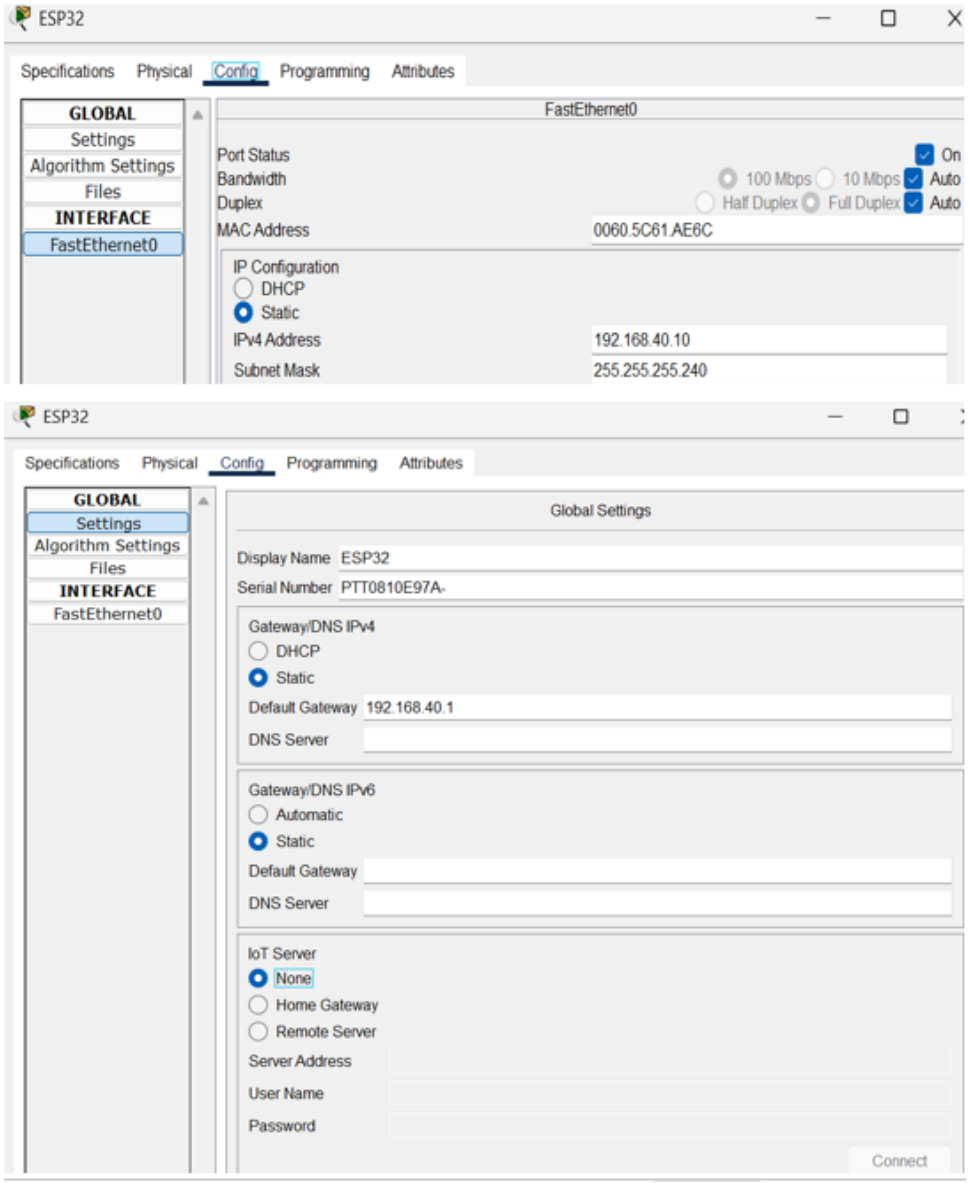
Gambar 5.3.13.2. Konfigurasi User 02 pada Smartphone

Smartphone User 01 dan User 02 dikonfigurasi sebagai perangkat klien yang terhubung ke jaringan Wi-Fi SWIFT melalui access point. Konfigurasi dilakukan dengan memilih SSID yang telah dibuat serta memperoleh alamat IP sesuai jaringan VLAN USER.

Setelah berhasil terhubung ke jaringan, smartphone dapat melakukan komunikasi dengan perangkat lain sesuai kebijakan keamanan yang diterapkan. Smartphone digunakan sebagai representasi pengguna yang mengakses layanan monitoring SWIFT melalui jaringan nirkabel.

5.3.14 Konfigurasi ESP32

- Konfigurasi Network ESP32-SWIFT

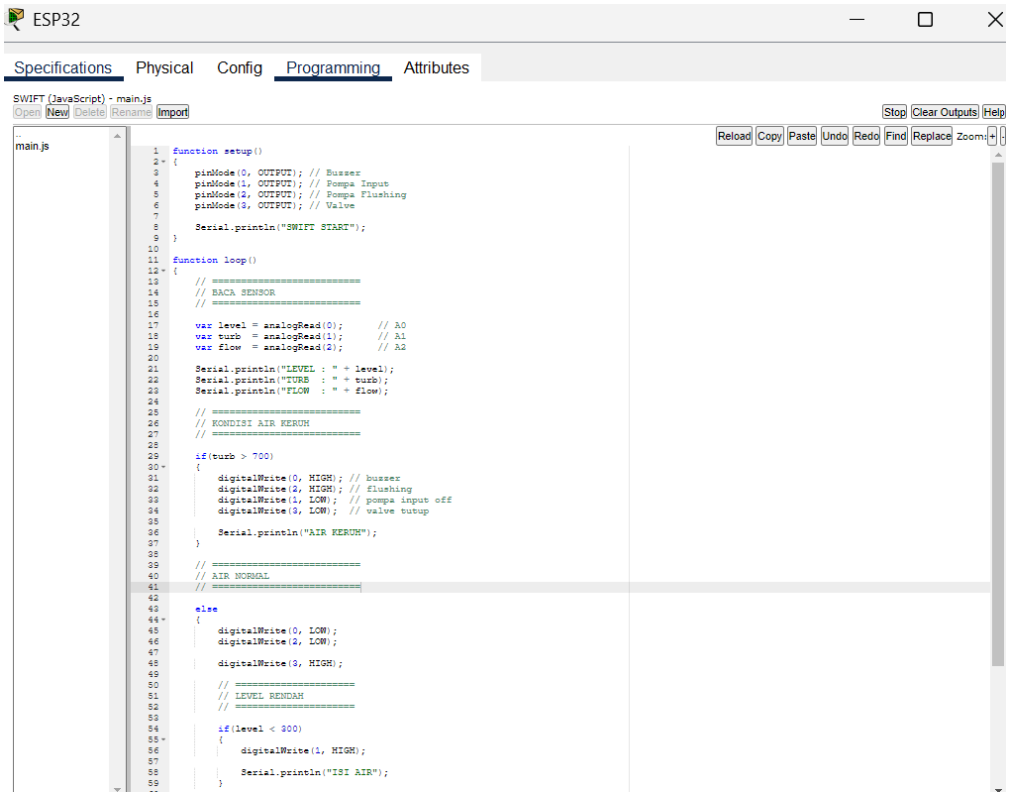


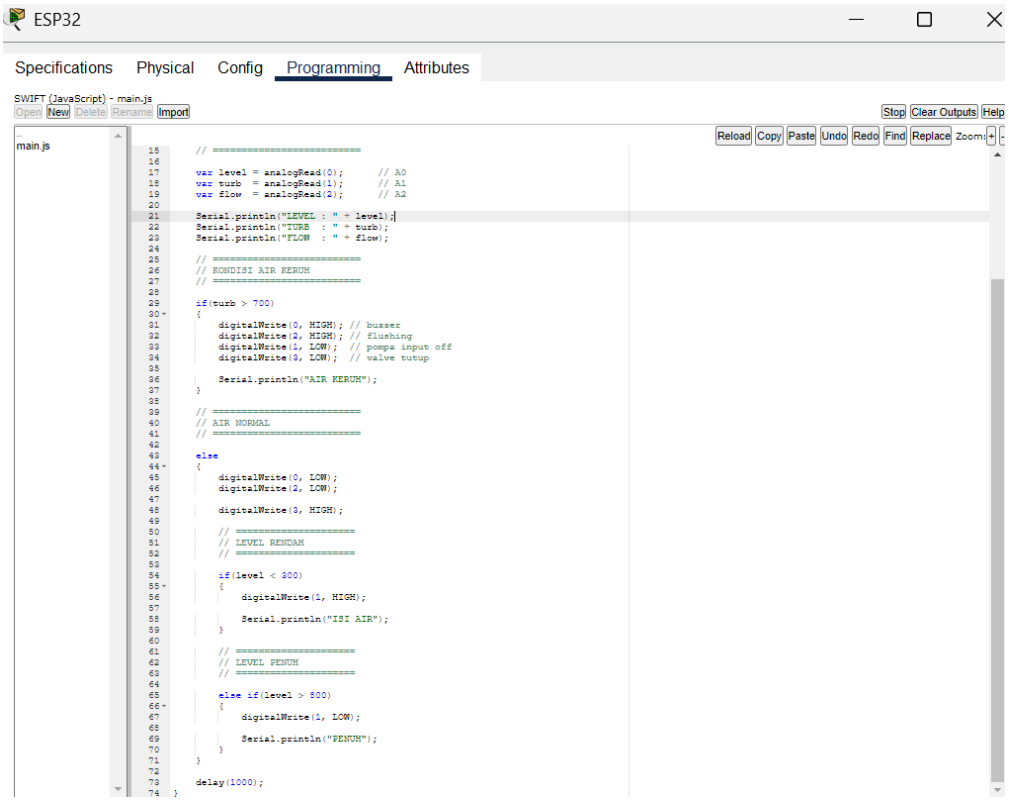
Gambar 5.3.14.1. Konfigurasi Network ESP32-SWIFT

ESP32 merupakan perangkat IoT utama pada sistem SWIFT yang bertugas membaca data sensor dan mengirimkan hasil pemantauan ke sistem monitoring. Konfigurasi jaringan dilakukan dengan menentukan SSID, password Wi-Fi, alamat server tujuan, serta parameter komunikasi yang diperlukan.

Melalui konfigurasi tersebut, ESP32 dapat terhubung ke jaringan SWIFT dan mengirimkan data hasil pembacaan sensor secara real-time. Penempatan ESP32 pada VLAN IoT juga meningkatkan keamanan karena lalu lintas perangkat IoT dipisahkan dari perangkat pengguna.

- Program SWIFT pada ESP32 (MCU-PT)





Gambar 5.3.14.2. Program ESP32-SWIFT

Program yang ditanamkan pada ESP32 berfungsi untuk mengelola seluruh proses sistem monitoring kualitas air. Program melakukan pembacaan sensor turbidity, water flow, dan sensor ultrasonik, kemudian mengolah data tersebut untuk menentukan kondisi sistem.

Selain melakukan pemrosesan data sensor, program juga mengendalikan aktuator seperti pompa, relay, buzzer, dan LCD serta mengirimkan data hasil monitoring ke dashboard SWIFT. Dengan program tersebut, proses pemantauan dan pengendalian sistem dapat berjalan secara otomatis dan real-time.

5.4 Desain Keamaan

5.4.1 Konfigurasi Access Control List (ACL)

- Pembuatan ACL

```
R-SWIFT>en
R-SWIFT#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R-SWIFT(config)#access-list 100 permit ip 192.168.10.0 0.0.0.15 any
R-SWIFT(config)#access-list 100 deny ip 192.168.30.0 0.0.0.15 192.168.20.0 0.0.0.15
R-SWIFT(config)#access-list 100 permit ip any any
R-SWIFT(config)#
```

Gambar 5.4.1.1. Konfigurasi pembuatan ACL pada keamanan jaringan SWIFT

- Pemasangan ACL

  Karena ACL harus dipasang pada VLAN USER:

```
R-SWIFT(config)#interface g0/0.30
R-SWIFT(config-subif)#ip access-group 100 in
R-SWIFT(config-subif)#exit
```

Gambar 5.4.1.2. 1Konfigurasi pemasangan ACL pada keamanan jaringan SWIFT

- Hasil Konfigurasi ACL pada keamanan jaringan SWIFT

```
R-SWIFT#show access-lists
Extended IP access list 100
    10 permit ip 192.168.10.0 0.0.0.15 any
    20 deny ip 192.168.30.0 0.0.0.15 192.168.20.0 0.0.0.15 (22 match(es))
    30 permit ip any any (19 match(es))

R-SWIFT#
```

Gambar 5.4.1.3. Konfigurasi ACL pada keamanan jaringan SWIFT

Access Control List (ACL) digunakan sebagai mekanisme keamanan untuk mengatur hak akses antar VLAN. Pada sistem SWIFT, ACL diterapkan pada VLAN USER untuk membatasi akses pengguna terhadap sumber daya tertentu di dalam jaringan.

Konfigurasi ACL dilakukan dengan membuat aturan yang menolak akses dari VLAN USER menuju VLAN SERVER, kemudian mengizinkan akses ke tujuan lain yang diperlukan. Setelah ACL dibuat, aturan tersebut dipasang pada interface yang sesuai pada router.

Hasil konfigurasi menunjukkan bahwa perangkat pengguna tidak dapat mengakses server secara langsung, sedangkan administrator tetap memiliki akses penuh terhadap seluruh layanan jaringan. Penerapan ACL ini berhasil meningkatkan keamanan jaringan SWIFT dengan membatasi akses yang tidak diperlukan.

5.4.2 Konfigurasi Port Security

- Pasang pada port ADMIN

```
SW-SWIFT>en
SW-SWIFT#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
SW-SWIFT(config)#interface fa2/1
SW-SWIFT(config-if)#switchport mode access
SW-SWIFT(config-if)#switchport port-security
SW-SWIFT(config-if)#switchport port-security maximum 1
SW-SWIFT(config-if)#switchport port-security violation shutdown
SW-SWIFT(config-if)#switchport port-security mac-address sticky
SW-SWIFT(config-if)#end
SW-SWIFT#
%SYS-5-CONFIG_I: Configured from console by console

SW-SWIFT#wr
Building configuration...
[OK]
SW-SWIFT#
```

Gambar 5.4.2.1. Konfigurasi Port Security pada PC ADMIN

- Hasil Konfigurasi

```
SW-SWIFT#show port-security
Secure Port MaxSecureAddr CurrentAddr SecurityViolation Security Action
               (Count)       (Count)        (Count)
---------------------------------------------------------------------
        Fa2/1        1           0                0          Shutdown
---------------------------------------------------------------------
SW-SWIFT#
```

Gambar 5.4.2.2. Hasil Konfigurasi Port Security pada PC ADMIN

Port Security digunakan untuk meningkatkan keamanan akses fisik pada switch. Fitur ini diterapkan pada port yang terhubung ke PC ADMIN untuk membatasi jumlah perangkat yang diperbolehkan menggunakan port tersebut.

Konfigurasi dilakukan dengan menetapkan satu alamat MAC yang diizinkan pada port administrator. Jika terdapat perangkat lain yang mencoba menggunakan port tersebut, switch akan melakukan tindakan sesuai kebijakan yang telah ditentukan, seperti menolak akses atau menonaktifkan port.

Hasil konfigurasi menunjukkan bahwa hanya PC ADMIN yang dapat menggunakan port tersebut sehingga risiko akses tidak sah melalui jaringan fisik dapat diminimalkan.

5.5 Simulasi dan Dokumentasi

5.5.1 Test VLAN

```
SW-SWIFT>en
SW-SWIFT#show vlan brief

VLAN Name                                Status    Ports
---- ----------------------------------- --------- -------
1    default                             active
10   ADMIN                               active    Fa2/1
20   IOT                                 active    Fa3/1
30   USER                                active    Fa4/1
40   IOT_DEVICE                          active    Fa5/1
1002 fddi-default                        active
1003 token-ring-default                  active
1004 fddinet-default                     active
1005 trnet-default                       active
SW-SWIFT#
```

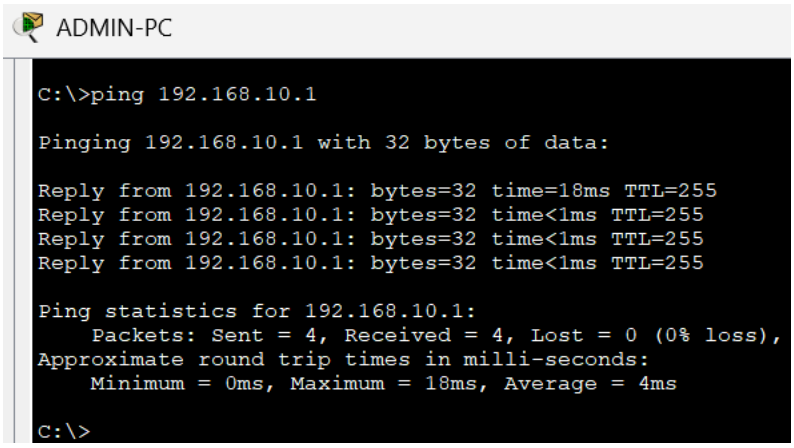
Gambar 5.5.1.1. Hasil Konfigurasi VLAN

5.5.2 Test ROUTING

```
R-SWIFT>en
R-SWIFT#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/0.30  192.168.30.1    YES manual up                    up
GigabitEthernet0/0.40  192.168.40.1    YES manual up                    up
GigabitEthernet1/0     unassigned      YES unset  administratively down down
Serial2/0              unassigned      YES unset  administratively down down
GigabitEthernet8/0     unassigned      YES unset  administratively down down
GigabitEthernet9/0     unassigned      YES unset  administratively down down
R-SWIFT#
```

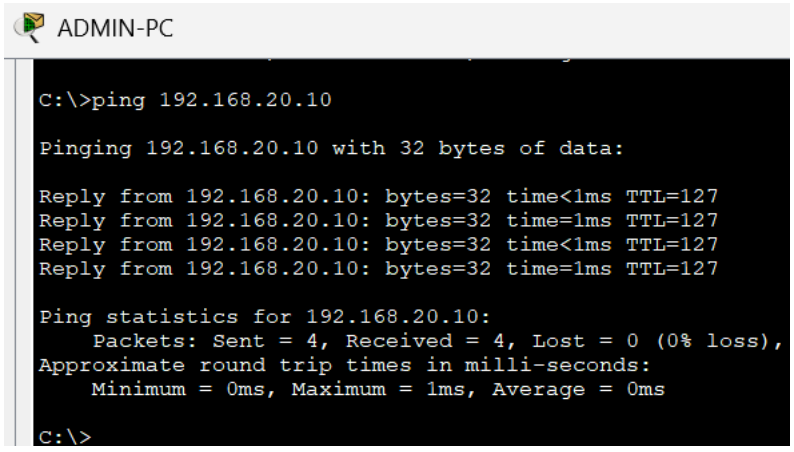
Gambar 5.5.2.1. Hasil Konfigurasi Routing

5.5.3 Test Ping PC ADMIN ke Router



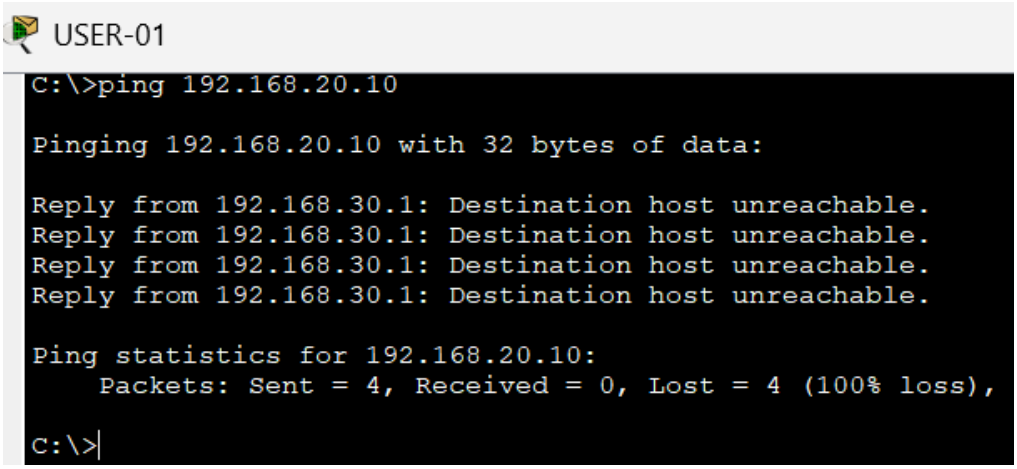
Gambar 5.5.3.1. Hasil Ping PC ADMIN ke Router

5.5.4 Test Ping PC ADMIN ke Server



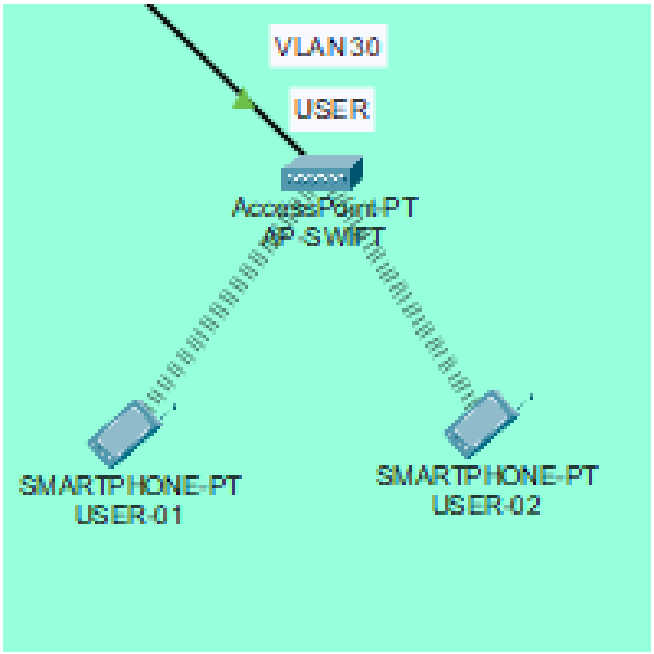
Gambar 5.5.4.1. Hasil Ping PC ADMIN ke Server

5.5.5 Test Ping User 01 ke Server (Jika ACL aktif, seharusnya Request Timed Out)



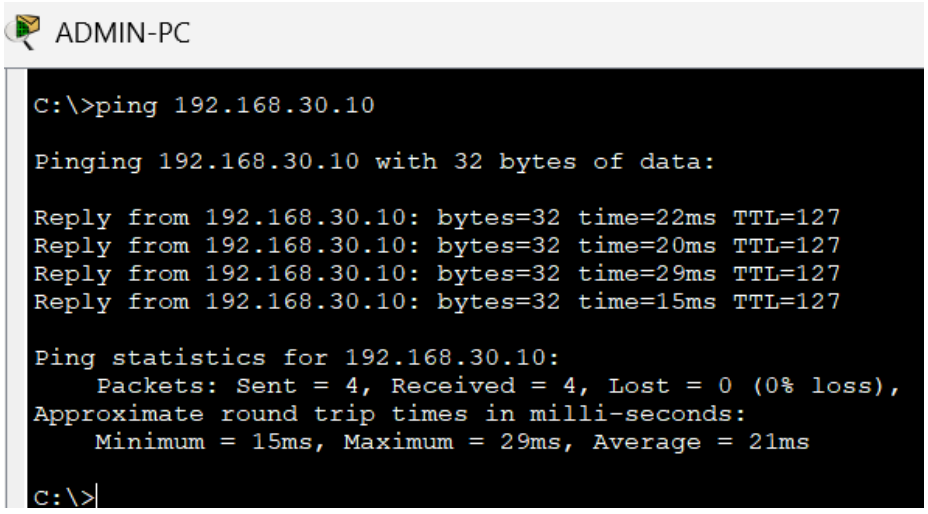
Gambar 5.5.5.1. Hasil Ping User 01 ke Server

5.5.6 Test Ping User 01 dan 02 ke Access Point



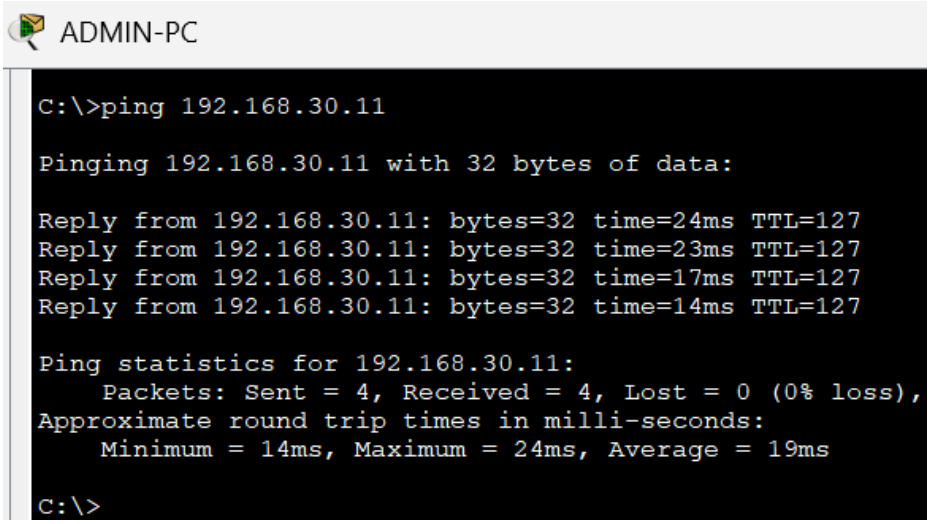
Gambar 5.5.6.1. Hasil User 01 dan User 02 berhasil terhubung ke AP

5.5.7 Test Ping PC Admin ke User 01



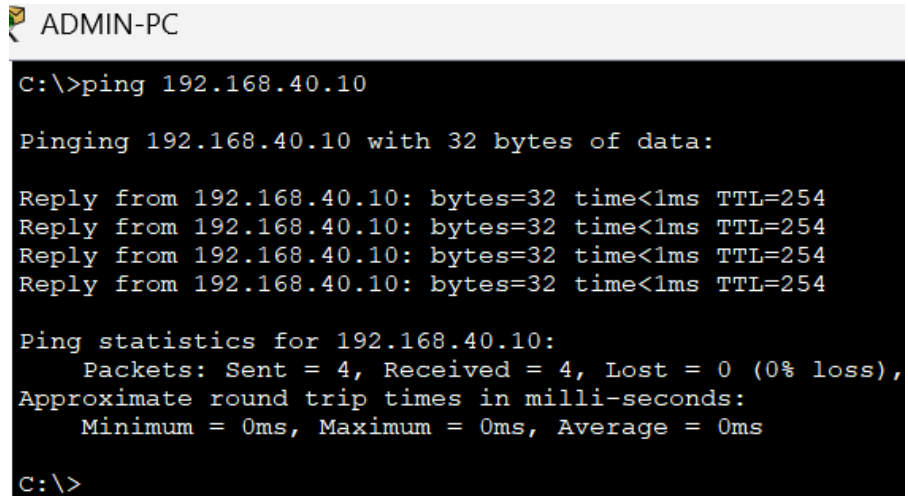
Gambar 5.5.7.1. Hasil Ping PC Admin ke User 01

5.5.8 Test Ping PC Admin ke User 02



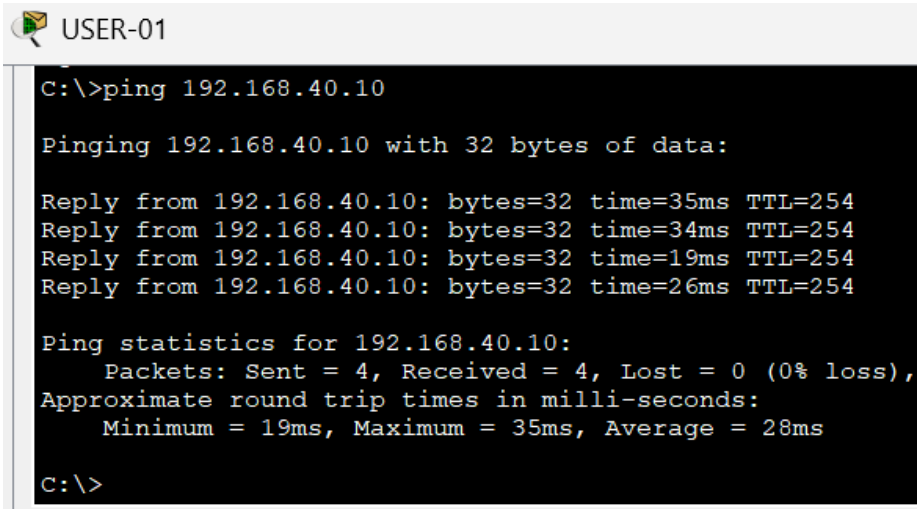
Gambar 5.5.8.1. Hasil Ping PC Admin ke User 02

5.5.9 Test Ping PC Admin ke ESP32



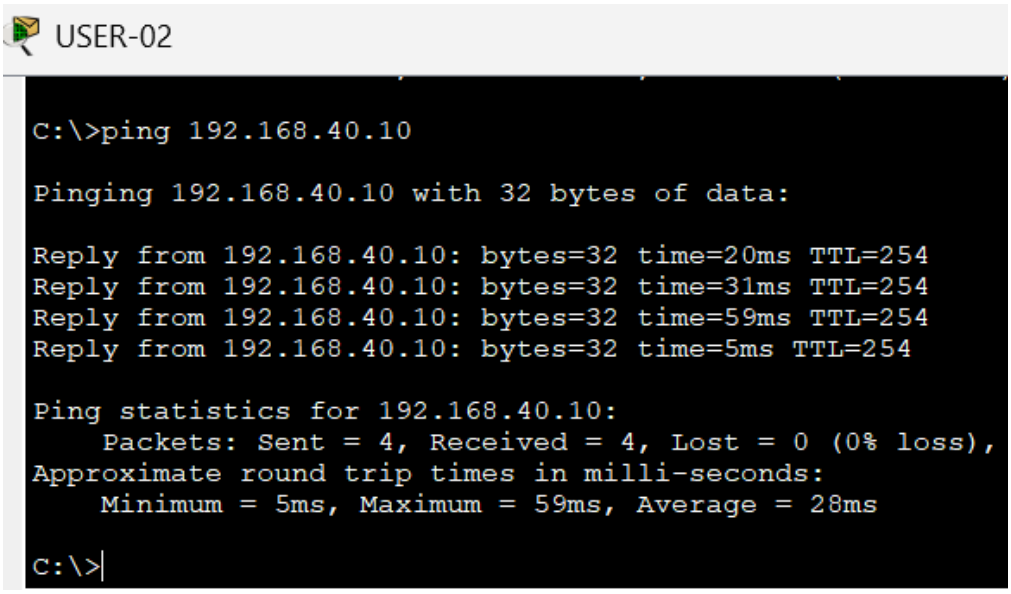
Gambar 5.5.9.1. Hasil Ping PC Admin ke ESP32

5.5.10 Test Ping User 01 ke ESP32



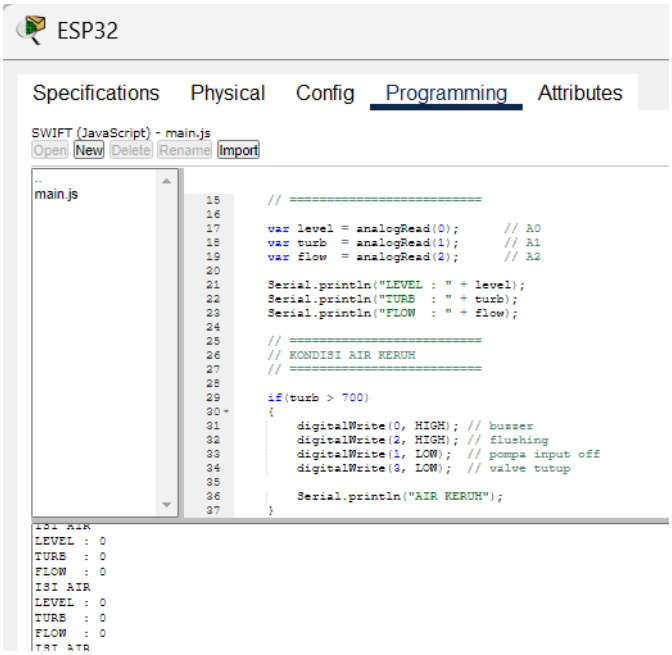
Gambar 5.5.10.1. Hasil Ping User 01 ke ESP32

5.5.11 Test Ping User 02 ke ESP32



Gambar 5.5.11.1. Hasil Ping User 02 ke ESP32

5.5.12 Test Ping User 02 ke ESP32



Gambar 5.5.12.1. Hasil Serial output ESP32

Pengujian dilakukan untuk memastikan seluruh konfigurasi jaringan yang telah dirancang dapat berjalan sesuai dengan kebutuhan sistem. Pengujian VLAN

menunjukkan bahwa perangkat berhasil ditempatkan pada VLAN masing-masing sesuai desain jaringan.

Pengujian routing membuktikan bahwa Router-on-a-Stick mampu melakukan komunikasi antar VLAN melalui gateway yang telah dikonfigurasi. Pengujian konektivitas antara PC ADMIN, server, access point, smartphone, dan ESP32 menunjukkan bahwa seluruh perangkat dapat saling berkomunikasi sesuai kebijakan yang berlaku.

Pada pengujian ACL, smartphone pengguna tidak dapat mengakses server secara langsung sehingga menghasilkan status Request Timed Out. Hasil ini menunjukkan bahwa ACL telah bekerja sesuai tujuan keamanan yang dirancang. Sementara itu, administrator tetap dapat mengakses server dan perangkat IoT tanpa kendala.

Secara keseluruhan, hasil simulasi menunjukkan bahwa segmentasi VLAN, routing antar VLAN, ACL, dan Port Security telah berfungsi dengan baik dan mampu mendukung kebutuhan operasional jaringan SWIFT. Hasil pengujian program ESP32 menunjukkan bahwa sistem mampu membaca data dari sensor water flow, sensor turbidity, dan sensor ultrasonik secara real-time. Data yang diperoleh kemudian diproses oleh ESP32 untuk menentukan kondisi kualitas air dan mengendalikan aktuator sesuai logika program yang telah dirancang.

Pompa, relay, buzzer, dan LCD berhasil beroperasi sesuai kondisi yang terdeteksi oleh sensor. Selain itu, ESP32 juga mampu mengirimkan data monitoring melalui jaringan Wi-Fi sehingga informasi kondisi sistem dapat ditampilkan pada dashboard SWIFT.

Berdasarkan hasil pengujian tersebut, dapat disimpulkan bahwa program ESP32 telah berjalan sesuai fungsi yang direncanakan dan mampu mendukung proses monitoring kualitas air secara otomatis.

BAB VI
PENUTUP

6.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem jaringan SWIFT berhasil dibangun menggunakan konsep segmentasi VLAN, Router-on-a-Stick, Access Control List (ACL), dan Port Security untuk meningkatkan keamanan serta efisiensi pengelolaan jaringan.

Penerapan VLAN berhasil memisahkan perangkat berdasarkan fungsi operasionalnya menjadi VLAN ADMIN, SERVER, USER, dan IoT sehingga lalu lintas jaringan menjadi lebih terorganisasi. Router-on-a-Stick mampu menyediakan komunikasi antar VLAN secara efektif, sedangkan ACL berhasil membatasi akses pengguna terhadap server sesuai kebijakan keamanan yang ditetapkan. Selain itu, Port Security mampu memberikan perlindungan tambahan terhadap akses fisik yang tidak sah pada perangkat administrator.

Dari sisi Internet of Things (IoT), perangkat ESP32 berhasil melakukan pembacaan data sensor, pengolahan informasi kualitas air, pengendalian aktuator, serta pengiriman data monitoring melalui jaringan. Hasil pengujian menunjukkan bahwa seluruh komponen sistem dapat beroperasi secara terintegrasi dan mendukung proses monitoring kualitas air secara real-time.

6.2 Saran

a. Menambahkan sensor kualitas air lainnya, seperti sensor pH, TDS, atau suhu air untuk meningkatkan akurasi monitoring kondisi air.

b. Mengembangkan sistem notifikasi jarak jauh melalui aplikasi pesan instan atau email agar pengguna dapat menerima peringatan secara langsung ketika terjadi kondisi abnormal.

c. Meningkatkan keamanan dashboard melalui penerapan sistem autentikasi yang lebih kuat dan enkripsi komunikasi data.

d. Mengembangkan fitur penyimpanan data historis yang lebih terstruktur untuk mendukung proses analisis penggunaan air jangka panjang.

e. Melakukan implementasi sistem pada skala yang lebih besar agar dapat diterapkan secara nyata pada fasilitas pengelolaan air di lingkungan kampus maupun sektor lainnya.

DAFTAR PUSTAKA

[1] Capasitor Tech, "Tutorial Cara Program Water Flow Sensor Jumlah dan Kecepatan Air YF S201 dengan ESP32 dan Blynk IoT" *YouTube*. [Video]. [Online]. Tersedia: https://youtu.be/8LW8ea080Oo.

[2] E. M. Mase, F. W. Witi, dan B. Y. Bhae, *"Optimization of Water Level Monitoring in Water Storage Tanks Using the Internet of Things (IoT) at Universitas Flores," Jurnal Ilmiah Sistem Informasi (JUISI)*, vol. 4, no. 1, hal. 1–10, Jan. 2025, doi: 10.51903/4xg62v92. [Online]. Tersedia: https://ejurnal.provisi.ac.id/index.php/JUISI/article/view/1283.

[3] H. M. Yasin, S. R. M. Zeebaree, M. A. M. Sadeeq, S. Y. Ameen, I. M. Ibrahim, R. R. Zebari, R. K. Ibrahim, dan A. B. Sallow, "IoT and ICT Based Smart Water Management, Monitoring and Controlling System: A Review," *Asian Journal of Research in Computer Science (AJRCOS)*, vol. 8, no. 2, hal. 42–56, 2021, doi: 10.9734/ajrcos/2021/v8i230198. [Online]. Tersedia: https://journalajrcos.com/index.php/AJRCOS/article/view/421.

[4] J. Anggara, Nehru, dan Y. R. Hais, "Rancang Bangun Sistem Monitoring dan Kontrol Penggunaan Air PDAM Berbasis Internet of Things," *Physics and Science Education Journal (PSEJ)*, vol. 3, no. 2, hal. 88–104, 2023, doi: 10.30631/psej.v3i2.1866. [Online]. Tersedia: https://e-journal.lp2m.uinjambi.ac.id/ojp/index.php/psej/article/view/1866.

[5] L. A. Swarga, L. T. Swarga, dan B. Hariadi, "Sistem Deteksi Kebocoran Pipa Air Menggunakan Sensor Flow dan IoT dengan Algoritma ANFIS," *Jurnal FORTECH*, vol. 7, no. 1, hal. 1–7, Mar. 2026, doi: 10.56795/fortech.v7i1.7101. [Online]. Tersedia: https://journal.fortei7.org/index.php/fortech/article/view/782.

[6] M. A. Wahid dan M. Noman, "Integrated IoT Based Water Quality and Quantity Monitoring System," *Journal of Mechanics of Continua and Mathematical Sciences (JMCMS)*, vol. 15, no. 11, Nov. 2020, doi: 10.26782/jmcms.2020.11.00008. [Online]. Tersedia: https://www.journalimcms.org/journal/integrated-iot-based-water-quality-and-quantity-monitoring-system/.

[7] R. D. Vinchi, M. Muhtarom, dan N. F. Muhammad, "Utilization of IoT System for Water Discharge Monitoring at PAMSIMAS Dukuh Jetak," *Jurnal Informatika dan Teknik*

Elektro Terapan (JITET), vol. 13, no. 3, 2025, doi: 10.23960/jitet.v13i3.7136. [Online]. Tersedia: https://journal.eng.unila.ac.id/index.php/jitet/article/view/7136.

[8] robo_army_, "DIY Water Level Detection System" *YouTube Shorts*. [Video]. [Online]. Tersedia: https://youtube.com/shorts/MNxw2Y85WyQ.

[9] Sensorim, "Water flow Sensor | YF – S402b Explained," *YouTube Shorts*. [Video]. [Online]. Tersedia: https://youtube.com/shorts/dpkgNERx8qc.

[10] T. V. Kumar dan S. K. Masthan, "IoT-Based Smart Water Flow Monitoring and Pipeline Leakage Detection System Using Arduino UNO," *International Journal of Information and Electronics Engineering (IJIEE)*, vol. 15, no. 7, 2025, doi: 10.48047/py50hs96. [Online]. Tersedia: https://ijiee.org/index.php/ijiee/article/view/1037.

[11] W. Nugroho, M. J. F. Arifianto, A. Afianto, A. Wicaksono, dan N. Nursim, "Web Based IoT Monitoring System for Ultrasonic Water Flow Measurement Using ESP32-S3 and Cloud Database," *Journal of Soft Computing Exploration (JOSCEX)*, vol. 6, no. 4, hal. 258–265, Des. 2025, doi: 10.52465/joscex.v6i4.625. [Online]. Tersedia: https://shmpublisher.com/index.php/joscex/article/view/625.

LAMPIRAN

A. Listing Kode Program

SWIFT_IoT_FINAL_kel1_FIX.ino

```
#include <PubSubClient.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ============================================================
// KONFIGURASI — SESUAIKAN DI SINI
// ============================================================
const char* WIFI_SSID      = "UGM-Hotspot";      // Nama WiFi kampus
const char* WIFI_PASS      = "";                 // Kosong jika open network
const char* MQTT_SERVER    = "10.33.102.177";    // IP VPS kampus
const int   MQTT_PORT      = 1883;
const char* MQTT_CLIENT    = "ESP32_SWIFT_01";
const char* MQTT_USER      = "esp32_swift";      // Username MQTT broker
const char* MQTT_PASS_AUTH = "swift123";         // Password MQTT broker
const char* MQTT_TOPIC_PUB = "swift/iot/data";
const char* MQTT_TOPIC_SUB = "swift/control";

// ============================================================
// PARAMETER WADAH — SUDAH DIISI SESUAI KONFIRMASI
// ============================================================
#define TINGGI_WADAH   15.0   // cm — tinggi total wadah fisik
#define JARAK_PENUH     3.0   // cm — jarak sensor saat wadah PENUH
#define JARAK_KOSONG   13.0   // cm — jarak sensor saat wadah HAMPIR KOSONG
                              //      (sengaja 18 bukan 20 agar pompa tidak dry-run)
#define JARAK_ERROR    98.0   // cm — jarak >= ini = sensor HC-SR04 error

// ============================================================
// DEFINISI PIN
// ============================================================
#define PIN_FLOW       19
#define PIN_TURBIDITY  34
#define PIN_TRIG        5
#define PIN_ECHO       18
#define PIN_POMPA1     26   // Relay CH1 — Pompa Isi Air
#define PIN_POMPA2     27   // Relay CH2 — Pompa Buang/Flushing
#define PIN_VALVE      14   // Relay CH3 — Solenoid Valve 12V
#define PIN_INDIKATOR  13   // Relay CH4 — Indikator 12V aktif
#define PIN_BUZZER     25

// ============================================================
// KONSTANTA SISTEM
// ============================================================
#define RELAY_ON        LOW   // Modul relay aktif LOW
#define RELAY_OFF       HIGH  // Modul relay non-aktif HIGH

#define TURB_AMAN        89.0  // NTU — di bawah ini = AMAN
#define TURB_WASPADA     95.0  // NTU — antara 5-25 = WASPADA, di atas 25 = BAHAYA

#define FLOW_MIN         0.3  // L/min — batas bawah aliran normal
#define FLOW_MAX         5.0  // L/min — batas atas aliran (waspada jika lebih)

#define TIMEOUT_FLOW    15000 // ms — 15 detik tidak ada flow = WASPADA
#define DURASI_FLUSHING 30000 // ms — durasi flushing otomatis maksimal

#define INTERVAL_BACA    2000 // ms — interval pembacaan sensor & kirim data

// ============================================================
// OBJEK GLOBAL
// ============================================================
WiFiClient    espClient;
```

SWIFT_IoT_FINAL_kel1_FIX.ino

```

// ============================================================
// OBJEK GLOBAL
// ============================================================
WiFiClient    espClient;
PubSubClient  mqtt(espClient);
LiquidCrystal_I2C lcd(0x27, 16, 2);
// Jika LCD blank setelah upload → ganti 0x27 menjadi 0x3F di baris atas

// ============================================================
// VARIABEL GLOBAL
// ============================================================
volatile uint32_t pulseCount     = 0;
float   flowRate                 = 0.0;
float   turbidityNTU             = 0.0;
float   jarakCm                  = 0.0;
float   levelPersen              = 0.0;

String  statusSistem             = "AMAN";
String  pesanNotif               = "Monitoring Aktif";
String  statusLama               = "";

unsigned long millisLalu         = 0;
unsigned long waktuFlowTerakhir  = 0;
unsigned long waktuFlushMulai    = 0;

bool sedangFlushing              = false;
bool sistemSiap                  = false;

// ============================================================
// INTERRUPT — FLOW SENSOR
// ============================================================
void IRAM_ATTR onFlowPulse() {
  pulseCount++;
}

// ============================================================
// FUNGSI: BACA HC-SR04 (median 5x untuk akurasi)
// ============================================================
float bacaJarakSingle() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(4);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  long durasi = pulseIn(PIN_ECHO, HIGH, 25000);
  if (durasi == 0) return 99.0;
  float jarak = (durasi * 0.0343f) / 2.0f;
  if (jarak < 1.0f || jarak > 400.0f) return 99.0;
  return jarak;
}

float bacaJarak() {
  float hasil[5];
  for (int i = 0; i < 5; i++) {
    hasil[i] = bacaJarakSingle();
    delay(15);
  }
  // Bubble sort sederhana
  for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4 - i; j++) {
```

SWIFT_IoT_FINAL_kel1_FIX.ino

```
// ============================================================
// FUNGSI: BACA HC-SR04 (median 5x untuk akurasi)
// ============================================================
float bacaJarakSingle() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(4);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
  long durasi = pulseIn(PIN_ECHO, HIGH, 25000);
  if (durasi == 0) return 99.0;
  float jarak = (durasi * 0.0343f) / 2.0f;
  if (jarak < 1.0f || jarak > 400.0f) return 99.0;
  return jarak;
}

float bacaJarak() {
  float hasil[5];
  for (int i = 0; i < 5; i++) {
    hasil[i] = bacaJarakSingle();
    delay(15);
  }
  // Bubble sort sederhana
  for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4 - i; j++) {
      if (hasil[j] > hasil[j + 1]) {
        float tmp    = hasil[j];
        hasil[j]     = hasil[j + 1];
        hasil[j + 1] = tmp;
      }
    }
  }
  return hasil[2]; // median
}

// ============================================================
// FUNGSI: BACA TURBIDITY — sensor inline/clamp
//   Karakteristik: ~2.9V = air jernih (0 NTU)
//                  ~0.5V = air sangat keruh (100 NTU)
//   PENTING: cek tegangan A0 di Serial Monitor saat pertama
//            kali setup. Jika > 3.0V pasang voltage divider
//            1kΩ + 2kΩ di jalur GPIO34 agar ESP32 tidak rusak.
// ============================================================
float bacaTurbidity() {
  long total = 0;
  for (int i = 0; i < 15; i++) {
    total += analogRead(PIN_TURBIDITY);
    delay(3);
  }
  float adcRaw = total / 15.0f;
  float volt   = adcRaw * (3.3f / 4095.0f);

  // Rumus kalibrasi linear untuk sensor inline/clamp
  float ntu = (2.9f - volt) / 2.4f * 100.0f;
  if (ntu < 0.0f)   ntu = 0.0f;
  if (ntu > 100.0f) ntu = 100.0f;
  return ntu;
}
```

```

// ============================================================
// FUNGSI: BUZZER
// ============================================================
void buzzer(int kali, int durasiMs = 200) {
  for (int i = 0; i < kali; i++) {
    digitalWrite(PIN_BUZZER, HIGH);
    delay(durasiMs);
    digitalWrite(PIN_BUZZER, LOW);
    delay(durasiMs);
  }
}

void buzzerTerus() {
  // Non-blocking, panggil berulang di loop saat error
  static unsigned long lastBip = 0;
  if (millis() - lastBip > 800) {
    digitalWrite(PIN_BUZZER, HIGH);
    delay(100);
    digitalWrite(PIN_BUZZER, LOW);
    lastBip = millis();
  }
}

// ============================================================
// FUNGSI: MATIKAN SEMUA AKTUATOR (safe state)
// ============================================================
void matikanSemua() {
  digitalWrite(PIN_POMPA1,    RELAY_OFF);
  digitalWrite(PIN_POMPA2,    RELAY_OFF);
  digitalWrite(PIN_VALVE,     RELAY_OFF);
  digitalWrite(PIN_INDIKATOR, RELAY_OFF);
  digitalWrite(PIN_BUZZER,    LOW);
}

// ============================================================
// FUNGSI: SETUP WiFi
// ============================================================
void setupWiFi() {
  Serial.print("Menghubungkan ke WiFi: ");
  Serial.println(WIFI_SSID);
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Connecting WiFi");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  int percobaan = 0;
  while (WiFi.status() != WL_CONNECTED && percobaan < 40) {
    delay(500);
    Serial.print(".");
    percobaan++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi TERHUBUNG! IP: " + WiFi.localIP().toString());
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("WiFi OK!");
    lcd.setCursor(0, 1); lcd.print(WiFi.localIP().toString());
    delay(2000);
```

SWIFT_IoT_FINAL_kel1_FIX.ino

```
// ============================================================
// FUNGSI: SETUP WiFi
// ============================================================
void setupWiFi() {
  Serial.print("Menghubungkan ke WiFi: ");
  Serial.println(WIFI_SSID);
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Connecting WiFi");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  int percobaan = 0;
  while (WiFi.status() != WL_CONNECTED && percobaan < 40) {
    delay(500);
    Serial.print(".");
    percobaan++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi TERHUBUNG! IP: " + WiFi.localIP().toString());
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("WiFi OK!");
    lcd.setCursor(0, 1); lcd.print(WiFi.localIP().toString());
    delay(2000);
  } else {
    Serial.println("\nWiFi GAGAL – sistem jalan oFfline.");
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("WiFi GAGAL");
    lcd.setCursor(0, 1); lcd.print("Mode OFfline");
    delay(2000);
  }
}

// ============================================================
// FUNGSI: RECONNECT MQTT (non-blocking, 5 detik cooldown)
// ============================================================
void reconnectMQTT() {
  static unsigned long waktuCobaMQTT = 0;
  if (millis() - waktuCobaMQTT < 5000) return;
  waktuCobaMQTT = millis();

  if (WiFi.status() != WL_CONNECTED) return;

  Serial.print("MQTT connecting... ");
  if (mqtt.connect(MQTT_CLIENT, MQTT_USER, MQTT_PASS_AUTH)) {
    Serial.println("OK!");
    mqtt.subscribe(MQTT_TOPIC_SUB);
  } else {
    Serial.println("Gagal (rc=" + String(mqtt.state()) + ")");
  }
}

// ============================================================
// FUNGSI: CALLBACK MQTT – perintah dari dashboard
// ============================================================
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String perintah = "";
  for (unsigned int i = 0; i < length; i++) {
    perintah += (char)payload[i];
  }
  perintah.trim();
  Serial.println("[MQTT CMD] " + perintah);
```

SWIFT_IoT_FINAL_kel1_FIX.ino

```
// ============================================================
// FUNGSI: CALLBACK MQTT – perintah dari dashboard
// ============================================================
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String perintah = "";
  for (unsigned int i = 0; i < length; i++) {
    perintah += (char)payload[i];
  }
  perintah.trim();
  Serial.println("[MQTT CMD] " + perintah);

  if      (perintah == "POMPA1_ON")  { digitalWrite(PIN_POMPA1, RELAY_ON);  }
  else if (perintah == "POMPA1_OFF") { digitalWrite(PIN_POMPA1, RELAY_OFF); }
  else if (perintah == "POMPA2_ON")  { digitalWrite(PIN_POMPA2, RELAY_ON);  digitalWrite(PIN_INDIKATOR, RELAY_ON);  }
  else if (perintah == "POMPA2_OFF") { digitalWrite(PIN_POMPA2, RELAY_OFF); digitalWrite(PIN_INDIKATOR, RELAY_OFF); }
  else if (perintah == "VALVE_ON")   { digitalWrite(PIN_VALVE,  RELAY_ON);  }
  else if (perintah == "VALVE_OFF")  { digitalWrite(PIN_VALVE,  RELAY_OFF); }
  else if (perintah == "RESET")      { ESP.restart(); }
  else if (perintah == "FLUSH_NOW") {
    if (!sedangFlushing) {
      sedangFlushing  = true;
      waktuFlushMulai = millis();
      Serial.println(">> Manual flush dimulai!");
      buzzer(3, 200);
    }
  }
}

// ============================================================
// FUNGSI: KIRIM DATA MQTT (JSON)
// ============================================================
void kirimMQTT() {
  if (!mqtt.connected()) return;

  String json = "{";
  json += "\"device\":\""    + String(MQTT_CLIENT)                        + "\",";
  json += "\"flow_rate\":"   + String(flowRate,      2)                   + ",";
  json += "\"turbidity\":"   + String(turbidityNTU,  1)                   + ",";
  json += "\"distance\":"    + String(jarakCm,       1)                   + ",";
  json += "\"level_pct\":"   + String(levelPersen,   1)                   + ",";
  json += "\"status\":\""    + statusSistem                               + "\",";
  json += "\"message\":\""   + pesanNotif                                 + "\",";
  json += "\"pump1\":"       + String(digitalRead(PIN_POMPA1)    == RELAY_ON ? 1 : 0) + ",";
  json += "\"pump2\":"       + String(digitalRead(PIN_POMPA2)    == RELAY_ON ? 1 : 0) + ",";
  json += "\"valve\":"       + String(digitalRead(PIN_VALVE)     == RELAY_ON ? 1 : 0) + ",";
  json += "\"flushing\":"    + String(sedangFlushing ? 1 : 0);
  json += "}";

  mqtt.publish(MQTT_TOPIC_PUB, json.c_str());
  Serial.println("[MQTT SEND] " + json);
}

// ============================================================
// FUNGSI: UPDATE LCD 16x2
// ============================================================
void updateLCD() {
  lcd.clear();

  // Baris 1: Turbidity + Flow
  String baris1 = "T:";
```

SWIFT_IoT_FINAL_kel1_FIX.ino

```
310
311  // ============================================================
312  // FUNGSI: UPDATE LCD 16x2
313  // ============================================================
314  void updateLCD() {
315    lcd.clear();
316
317    // Baris 1: Turbidity + Flow
318    String baris1 = "T:";
319    baris1 += String((int)turbidityNTU);
320    baris1 += "N F:";
321    baris1 += String(flowRate, 1);
322    while (baris1.length() < 16) baris1 += " ";
323    lcd.setCursor(0, 0);
324    lcd.print(baris1.substring(0, 16));
325
326    // Baris 2: Status + sisa waktu flushing atau jarak
327    String baris2 = "";
328    if (jarakCm >= JARAK_ERROR) {
329      baris2 = "SENSOR ERR!     ";
330    } else if (sedangFlushing) {
331      unsigned long sisa = (DURASI_FLUSHING - (millis() - waktuFlushMulai)) / 1000;
332      baris2 = "FLUSH:" + String(sisa) + "s        ";
333    } else {
334      baris2 = statusSistem + " ";
335      baris2 += String((int)jarakCm) + "cm ";
336      baris2 += String((int)levelPersen) + "%";
337    }
338    while (baris2.length() < 16) baris2 += " ";
339    lcd.setCursor(0, 1);
340    lcd.print(baris2.substring(0, 16));
341  }
342
343  // ============================================================
344  // FUNGSI: LOG SERIAL MONITOR
345  // ============================================================
346  void logSerial() {
347    Serial.println("==============================");
348    Serial.println(" STATUS   : " + statusSistem);
349    Serial.println(" PESAN    : " + pesanNotif);
350    Serial.println(" TURB     : " + String(turbidityNTU, 1) + " NTU");
351    Serial.println(" FLOW     : " + String(flowRate,     2) + " L/min");
352    Serial.println(" JARAK    : " + String(jarakCm,      1) + " cm");
353    Serial.println(" LEVEL    : " + String(levelPersen,  1) + " %");
354    Serial.println(" FLUSHING : " + String(sedangFlushing
355                     ? "YA (" + String((millis() - waktuFlushMulai) / 1000) + "s)"
356                     : "TIDAK"));
357    Serial.println(" POMPA1   : " + String(digitalRead(PIN_POMPA1)    == RELAY_ON ? "ON"   : "OFF"));
358    Serial.println(" POMPA2   : " + String(digitalRead(PIN_POMPA2)    == RELAY_ON ? "ON"   : "OFF"));
359    Serial.println(" VALVE    : " + String(digitalRead(PIN_VALVE)     == RELAY_ON ? "BUKA" : "TUTUP"));
360    Serial.println(" MQTT     : " + String(mqtt.connected() ? "TERHUBUNG" : "PUTUS"));
361    Serial.println("==============================\n");
362  }
363
364  // ============================================================
365  // SETUP
366  // ============================================================
367  void setup() {
368    Serial.begin(115200);
369    delay(500);
```

Output Serial Monitor X

SWIFT_IoT_FINAL_kel1_FIX.ino

```
363
364  // ============================================================
365  // SETUP
366  // ============================================================
367  void setup() {
368    Serial.begin(115200);
369    delay(500);
370    Serial.println("\n==============================");
371    Serial.println("  SWIFT IoT – Kelompok 1");
372    Serial.println("==============================\n");
373
374    // Init relay ke OFF SEBELUM pinMode untuk cegah glitch saat boot
375    pinMode(PIN_POMPA1,    OUTPUT); digitalWrite(PIN_POMPA1,    RELAY_OFF);
376    pinMode(PIN_POMPA2,    OUTPUT); digitalWrite(PIN_POMPA2,    RELAY_OFF);
377    pinMode(PIN_VALVE,     OUTPUT); digitalWrite(PIN_VALVE,     RELAY_OFF);
378    pinMode(PIN_INDIKATOR, OUTPUT); digitalWrite(PIN_INDIKATOR, RELAY_OFF);
379    pinMode(PIN_BUZZER,    OUTPUT); digitalWrite(PIN_BUZZER,    LOW);
380
381    // Sensor
382    pinMode(PIN_FLOW, INPUT_PULLUP);
383    pinMode(PIN_TRIG, OUTPUT);      digitalWrite(PIN_TRIG, LOW);
384    pinMode(PIN_ECHO, INPUT);
385    // PIN_TURBIDITY (GPIO34) adalah input-only pin – JANGAN pinMode, biarkan ADC handle sendiri
386
387    // ADC resolusi & atenuasi – WAJIB diset sebelum analogRead pertama
388    analogSetWidth(12);            // 12-bit (0–4095)
389    analogSetAttenuation(ADC_11db); // range 0–3.3V (full range)
390    delay(100);                    // beri waktu ADC stabil setelah konfigurasi
391
392    // Interrupt flow sensor – FALLING sesuai YF-S401
393    attachInterrupt(digitalPinToInterrupt(PIN_FLOW), onFlowPulse, FALLING);
394
395    // LCD
396    Wire.begin(21, 22);
397    lcd.init();
398    lcd.backlight();
399    lcd.setCursor(0, 0); lcd.print("SWIFT IoT v5.0");
400    lcd.setCursor(0, 1); lcd.print("Kelompok 1");
401    delay(2000);
402
403    buzzer(2, 100);
404
405    // Test HC-SR04
406    Serial.println("Test HC-SR04...");
407    float testJarak = bacaJarak();
408    if (testJarak < JARAK_ERROR) {
409      Serial.println("HC-SR04 OK! Jarak: " + String(testJarak, 1) + " cm");
410    } else {
411      Serial.println("PERINGATAN: HC-SR04 tidak terdeteksi – cek voltage divider!");
412      lcd.clear();
413      lcd.setCursor(0, 0); lcd.print("HC-SR04 ERROR!");
414      lcd.setCursor(0, 1); lcd.print("Cek voltage div");
415      delay(3000);
416    }
417
418    // Test Turbidity – cek tegangan, WARNING SAJA tidak menghentikan boot
419    Serial.println("Test Turbidity...");
420    long adcSum = 0;
421    for (int i = 0; i < 3; i++) { adcSum += analogRead(PIN_TURBIDITY); delay(20); }
422    float voltAwal = (adcSum / 3.0f) * (3.3f / 4095.0f);
```

Output Serial Monitor X

SWIFT_IoT_FINAL_kel1_FIX.ino

```
  // Test Turbidity — cek tegangan, WARNING SAJA tidak menghentikan boot
  Serial.println("Test Turbidity...");
  long adcSum = 0;
  for (int i = 0; i < 3; i++) { adcSum += analogRead(PIN_TURBIDITY); delay(20); }
  float voltAwal = (adcSum / 3.0f) * (3.3f / 4095.0f);
  Serial.print("[TURB] Tegangan awal: ");
  Serial.print(voltAwal, 3);
  Serial.println(" V");
  if (voltAwal > 3.0f) {
    Serial.println("[TURB] PERINGATAN: Tegangan > 3.0V! Pasang voltage divider 1k+2k di A0 ke GPIO34!");
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("TURB >3V WARNING");
    lcd.setCursor(0, 1); lcd.print("Lanjut boot...");
    delay(2000);
  }
  float testTurb = bacaTurbidity();
  Serial.println("Turbidity awal: " + String(testTurb, 1) + " NTU");

  // WiFi
  setupWiFi();

  // MQTT
  mqtt.setServer(MQTT_SERVER, MQTT_PORT);
  mqtt.setCallback(mqttCallback);
  mqtt.setKeepAlive(30);

  waktuFlowTerakhir = millis();
  millisLalu        = millis();
  sistemSiap        = true;

  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Sistem SIAP!");
  lcd.setCursor(0, 1); lcd.print("Monitoring ON");
  Serial.println("Setup selesai! Mulai monitoring...\n");
  delay(1000);
}

// ============================================================
// LOOP UTAMA
// ============================================================
void loop() {
  // MQTT keep-alive
  if (!mqtt.connected()) reconnectMQTT();
  mqtt.loop();

  unsigned long sekarang = millis();
  if (sekarang - millisLalu < INTERVAL_BACA) return;
  millisLalu = sekarang;

  // ── TAHAP 1: BACA SENSOR ──────────────────────────
  // Flow rate — hitung dari pulse sejak interval terakhir
  noInterrupts();
  uint32_t pulse = pulseCount;
  pulseCount     = 0;
  interrupts();
  // YF-S401: ~7.5 pulse/detik per 1 L/menit, interval 2 detik
  flowRate = (pulse / 7.5f) / (INTERVAL_BACA / 1000.0f);
  if (flowRate > FLOW_MIN) waktuFlowTerakhir = sekarang;

  turbidityNTU = bacaTurbidity();
```

SWIFT_IoT_FINAL_kel1_FIX.ino

```
// LOOP UTAMA
// ============================================================
void loop() {
  // MQTT keep-alive
  if (!mqtt.connected()) reconnectMQTT();
  mqtt.loop();

  unsigned long sekarang = millis();
  if (sekarang - millisLalu < INTERVAL_BACA) return;
  millisLalu = sekarang;

  // ── TAHAP 1: BACA SENSOR ──────────────────────────
  // Flow rate — hitung dari pulse sejak interval terakhir
  noInterrupts();
  uint32_t pulse = pulseCount;
  pulseCount     = 0;
  interrupts();
  // YF-S401: ~7.5 pulse/detik per 1 L/menit, interval 2 detik
  flowRate = (pulse / 7.5f) / (INTERVAL_BACA / 1000.0f);
  if (flowRate > FLOW_MIN) waktuFlowTerakhir = sekarang;

  turbidityNTU = bacaTurbidity();
  jarakCm      = bacaJarak();

  // Hitung persentase level air
  if (jarakCm >= JARAK_ERROR) {
    levelPersen = -1; // error
  } else {
    float range = JARAK_KOSONG - JARAK_PENUH;
    levelPersen = ((JARAK_KOSONG - jarakCm) / range) * 100.0f;
    levelPersen = constrain(levelPersen, 0.0f, 100.0f);
  }

  // ── TAHAP 2: TENTUKAN STATUS SISTEM ──────────────
  if (jarakCm >= JARAK_ERROR) {
    statusSistem = "ERROR";
    pesanNotif   = "Sensor HC-SR04 Off";
  }
  else if (turbidityNTU > 95.0) {
    statusSistem = "BAHAYA";
    pesanNotif   = "Air Sangat Keruh!";
  }
  else if (turbidityNTU >= 90.0 && turbidityNTU <= 95.0) {
    statusSistem = "WASPADA";
    pesanNotif   = "Air Mulai Keruh";
  }
  else {
    statusSistem = "AMAN";
    if      (jarakCm > JARAK_KOSONG)  pesanNotif = "Isi Air (Volume <25%)";
    else if (jarakCm <= JARAK_PENUH)  pesanNotif = "Wadah Penuh";
    else                              pesanNotif = "Air Mengalir Normal";
  }

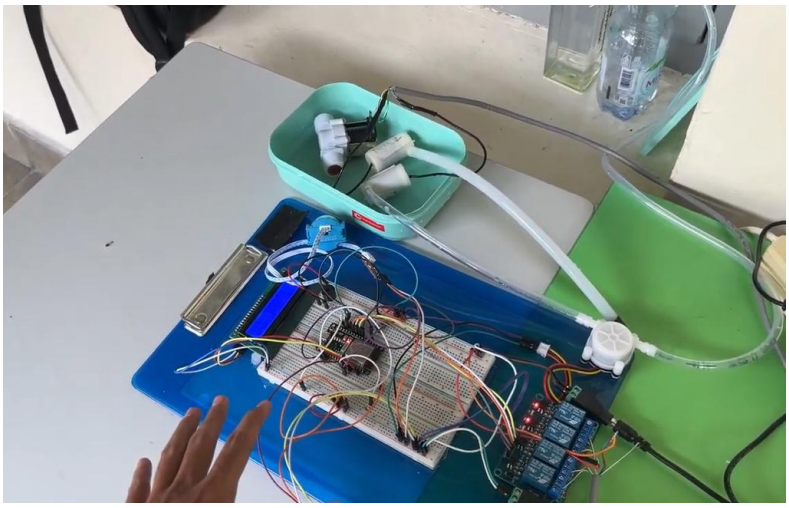
  // ── TAHAP 3: KONTROL AKTUATOR ─────────────────────

  // --- PRIORITAS 0: Sensor error — matikan semua ---
  if (statusSistem == "ERROR") {
    matikanSemua();
    sedangFlushing = false;
    buzzerTerus();
```

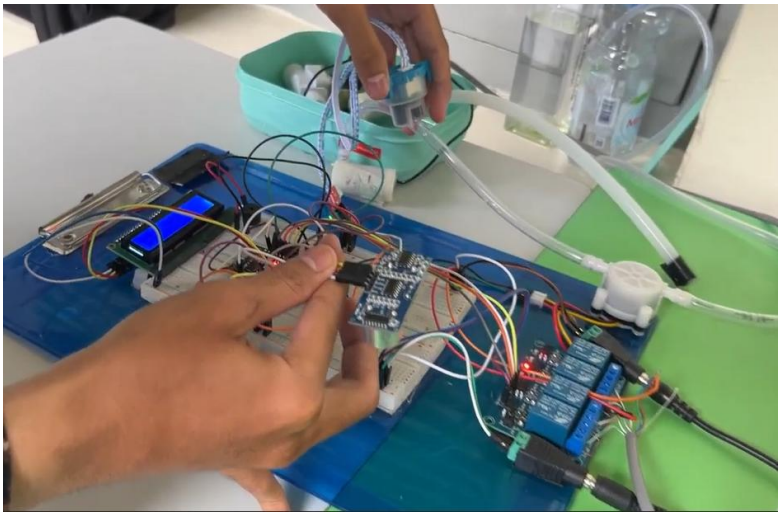


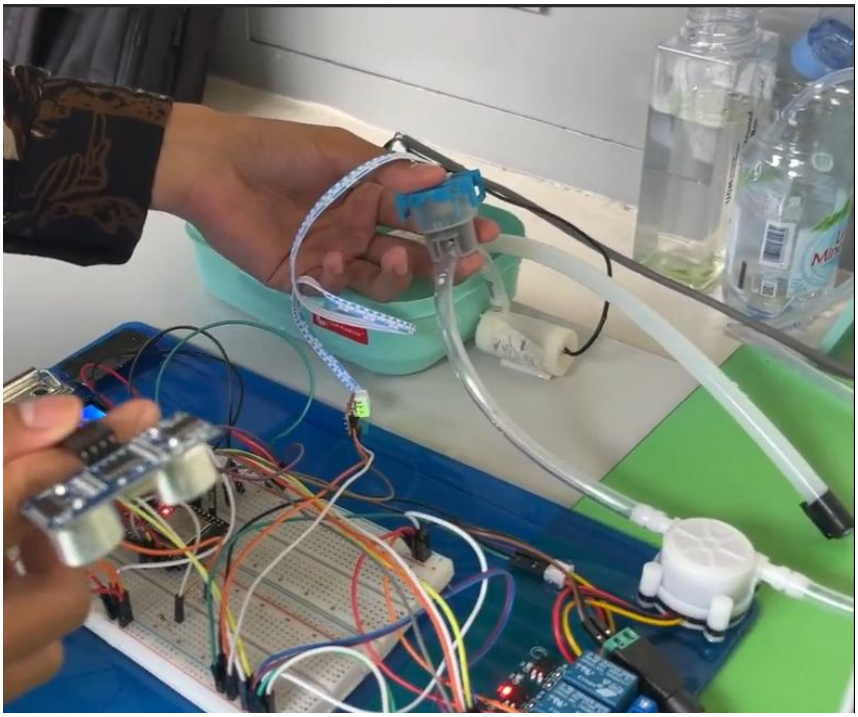
Gambar Kode Final SWIFT

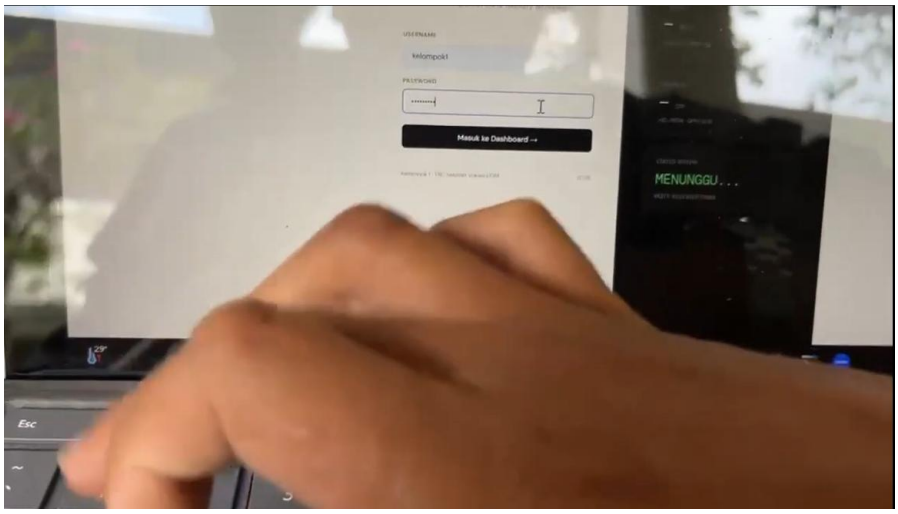
B. Gambar pendukung



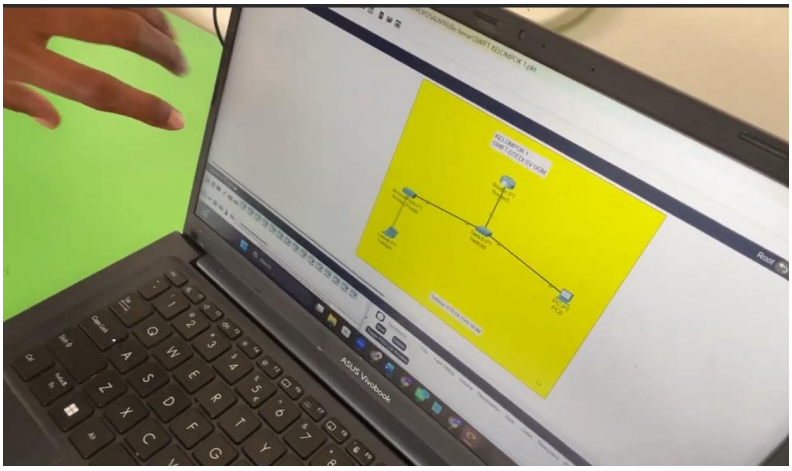


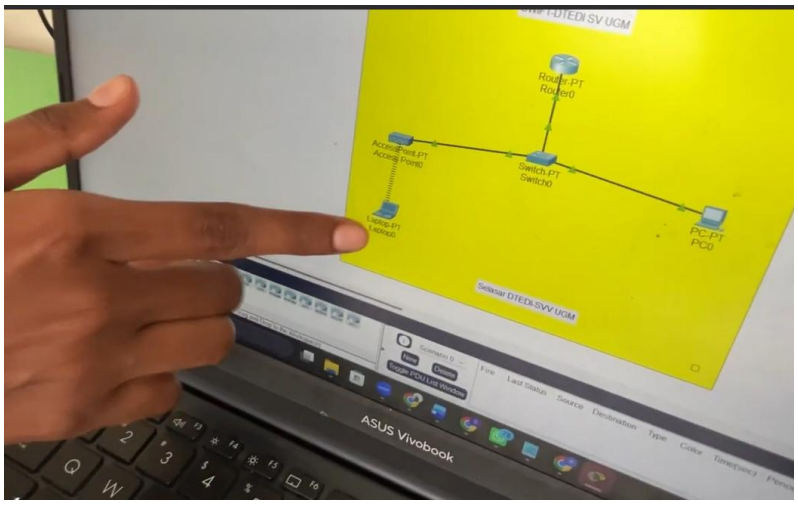





kelompok1
Masuk ke Dashboard →
MENUNGGU...
Esc



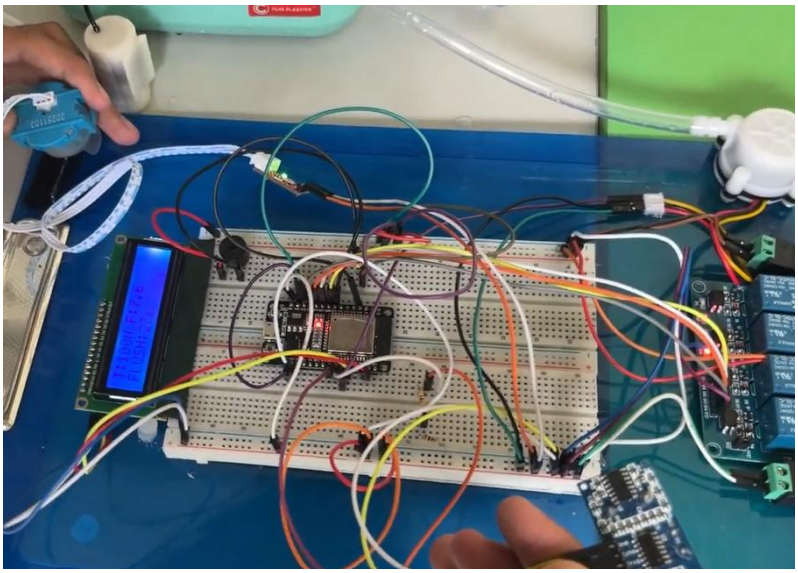

ASUS Vivobook

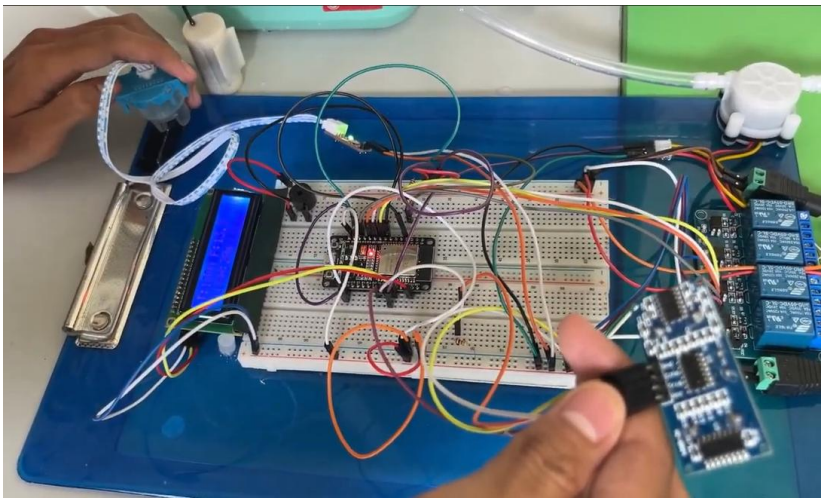

Router-PT
Router0
Switch-PT
Switch0
PC-PT
PC0
ASUS Vivobook

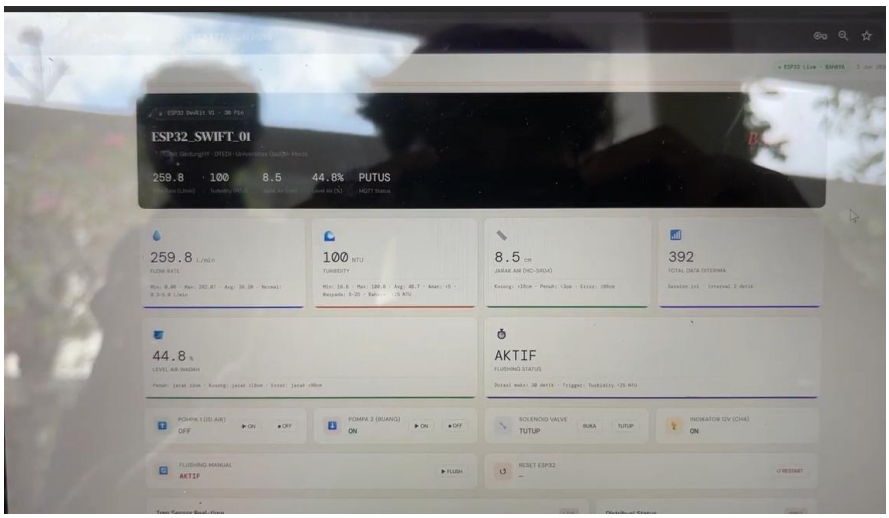


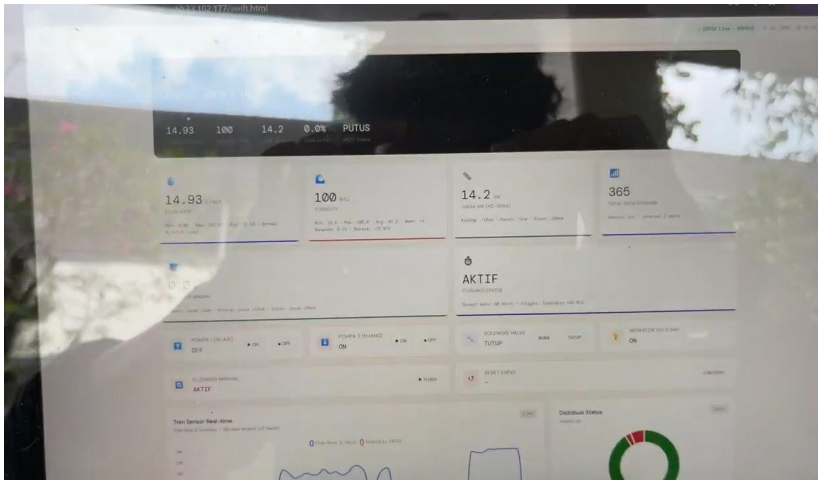
Gambar Pendukung Demo IoT 13 Juni 2026

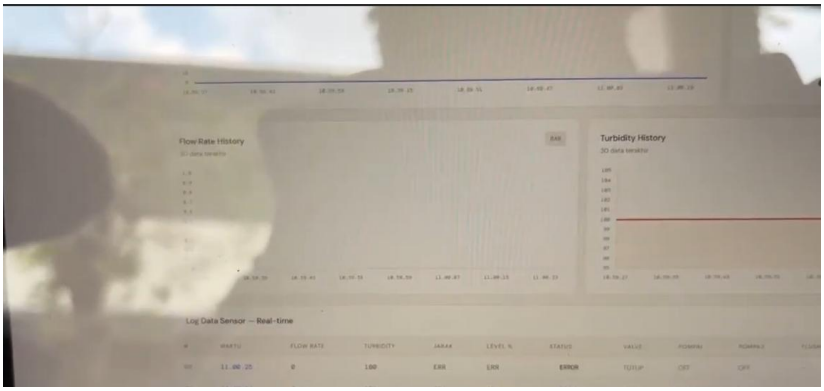
C. Hasil pengukuran

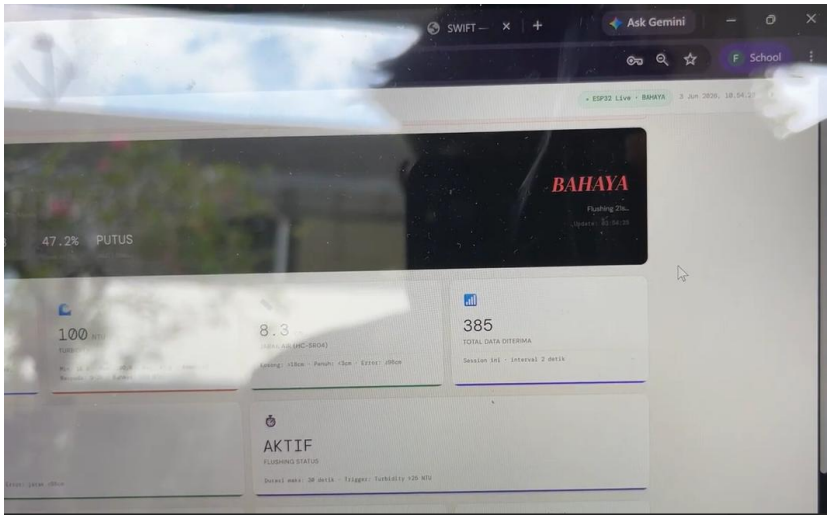






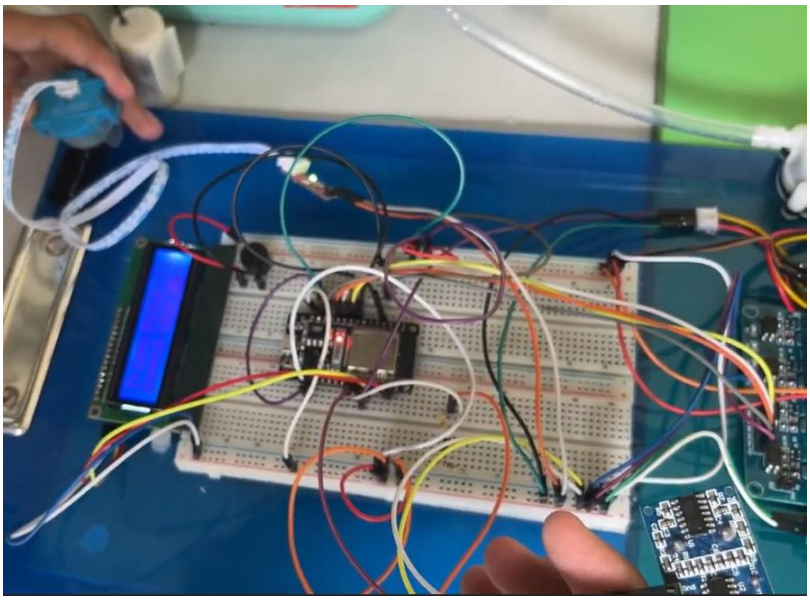




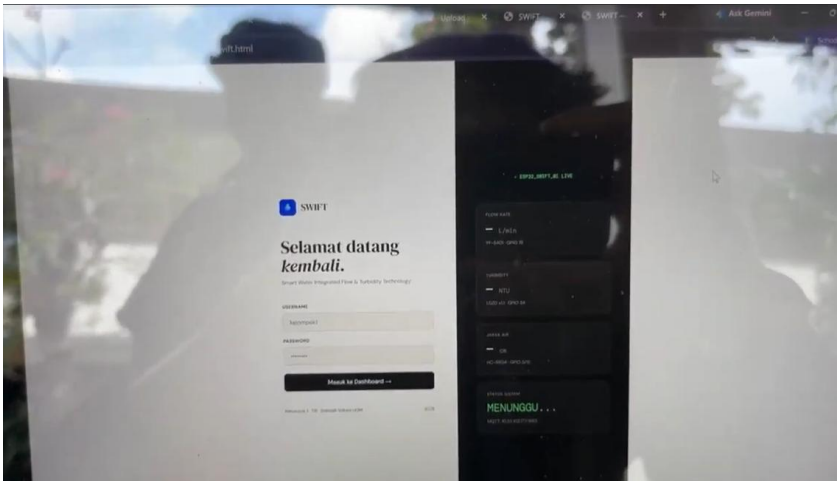


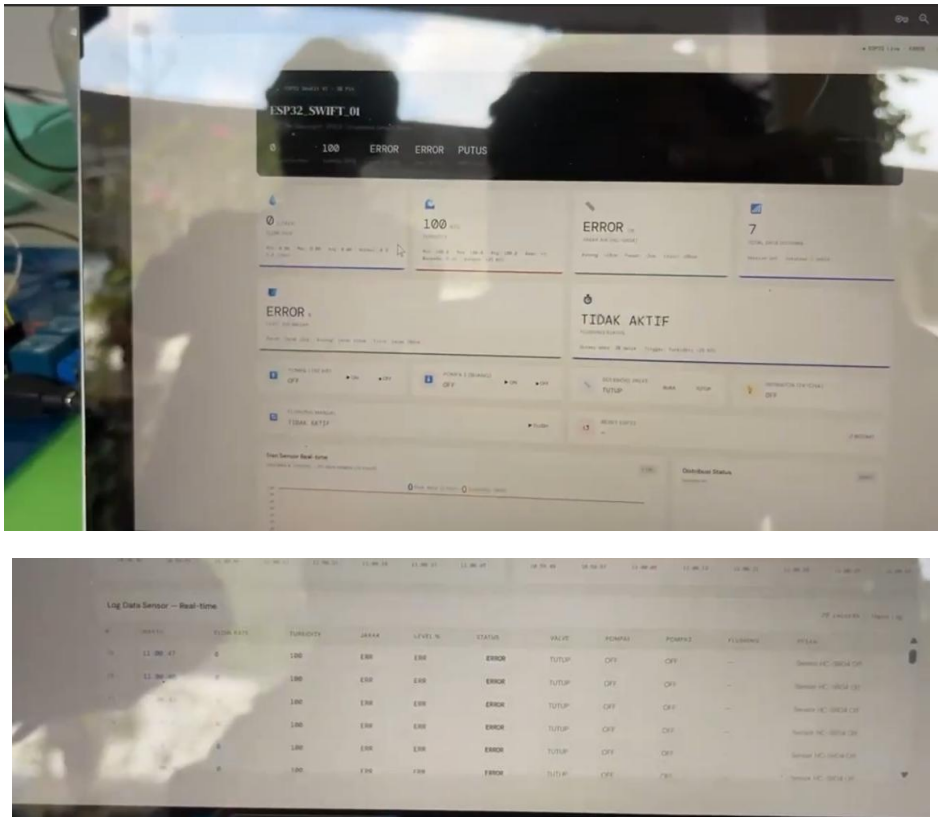
Gambar Hasil Pengukuran pada Demo Iot 29 Mei 2026

D. Dokumentasi foto alat atau tampilan antarmuka



Gambar Dokumentasi Alat Iot Final





Gambar Hasil Data Log pada Dashboard SWIFT

Link Demo: https://drive.google.com/drive/folders/19752DwoTnUXFi7SS-ZMlOaT4sFj5VMWj?usp=drive_link